A.A. 2019/2020

INTELLIGENT SYSTEMS

Prof. Michela Milano

Laura Gruppioni

Sommario

1 – PLANNING

Automated Planning is an important problem-solving activity which consists in synthesizing a sequence of actions performed by an agent that leads from an initial state of the world to a given target state (goal).
Planning is an action we do every time we want to do something.
Suppose that we have to move a phone on the desk: we can use the action "move" ore we can use the actions "pick it up" and "put it down". What action the KB will decide? The more complex are the actions, the more complex will be the code application.
Planning and executing are the main things to think about when we are dealing with AI, so if we think about execution:

- If I have an agent that can pick up and put down, I've to use the two actions.
- If I have a smarter agent that is able to use the move action, we can use the move action.

Planning is used as an application or as a support activity for diagnosis, scheduling and robotics.

An **automated planner** is an intelligent agent that operates in a certain domain **described by:**

1. A **representation** of the **initial state**
2. A **representation** of a **goal**
3. A **formal description** of the **executable actions** (also called **operators**)

It dynamically summarizes the plan of actions needed to reach the goal from the initial state.

A planner relies on a **formal description of the executable actions**. It is called **Domain Theory**.
Each action is identified by a name and declaratively modelled through:

- **Preconditions:** conditions which must hold to ensure that the action can be executed
- **Postconditions:** represent the effects of the action on the world

Often the Domain Theory consists of **operators containing variables that define classes of actions**. A different instantiation of the variables corresponds to a different action. The variable should be ground. In general, we have something that is deleted and something that is added.

IN AI THERE ARE MANY PLANNING TECHNIQUES:

1. **DEDUCTIVE PLANNING/PLANNING AS SEARCH/LINEAR PLANNING (Foundations of AI)**: The problem of Linear Planning is that we have to order the whole sequence of actions. Furthermore, when we have sub goals that can interact among themselves, that can be a problem. Linear planners are **search algorithms that explore the state space**. The plan is a linear sequence of actions to achieve the goals.
2. **NON-LINEAR PLANNING:** search algorithms that generate a plan as a **search problem in the space of plans**. In the search tree **each node is a partial plan** and operators are plan refinement operations. In some cases, there's not need to order actions: if you have to wear socks and shoes, it's ok if you put before the right sock and the left sock or vice versa.
   A non-linear generative planner assumes that the initial state is fully known (**CWA[1]). Least Commitment planning** never imposes more restrictions than those that are strictly necessary. It avoids making decisions when they are not required and making many backtrackings.
3. **PARTIAL ORDER PLANNING (POP)**
4. **HIERARCHICAL PLANNING**: First of all, when we plan a trip, we have to focus on the final goal, like from bologna to NY, then we go down and we add pieces of the path like casa → bologna and airport → NY and so and so on.
5. **CONDITIONAL PLANNING**: Our planning can fail for many reasons, so we need a plan for every possible contingency, it is a prediction of what can go wrong. What a semi-decidable problem is? If the plan doesn't exist, there's not an algorithm that can tell us "no".
6. **GRAPH-BASED PLANNING**
7. **PLANNING FOR ROBOTICS PATHS**
8. **PLANNING AS EMERGENT BEHAVIOR: SWARM INTELLIGENCE**

[1] everything that is not explicitly stated in the initial state is considered as false

1.1 – NON-LINEAR PLANNING

A non-linear plan is represented as:

- **A set of actions** (instances of operators)
- **A (not exhaustive) set of orderings between actions**
- **A set of "causal links"**

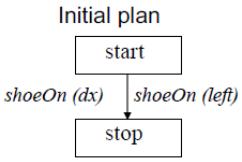
Initial plan: it is an empty plan with two fake actions:

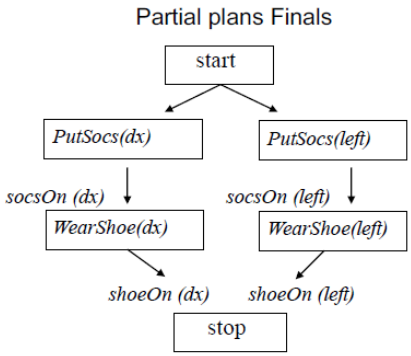
- **Start**: no preconditions. Its effects match the initial state.
- **Stop**: no effects. Its pre-conditions match the goal. (Ordering: start < stop)

At each step either the set of operators/orderings/causal links is increased until all goals are met. Non required orderings are not posted. A **solution** is a **set of partially specified and partially ordered operators**. To obtain a real plan, the partial order should be linearized in one of the possible total orders **(linearization operation)**. Non-linear plans are not really executed.

Example:
Goal: shoeOn (dX), shoeOn (sX)
Actions:

- **WearShoe(Foot)**
  PRECOND: socsOn (Foot)
  EFFECT: shoeOn (Foot)
- **PutSocs (Foot)**
  PRECOND: ¬ socsOn (Foot)
  EFFECT: shoeOn (Foot)





1.2 – PARTIAL ORDER PLANNING (POP)

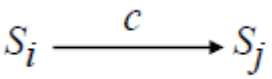
1.2.1 -INTUITIVE ALGORITHM

While (plan not complete) do {

- **select** an **action SN** that has a **precondition not satisfied**;
- **select** an **action S** (new or already in the plan) that has **C among its effects**;
- **add the order constraint S <SN;**
- if S is a **new action** add the **constraint Start <S <Stop**;
- **add the causal link <S, SN, C>;**
- **solve any threat** on causal links

} **end**

CAUSAL LINKS



- They're a triple that consists of two **operators Si, Sj** and a **subgoal c.**
- **c should be precondition of Sj and effect of Si.**
- They store the causal relations between actions: a causal link **traces why a given operator has been introduced** in the plan.
- They help tackling the problem of **interacting goals**

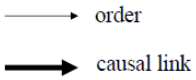
In case of failure if choice points exist, the algorithm backtracks and it explores alternatives.
The POP proceeds backward. We want to keep track of these causal links because thanks to this we can understand if a causal link is threatened or not during the planning.
Intuitively, in the algorithm we start with an empty plan (where we have two fake actions: start and stop). Since start has no preconditions and end has no postconditions, we have to add a constraint between start and stop and then we have to provide and add a causal link that reminds us why we have inserted it in the plan.
In general, we call these **preconditions not yet satisfied** as **open conditions**.

DEMOTION AND PROMOTION

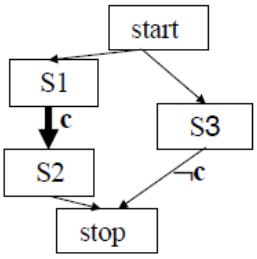
THREAT: an action S3 is a **threat for a causal link** <S1, S2, c> if it has an **effect that negates c and no ordering constraint exists that prevents S3 to be performed between S1 and S2**.

Possible solutions:

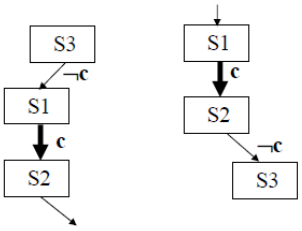
- **DEMOTION**: The constraint S3 <S1 is imposed
- **PROMOTION**: The constraint S2 <S3 is imposed



Example: <s1, s2, c> has a problem because we need to execute s1 before s2, that's sure. What about the linearization? Any linearization is a good plan, so the linearization s1, s2, s3 is good. Also s3, s1, s2 is a good plan. **s1, s3, s2 instead, is not good and there's any constraint to prevent this bad plan:** that's because **if I execute s3, I delete some information that's useful for s2.**



Now, suppose that on another plan we have s3 and the effect of s3 is deleting C. If we leave this plan as it is, we have any possibility, but if we choose s3 between s1 and s2 we've a problem: s1 has an effect on the condition c, s3 deletes c and s2 cannot be executed because the precondition c is not satisfied. We can do two things: **we can insert an ordering constraint that say to execute s3 before s1 (demotion) or we can use promotion (s2 before s3).**



Suppose that both demotion and promotion cannot be imposed because these constraints are incompatible with the other constraints of the plan: in this case we can use the **WHITE KNIGHT.**

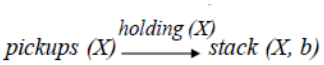
1.2.2 – THE POP ALGORITHM

```
function POP (initialGoal Operators) returns plan
        plan: = INITIAL_PLAN (start, stop, initialGoal)
        loop
                if SOLUTION (plan) then return plan;
                SN, C: = SELECT_SUBGOAL (plan); → the action is SN and the condition still open is called C.
                CHOOSE_OPERATOR (plan, operators, SN, C);
                RESOLVE_THREATS (plan)
        End

function SELECT_SUBGOAL (plan)
        select SN from STEPS (plan) with unsolved precondition C; → this looks for an action already in the plan
        return SN, C

procedure CHOOSE_OPERATOR (plan, ops, SN, C)
        pick an S with effect C from ops or from STEPS (plan);
        if S does not exist then fail;
        add the causal link <S, SN, C>
        add the ordering constraint S <SN
        if S is a new action added to the plan
        then add S to STEPS(plan)
                add the constraint Start <S <Stop

procedure SOLVE_THREAT (plan)
        for each action S that threats a causal link between Si and Sj,
        choose either
                demotion: add the constraint S < Si
                promotion: add the constraint Sj <S
                if NOT_CONSISTENT (plan) then fail → procedure backtracking
```

1.2.3 - MODAL TRUTH CRITERION (MTC)

Promotion and demotion are not enough: we have 2 other ways to solve threats. A planner is complete if it always finds a solution if a solution exists. The original paper that described the POP has called all possible ways to refine a plan and it is the modal truth criterion. **The Modal Truth Criterion is a construction process that guarantees planner completeness.**
Partial Order Planning algorithm interleaves goal achievement steps with threat protection steps. The MTC provides **FIVE PLAN REFINEMENT METHODS** (one for the open goal achievement and 4 for threat protection) that ensure the completeness of the planner:

1. **ESTABLISHMENT**: open goal achievement by means of:
    a. a **new action** to be **inserted** in the plan
    b. an **ordering constraint** with Action already in the plan
    c. a **variable assignment**
2. **PROMOTION**: ordering constraint that imposes the **threatening action before** the 1st of the causal link
3. **DEMOTION**: ordering constraint that imposes the **threatening action after** the 2nd of the causal link
4. **WHITE KNIGHT**: **insert a new operator or use one already in the plan between S1 and S3** such that it establishes the precondition of S3 threatened by S1. So, if no order is possible, the white knight basically enforces it again. It re-establishes again the precondition. This is not really efficient in terms of length of the plan because it adds a new action, but this condition enforces the completeness of the plan.
5. **SEPARATION**: **insert non codesignation constraints** between the variables of the negative effect and the threatened precondition so to **avoid unification.** This is useful when variables have not yet been instantiated. We can use it when we have different constraint among the variables.

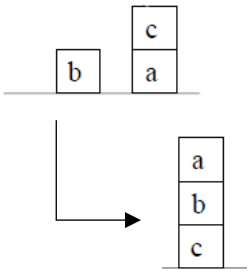


In the figure, any threat imposed by stack(Y,c) can be solved by imposing X ≠ Y.

In general, it is always preferable to apply firstly promotion and demotion and then the white knight, especially if it adds new actions. **Non-linear planners can generate very inefficient plans, even if correct**, and white knights are one of the reasons of this inefficiency: they have a high number of actions.
Planning is a semi-decidable problem, so if the plan exists, ok, but if the plan doesn't exist, the planner can work indefinitively.
In domains of increasing complexity, it is hard to use a correct and complete planner, which is efficient and domain independent. Often, we use ad-hoc methods.
In case we have domain knowledge, so we know for example that we are building a trip or a house, **we can apply heuristics to simplify the task for the planner** and this is something that can be done for **specific domain**. **Using this approach, we lose generality**, of course. If we want to keep generality, we can inject in the planning algorithm some knowledge: we talk about **general purpose planning**.

Classical algorithms have efficiency problems in case of complex domains. There are techniques that make the algorithms more efficient → HIERARCHICAL PLANNING.

1.2.4 – SUSSMANN ANOMALY



Initial state ([Yes]): clear (b), clear (c), on (c, a), ontable (a), ontable (b), handempty.
Goal ([g]): on (ab), on (b, c).
Actions:
pickup (X)
    PRECOND: ontable (X), clear (X), handempty
    POSTCOND: holding (X) not ontable (X), not clear (X), not handempty
putdown (X)
    PRECOND: holding (X)
    POSTCOND: ontable (X), clear (X), handempty

stack (X, Y)
PRECOND: holding (X), clear (Y)
POSTCOND: not holding (X), not clear (Y), handempty, on (X, Y), clear (X)
unstack (X, Y)
PRECOND: handempty, on (X, Y), clear (X)
POSTCOND: holding (X), clear (Y) ,. not handempty, not on (X, Y), not clear (X),

Intuitively, we suppose that we want to achieve first "on(a,b)". To achieve it, we have to remove "c" from "a", then we pick "a" and put it on top of "b". For achieving the second goal "on(b,c)", we have to remove "a" from "b", then we pick "b" and put it on top of "c". At the end of the second goal, we don't have the situation that we want, but "a" on the table and "b" on top of "c".
If we want really to order the goals and these two subgoals are interrupted, then, after achieving the first goal and after achieving the second goal, **it can happen that the goals are not true in the final world description.** So, the **Sussmann anomaly** says that **the number of actions that we achieved to the first goal destroyed what we did before for achieving the second goal.**

Initial plan: START → STOP
Orderings: [start <stop]
List of Causal links: [].
Goal agenda: [on(a, b), on(b, c)]

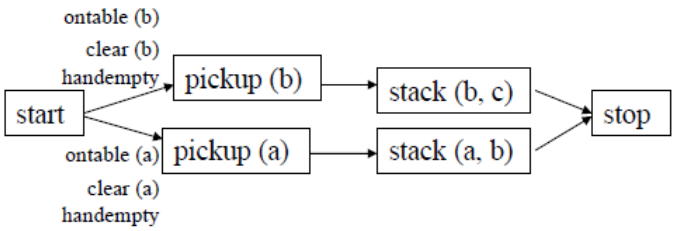
Establishment: chose two actions (without orders) that meet the goals
stack (a, b)
PRECOND: holding (a), clear (b)
POSTCOND: handempty, on (a, b), not clear (b), not holding (a)
stack (b, c)
PRECOND: holding (b), clear (c)
POSTCOND: handempty, on (b, c), not clear (c), not holding (b)
Orderings: [start <stop, start < stack (a, b), start <stack (b, c), stack (b, c) <stop]
List of Causal links: [<stack (a, b), stop, on (a, b)>, <stack (b, c), stop, on (b, c)>].
Goal agenda: [holding (a), clear (b), holding (b), clear (c)]

The actions are not ordered. At the moment we only know that both of them will occur.
Some of the preconditions on the agenda (clear (b) and clear (c)) are already met in the initial state: just add the causal link.
Orderings: [see. above]
List of causal link: [..., <start, stack (a, b), clear (b)>, <start, stack (b, c), clear (c)>].
Goal Agenda: [holding (a), holding (b)]

Now we proceed with the establishment of: holding(a), holding (b)
pickup (a)
PRECOND: ontable (a), clear (a), handempty
POSTCOND: not ontable (a), not clear (a), holding (a), not handempty,
pickup (b)
PRECOND: ontable (b), clear (b), handempty
POSTCOND: not ontable (b), not clear (b), holding (b), not handempty

List of orders : [..., pickup (a) < stack (a, b), pickup (b) < stack (b, c)]
Causal links: [...,<pickup(a), stack(a,b), holding(a)>, <pickup(b), stack(b,c), holding(b)>]
Goal Agenda: [ontable (a), clear (a), ontable(b), clear(b), handempty]

Partial current plan: ontable(b), ontable(a) clear(b) are met in the initial state.



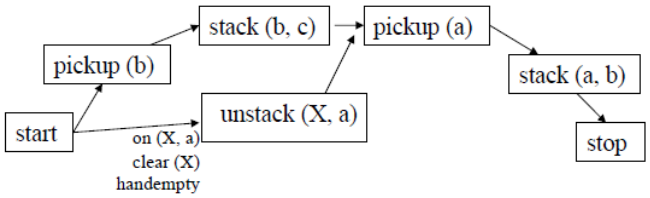
Orderings: [see. above]
Causal links: [..., <start, pickup(a), ontable (a)>, <start, pickup (b), ontable (b)>, <start, pickup (b), clear (b)>]
Goal Agenda: [handempty, clear(a)]

stack(a, b) threats <start, pickup(b), clear (b)> as not clear(b) is one of its effects and nothing prevents it to precede pickup (b) and invalidate its precondition clear (b). → **we impose promotion with pickup(b) < stack(a,b)**
On one hand, the pickup(b) threats the causal link down before pickup(a), on the other hand, the pickup(a) threats the other causal link: it is like a **crossed threat**, so we cannot use demotion and promotion alone: we can use promotion, but we have to add an action. The idea is that stack(a, b) threats the causal link between start and pickup(b).

handempty is true in the initial state.
The causal link <start,pickup(b),handempty> is threatened by pickup(a) → **promotion: pickup(b) < pickup(a).**
The precondition handempty of pickup(a) cannot be satisfied from the start because pickup(b) preceding pickup(a) would negate it.
We **apply the white knight** using the plan action stack(b,c) that reinforces handempty: pickup(b) < stack (b, c) <pickup(a).
Orderings: [..., pickup(b)< stack (a,b), pickup(b) < pickup(a), stack (b,c) <pickup (a)]
Causal Links: [..., <start, pickup(b), handempty>, <stack (b,c), pickup(a), handempty>].
Goal Agenda: [clear(a)]

To satisfy clear (a) we can add the action:
unstack (X, a)
PRECOND: handempty, on (X, a), clear (X)
POSTCOND: handempty not, clear (a), holding (X) not on (X, a)

So the current partial plan is:



Orderings: [..., start < unstack (X, a), unstack (X, a) < stop, unstack (X, a) <pickup (a)]
The causal link list: [..., <unstack(X, a), pickup(a), clear (a)>
Agenda of goals: [On (X, a), clear (X), handempty]
The three preconditions are met on the agenda in the initial state by imposing X = c in the move unstack (X, a).
unstack (c, a) is a threat to the causal link <start, pickup(b), handempty> → **add a new move white knight**

putdown (c)
PRECOND: holding (c)
POSTCOND: ontable (c), clear (c), handempty, not holding (c)

Ordering as well: unstack (c, a) <putdown (c) <pickup (b)
At the end, **we used two types of white knights, one using an action which is already in the plan, and one introducing a new action.** Now all the preconditions are met so we can linearize the plan (add the ordering constraints) and possibly instantiating not yet instantiated variables, to get a concrete plan.
1) unstack (c, a)
2) putdown (c)
3) pickup (b)
4) stack (b, c)
5) pickup(a)
6) stack (a, b)

1.3 – HIERARCHICAL PLANNING

Hierarchical planners are search algorithms that manage the creation of complex plans at different levels of abstraction, by considering the **simplest details only after** finding a **solution for the most difficult ones**.

We need a **language that enables operators at different levels of abstraction**:

- **Values of criticality assigned to the preconditions**: when we have goals and open goals that can be the goals of the stop action or precondition of actions that we have to achieve, there are some preconditions that are easier to achieve. e.g. we are working in the block world: it's easier to achieve an handempy because if we have something in the hand, we release it and it's done.
- **Atomic operators vs. Macro operators**

Popular hierarchical planning algorithms:

- **STRIPS-Like**
- **Partial-Order**

All operators are again defined by PRECONDITIONS and EFFECTS.
Given a goal, the **hierarchical planner performs a meta-level search to generate a meta-level plan** which leads from a state that is very close to the initial one to a state which is very close to the goal.

In general, **we can make a hierarchical planner on top of every other planner** (linear, non-linear, graph...) for example in this way: if you have macro operators that embed atomic actions/operators, when we want to fly from Bologna to Roma and then from Roma to NY, there are atomic actions and the first plan that we do has an high level of abstraction. **So, we start dividing the planning into big steps/macro operator and then we divide these steps in other sub steps**.

The plan is then completed with a lower level search, taking account of details omitted at the previous level.
So, a hierarchical algorithm must be able to:

- **Organize high (meta-level)**
- **Expand abstract plans into concrete plans**
    - Planning abstract parts in terms of more specific actions (planning basic level)
    - Expanding already prebuilt plans

The macro-operators can be decomposed in atomic actions that come with the composition and we can plan for them with another planner.

1.3.1 - ABSTRIPS

ABSTRIPS is a hierarchical planner who enhances the Strips definition of actions with a criticality value (proportional to the complexity of its achievement) **to each precondition**.
The planning algorithm proceeds at **different levels of abstraction spaces**. **At each level, lower level** preconditions are **ignored**. ABSTRIPS fully explores the space of a certain level of abstraction before moving on to a more detailed level**: length search**. **At every level of abstraction, it generates a complete plan**.

Application examples:

- **Construction of a building**
- **Organization of a trip**
- **Draft a top-down program**

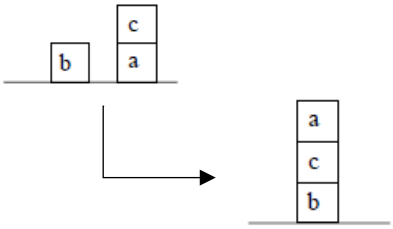
Important features:

- **A threshold value is fixed.**
- **All the preconditions, whose criticality value is less than the threshold value, are considered true**.

- **STRIPS** (or another different planner) **finds a plan that meets all the preconditions** whose **value** is **greater or equal to the threshold** value.
- It then **uses the full plan pattern obtained** as a guide and lowers the value of the threshold.
- **It extends the plan with operators that meet the preconditions whose level of criticality is higher or equal** than to the new threshold value.
- **It lowers the threshold value until all the preconditions of the original rules have been considered**.

It is very important to model the domain in a proper way and this is not very easy. Suppose that we can do it, in this level of abstraction, so we have to attach some criticality values, a knowledge. If we give a wrong value to the preconditions, we cannot continue in a proper way. **That's why it's fundamental to give good values to the critical preconditions.**

1.3.2 – ALGORITHM STRIPS-LIKE



Initial state:

clear (b)	clear (c)	on (c, a)
handempty	ontable (a)	ontable (b)

Goal: on (c, b) ∧ on (a, c)

We specify a hierarchy of goals and preconditions assigning criticality values (that reflect the degree of difficulty in satisfying them):

on	**(3)**
ontable, clear, holding	**(2)**
handempty	**(1)**

The criticality condition is the following:

- handempty has the lower criticality level
- on(x, y) has the highest criticality level because we have to use many actions
- We have three other similar criticality level for table, clear and holding, e.g. for holding we have to take off the piece from the table and to take it in our hands.

We use the STRIPS rules:

pickup (X)
PRECOND: ontable (X), clear (X), handempty
DELETE: ontable (X), clear (X), handempty
ADD: holding (X)

putdown (X)
PRECOND: holding (X)
DELETE: holding (X)
ADD: ontable (X), clear (X), handempty

stack (X, Y)
PRECOND: holding (X), clear (Y)
DELETE: holding (X), clear (Y)
ADD: handempty, on (X, Y), clear (X)

unstack (X, Y)
PRECOND: handempty, on (X, Y), clear (X)
DELETE: handempty, on (X, Y), clear (X)
ADD: holding (X), clear (Y)

First level of abstraction: we consider only the preconditions with criticality value 3 and you get the following goal stack:
on (c, b)
on (a, c)
on (c, b) ∧ on (a, c)
The action stack (c, b) is added to the plan to meet on (c, b). Since its preconditions are all less critical than 3 they are considered to be met. A new state is generated by simulating the execution of action stack (c, b).
We can see that in this description there are inconsistency because b is clear and, in the meantime, there is something on top. It's ok, the preconditions that have a criticality level under 3 are simply not considered.

State description			**Goal stack**
clear (b)	clear (c)	ontable(a)	on (a, c)
ontable (b)	on (c, b)	handempty	on (c, b) ∧ on (a, c)
on (c, a)			

The action stack (a, c) is added following the same process for stack (c, b). The complete plan at Level 1 is:
1. stack (a, c)
2. stack (c, b)

Second level of abstraction: we restart from initial state and in the goal stack we insert the initial goals, the actions computed at level 1 and their preconditions (along with a goal ordering):

holding (c)	clear (b)	holding (c) ∧ clear (b)
stack (c, b)	holding (a)	clear (c)
holding (a) ∧ clear (c)	**stack (a, c)**	on (c, b) ∧ on (a, c)[2]

The condition holding(c) is satisfiable with the action unstack(c, X) and the stack becomes:

clear (c)	on (c, X)	clear (c) and on (c, X)
unstack (c, X)	clear (b)	holding (c) ∧ clear (b)
stack (c, b)	holding (a)	clear (c)
holding (a) ∧ clear (c)	**stack (a, c)**	on (c, b) ∧ on (a, c)

The preconditions of unstack (c, X) to be considered at level 2 (clear(c) and on(c, X)) are all met in the initial state with the substitution X/a.

Executing the action unstack (c, a) on the initial state results in:

State Description	**Goal Stack**
clear (b)	clear (b)
clear (a)	holding (c) ∧ clear (b)
ontable (a)	stack (c, b)
ontable (b)	holding (a)
handempty	clear (c)
holding (c)	holding (a) ∧ clear (c)
	stack (a, c)
	on (c, b) ∧ on (a, c)

And so on until you get the full plan of level 2:
1. unstack (c, a)
2. stack (c, b)
3. pickup (a)
4. stack (a, c)

Third level of abstraction: we have the following goal stack

handempty ∧ clear (c) ∧ on (c, a)	**unstack (c, a)**	holding (c) ∧ clear (b)
stack (c, b)	handempty ∧ clear (a) ∧ ontable (a)	**pickup (a)**
holding (a) ∧ clear (c)	**stack (a, c)**	on (c, b) ∧ on (a, c)

Only the precondition handempty still needs to be considered. The search for a solution to level 3 in this case is simply a check: the solution at level 2 is also correct for level 3 and the search stops.

Note: A level 1 the valid plan

1.stack (a, c)	2.stack (c, b)	→ would fail causing backtracking

[2] Initial goal

1.3.3 – MACRO-OPERATORS

This is the other type of hierarchical planning and this is more intuitive. **There are two types of operators:**

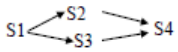
- **ATOMIC OPERATORS**
  - Represent elementary actions typically defined as STRIPS rules
  - Can be directly executed by an agent
- **MACRO OPERATORS**
  - Represent a set of elementary actions: they are decomposable into atomic operators
  - Before execution should be further decomposed
  - Their decomposition can be precompiled or from plan

Types of decomposition:

- **Precompiled**
- **Planned**

PRECOMPILED DECOMPOSITION: the description of the macro operator also contains the decomposition, i.e. the sequence of basic operators to be executed at run-time.

Example
ACTION: Cook (X)
PRECOND: have (X), have (pot), have (salt) in (water, pot)
EFFECT: cooked(X)
DECOMPOSITION: S1: boil (water), S2: put (salt, water), S3: put (X,pot), S4: boilinWater (X)
ORDER: S1 <S2, S1 < S3, S2 < S4, S3 < S4



PLANNED DECOMPOSITION: the planner must perform a low-level search for synthesising the atomic action plan that implements the macro action.

The planning algorithm can be either linear or non-linear. **A hierarchical non-linear algorithm is similar to POP** where at each step one can choose between:

- **Reach an open goal with an operator** (including macro operators)
- **Expand a macro step of the plan** (the decomposition method can be precompiled or schedule).

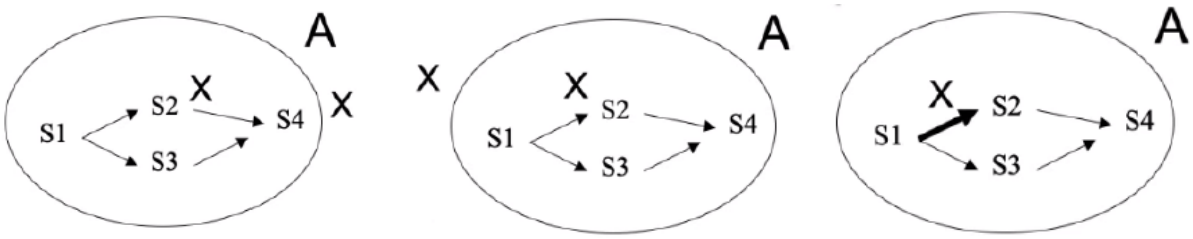
```
function HD_POP (initialGoal, methods, operators) returns plan
        plan: = INITIAL_PLAN (start, stop, initialGoal)
        loop
                if SOLUTION (plan) then return plan;
                choose between
                    • SN, C: = SELECT_SUBGOAL (plan);
                      CHOOSE_OPERATOR (plan, operators, SN, C);
                    • SnonPrim: = SELECT_MACRO_STEP (plan) → if the macro selected is the macro step,
                      we have to be careful because threats have to be taken into account.
                      CHOOSE_DECOMPOSITION (SnonPrim, methods, plan)
                SOLVE_THREATS (plan)
end
```

PROPERTIES OF A SAFE DECOMPOSITION

To ensure the decomposition is safe, some properties must be guaranteed and they are called "**constraints on the decomposition".** Only under these conditions you can replace the macro action A with the plan P.
For decomposing the action we should be careful because there can be some threats that appear and we have to protect them.

If the A macro action has the effect X and is expanded with the plan P:

- **X must be the effect of at least one of the actions in which A is decomposed** and it should be protected until the end of the plan P
- **Each precondition of the actions in P**:
  - Must be **guaranteed by the previous actions in P**
  - **Or** it must be a **precondition of A**
- **The P action must not threat any causal link** when P is substituted for A in the plan



When replacing A with P, the orderings and causal links should be added.
Orderings

- For each B such that B <A then B <first(P) is imposed (**first action of P)**
- For each B such that A <B then last (P) <B is imposed (**last action of P)**

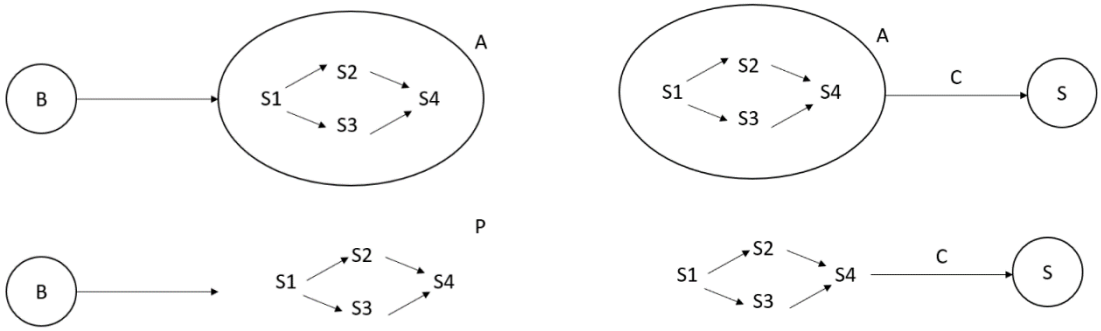
Causal links

- **If <S, A, C> is a causal link in the initial plan**, then it must be **replaced by** a set of causal links **<S, Si, C>** where Si are the actions of P that have C as a precondition and no other step of A before it has a C as a precondition
- **If <A, S, C> is a causal link in the initial plan**, then it must be **replaced by** a set of causal links **<Si, S, C>** where Si are the actions that have C as an effect and no other step of P after it has the effect C

These macro actions cannot be executed, we need to execute the plan. So, a macro is like a box (circle) and we need to substitute this box A with something that can be executed. A is decomposed in a number of atomic actions and when we make the substitution, we have to take care of putting correct ordering constraints and so on. **This is a partial order, so we need to insert many constraints as many last actions of A we have.**

When we have a causal link where A requires C, we open the macro action and we substitute the composition, in this case we have only S2. When we open the box, the macro A is no longer there, so we take into account the causal link of the specifics Sn.

e.g. decomposition with actual actions: example of cook. The macro action is “cook” and when we decompose the macro, we have a partial order. If we want to change topic, maybe we can think that the macro is “from Bologna to Rome”. The micro actions are “from home to the airports”, etc...



1.3.4 – POP-LIKE ALGORITHM

INITIAL STATE GOAL
have (pot), have (pan), have (oil), have (onion),
made (pasta, tomato), have (pasta), have (tomato), have (salt), have (water)

ATOMIC ACTIONS:

ACTION makePasta (X)
PRECOND: have (pasta), cooked(pasta), Sauce (X)
EFFECT:made(pasta, X)

ACTION boil (X)
PRECOND: have (X), have (pot), in (X, pot), plain (X)
EFFECT: boiled (X)

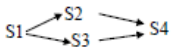
ACTION boilinWater (X)
PRECOND: have (X), have (pot), in (water, pot), boiled (water), in (salt, water), in (X, water)
EFFECT: cooked (X)

ACTION cookSauce (X)
PRECOND: have (X), have (pan), in (onion, pan), fried (onion), in (X, oil)
EFFECT: sauce (X)

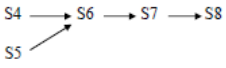
ACTION fry (X)
PRECOND: have (X), have (pan), have (oil), in (oil pan), in (X, pan), plain (oil)
EFFECT: fried (X)

ACTION: put (X, Y)
PRECOND: have (X), have (Y)
EFFECT: in (X, Y), ¬ plain (Y)

ACTIONS MACRO:

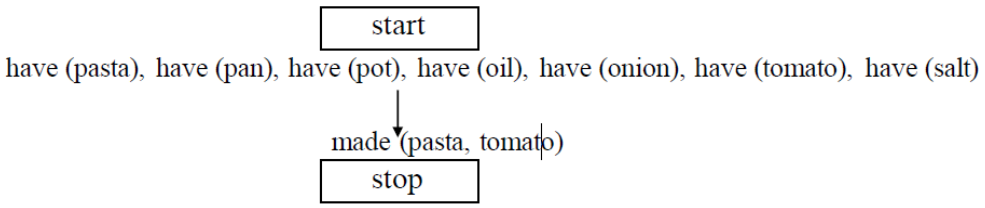


ACTION: Cook (X)
PRECOND: have (X), have (pot), have (salt) in (water, pot)
EFFECT: cooked(X)
DECOMPOSITION: S1: boil (water), S2: put (salt, water), S3: put (X, pot), S4: boilinWater (X)
ORDER: S1 < S2, S1 < S3, S2 < S4, S3 < S4

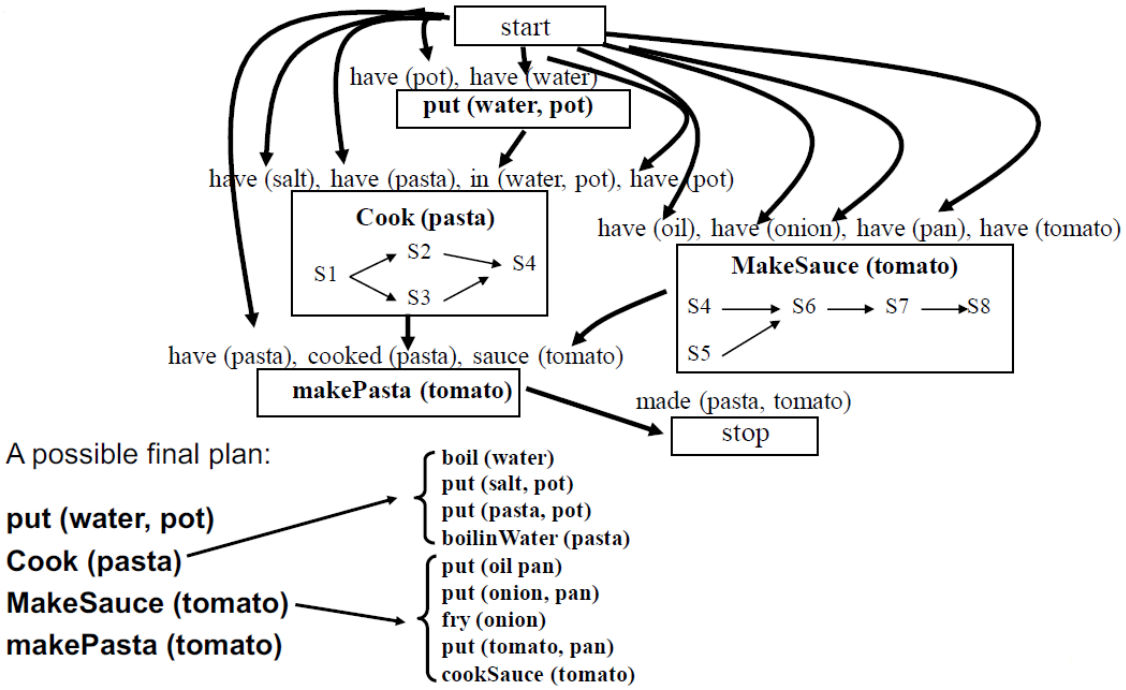


ACTION: MakeSauce (X)
PRECOND: have (X), have (oil), have (onion), have (pan)
EFFECT: sauce (X)
DECOM: S4: put (oil pan), S5: put (onion, pan), S6: fry (onion), S7: put (X, oil), S8: cookSauce (X)
ORDER: S4 < S6, S5 < S6, S6 < S7, S7 < S8

Initial empty plan:



Complete plan:



1.4 – PROBLEMS OF EXECUTION

We make some assumptions: we're offline and no other agent can interact with our plan.
This can work in some domain, e.g. if we are describing a set of users that are using a specific app in a specific moment, this cannot be efficient because we need dynamic solutions. We can use this kind of technology if the world we are working in, doesn't change a lot.
For example, if we have a robot who walks in a room and people are moving chairs and tables, it's difficult.

The effect of an action sometimes can be something that we are not expecting: we can have errors, e.g. we have a robot which takes a glass and the robot destroys the glass.
Or maybe we have unpredictable effect: e.g. during the movement of a block on a table, the block falls down and so the effect is unpredictable.

In real life things are not as clear as we think they are: CWA means that our knowledge of the initial state of the world is correct and complete, so if something is not explicitly in the initial state, this is false.
But **the knowledge about the initial state, could be incomplete**... So how can we face the problem?
We use OWA, so everything written in the initial state is true, what is not written, is unknown. So, we can perceive the changes: it is very important that we can retrieve the knowledge we don't have.
Every time we want to execute one action, we can perceive whether the precondition are true and respected and, for example, when we do an action, we can perceive and check the effect of the actions.

To resume**, generative planners build plans that are then executed by an executing agent.**
Possible problems encountered during execution:

- **An action should be executed and its preconditions are not satisfied**
  - **Incomplete or incorrect knowledge (OWA)**
  - **Unexpected conditions** (e.g. broken glass)
  - **The world changes independently from the planner** (e.g. chairs)

- **Action effects are not the one expected**
    - **Errors of the executing agent**
    - **Non deterministic/unpredictable effects** (e.g. the block falls down)

While executing, the agent should perceive the changes in the world and Act accordingly.

Some planners run under the hypothesis of OWA as opposed to the CWA: they consider that the information that is not explicitly stated in a state is not false, but unknown. The unknown information can be retrieved via **sensing actions added to the plan** and those sensing actions are **modelled as causal actions**.
The **preconditions** are the conditions that must be true to perform a certain observation, while the **postconditions** are the result of the observation.

Two possible approaches:

- **Conditional Planning:** extreme, robust, it wants to plan every possible path. (e.g. Cassandra, Buridan)
- **Integration** between Planning and Execution (e.g. IPEM, SAGE, XII)

Sensing actions do not change anything, they just observe something in the world, like the temperature in a room. We don't change the temperature, but if the temperature changes, we observe it.
How to describe these sensing actions? If we want to observe a specific temperature and check if it is higher than X, we need a thermometer. It's a precondition.

1.5 – CONDITIONAL PLANNING

A Conditional Planner (or Contingency Planner) is a search algorithm that generates various alternative plans for each source of uncertainty of the plan. A conditional plan it is therefore **constituted by:**

- **Causal Actions**
- **Sensing Actions** for retrieving unknown information
- **Several alternative Partial Plans** of which **only one will be executed** depending on the results of the observations.

Example
INITIAL STATE: On (tire1, HUB1), flat (tire1), inflated (spare), off (spare)
GOAL: On (X, HUB1), inflated (X)

ACTIONS:

remove (X, Y)
PRECOND: on (X, Y), ¬ intact (X)
EFFECT:¬ on (X, Y), off (X), clearHub (Y)

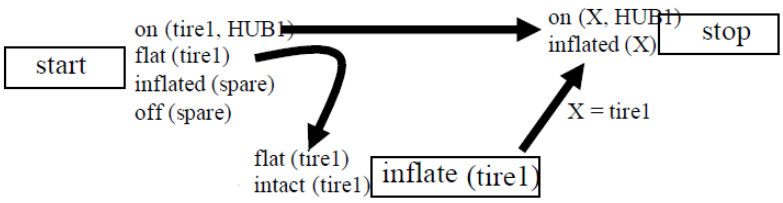
putOn (X, Y)
PRECOND: off (X), clearHub (Y)
EFFECT: on (X, Y), ¬ off (X), ¬ clearHub (Y)

inflate (X)
PRECOND: intact (X), Flat (X)
EFFECT: inflated (X), ¬ flat (X)

SENSING ACTION:

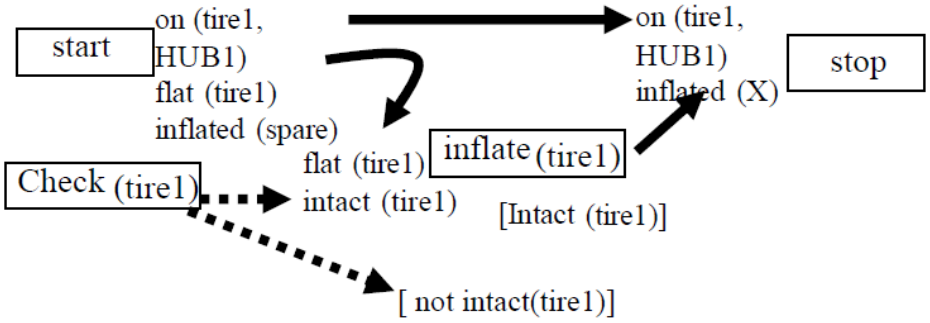
checkTire (X)
PRECOND: tire (X)
EFFECT: knowsWhether (intact (X))

A traditional planner (without sensing actions) would produce the following plan:

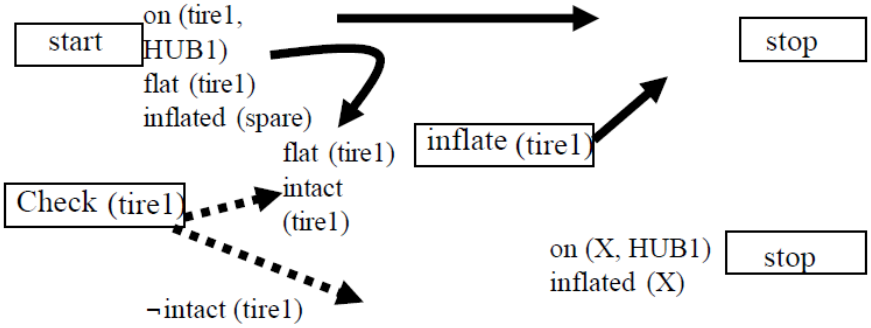


inflated(tire1) has 2 preconditions:
- flat(tire1) → ok
- intact(tire1) → and here? A traditional planner would fail because there's no causal actions that provide us this precondition. In fact, the condition intact (tire1) does not appear in the description of the initial state and cannot be obtained as an effect of any action: **failure of the plan.**
Upon backtracking the plan fails again as we get X = spare: remove (tire1, HUB1) - putOn (spare, HUB1) as the precondition ¬ intact (tire1) cannot be achieved in any way.

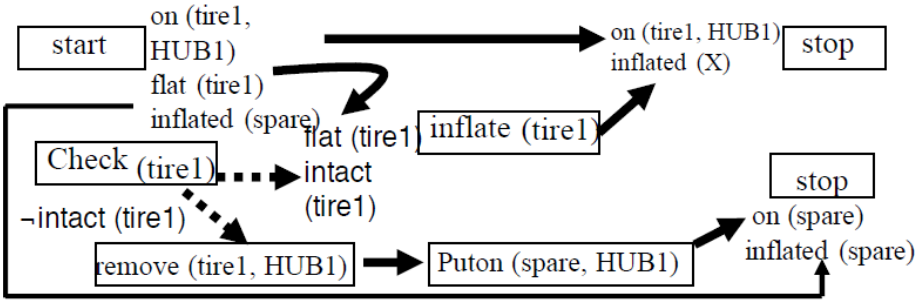
Using the sensing action one can build a conditional plan:



inflate(tire1) is the correct plan in only one executing scenario: a context in which intact (tire1) is true in the initial state. **We should generate a copy of the goal for every other executing scenario** and generate a corresponding plan for each of them. We have an **exponential number of plans**.



So, putting all together, we have the following conditional plan:



It is exponential because **if we have N sensing actions with 2 outcomes, we have 2^N possible scenarios**. it's **unaffordable**.

PROS	CONS
• **Really** important for safety of executing agents • **Extreme approach, but** very effective • Very robust • Perfect for applications **that need to be** deterministic	• **Combinatorial explosion of the search tree** with high numbers of alternative contexts. • **Require a lot of memory** • **Not always all alternative contexts are known in advance** • Often **associated with probabilistic planners** that plan only for the most probable contexts

1.6 – REACTIVE PLANNING (Brooks, 1986)

We have described a deliberative planning process where the plan is built before execution, but there's another possibility: **reactive planners are on-line algorithms**, capable of **interacting with the world** to deal with the **dynamicity** and the **non-determinism of the environment**:

- They **observe** the world **in the planning stage**
- They **acquire unknown information**
- They **monitor the implementation of actions** and **check the effects**
- They **interleave planning and execution**

Pure reactive systems do not plan, they only react as triggers to world variations.

They have access to a **knowledge base that describes what actions must be carried out and under what circumstances**. **Choose one action at a time**, without any lookahead activity.

E.g. A thermostat uses the simple rules:

- If the temperature T of the room is K degrees above the threshold T0, turn the air conditioner on;
- If the room temperature is below T0 K degrees, turn the air conditioner off.

PROS	CONS
• Able to **interact with the real system**. • **Robust in domains for which it is difficult to provide complete and accurate models**. • **Extremely fast** in responding because they don't use models	• **Quite low performance** in predictable domains that require **reasoning** • **Unable to generate plans**

1.7 – HYBRID SYSTEMS

Modern responsive planners are hybrids integrate a generative approach and a reactive approach in order to exploit the computational capacity of the first and the ability to interact with the system of the second. They:

- **Generates a plan** to achieve the goal
- **Checks the preconditions** of the action that is about to run
- **Checks the effects of the action** that just executed
- **Disassembles the effects of action** (importance of action reversibility)
- **Reschedules in case of failures**
- **Corrects the plans if unforeseen external events occur**.

In the real world, with a reactive approach, some actions cannot be backtracked, reverted; there are some actions, instead, that can be undone because they are easy to be reverted.
In some cases, we need a backtracking plan and, depending on the kind of action, the backtrack is forbidden or possible (simple or complex). To stay on the safe side, we can just execute actions that can be backtracked and provide the backtrack plan.

2 – GRAPH PLAN

In 1995 a new concept of planner based on graph has been proposed by Blum and Furst CMU: the GraphPlan. While planning creates a graph called Planning Graph in which, at each step of the search, the data structure is extended, **GraphPlan inserts the TIME DIMENSION in the plan construction process**.
GraphPlan is a **correct and complete** planner among the most **efficient** that have been built.

2.1 - FEATURES

- **Uses the CWA** falling into the category of **off-line planners**
- **Returns** either the **shortest possible plan or** returns an **inconsistency.** This is a very important feature! The inconsistency can be returned if we limit the number of levels or time steps that the algorithm has to take into account
- **Inherits from linear planners the EARLY COMMITMENT feature**. When it selects an action, this action is considered true. e.g. action A is executed at time step 2
- **Inherits from non-linear POP** the ability to **create partially ordered sets of actions**
    - **Generates parallel plans,** but they can be executed only if we have parallel agents
- **Actions** are represented as the ones in STRIPS
    - **PRECONDITIONS**
    - **ADD LIST**
    - **DELETE LIST**
- **Objects have a TYPE**: typing objects are the terms that we have seen in the propositions of POP of STRIPS, e.g. P1 is a term but is also a position, A is a term but is also a block. **Useful to avoid ambiguity**.
- There is an **action "NO-OP/FRAME ACTIONS" that does not change the state** (**frame problem**). It is an action that enables you to bring forward conditions not changed by the action.
- **States are represented as sets of predicates** that are true in A given state. (We are in the CWA, so there are the same assumptions for STRIPS and POP)

2.2 - PLANNING GRAPH

The planning graph is a directed levelled graph:

- Nodes belong to different levels (that's why levelled)
- Arcs connect nodes in adjacent levels (that's why directed).

We have 2 kind of levels:

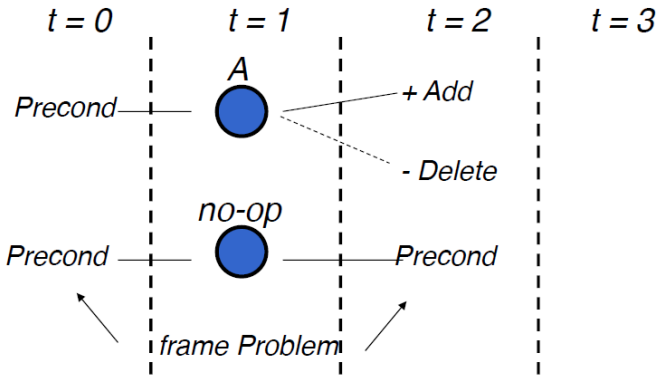
- **PROPOSITION LEVELS**: here we have properties.
    - Level 0 corresponds to the initial state and it is the proposition level with all the properties.
- **ACTION LEVELS**: it contains all the actions that can be executed in this level

In the planning graph those levels are interleaved and correspond to increasing time steps, nonetheless interfering actions and propositions in a time step T can appear. In fact, if I say that the planning graph contains all possible plans, there's no guarantee that the plans are consistent one with respect to the other.

Arcs are divided into:

- **Precondition arcs** (proposition → action)
- **Add arcs** (action → proposition, similar to postconditions)
- **Delete arcs** (action → proposition, similar to postconditions)

In each time step an action A can be inserted if in the previous time step all preconditions of A exist. There are special actions representing the "doing nothing" activity: these actions are called **no-op or frame actions**.



Each action level contains:

- All **actions** that are **applicable** at that time step
- **Incompatibility constraints** connecting pairs of actions that cannot be performed simultaneously

Each proposition level contains all literal that might result from any choice of actions in the previous time step including no-op. Even these literals can be inconsistent. **Only when the planning graph is completely built, we can look to the real plan to be executed:** we work with a **breadth-first strategy** more or less.

2.3 - INCONSISTENCIES

During the construction of the planning graph inconsistencies are Identified, in particular:

- Two **actions** can be **inconsistent** in the **same time step**
- Two **propositions** can be **inconsistent** in the **same time step**

In this case the **actions/propositions are mutually exclusive**:

- They **cannot appear together in a plan**
- But **they may appear in the same level** of the planning graph

2.3.1 – INCONSISTENT ACTIONS

There are 3 types of inconsistent actions:

- **INCONSISTENT EFFECTS**: one action negates the effect of another.
  e.g. The move action(part, dest) has the effect NOT at(part), while the no-op action on at(part) has this effect. The effects of those 2 actions are incompatible.
- **INTERFERENCE**: an action deletes a precondition of the other.
  e.g. The move action(part, dest) has the effect NOT at (part), while the no-op action on at(part) has this as a precondition. This is the **most frequent**.
- **COMPETING NEEDS**: two actions that have mutually exclusive preconditions.
  e.g. The load(x) has as a precondition NOT in(x) while the action unload(x) has as a precondition in(x)

2.3.2 – INCONSISTENT PROPOSITIONS

Two propositions are inconsistent if:

- **One is the negation of the other**
- **All the ways to reach them are mutually exclusive**

Furthermore, **there might be domain dependent inconsistencies,** e.g. an object cannot be in two places at the same time in the same time step

2.4 – PLANNING GRAPH ALGORITHM

The planning graph is built as follows:

1. **All true propositions in the initial state** are inserted in the first proposition level
2. **Creation of action level**
    a. For every operator and every way to unify its preconditions to propositions in the previous proposition level, enter an **action node IF its preconditions are not labelled as mutually exclusive**
    b. In addition, **for every proposition** in the previous proposition level, **add a no-op operator**
    c. **Check if the action nodes do not interfere each other** otherwise mark them as mutually exclusive
3. **Creation of proposition level**
    a. For each action node in the previous level, **add propositions in its add list through solid arcs** and add **dotted arcs connected to the propositions in the delete list**
    b. Do the **same process for the no-op operators**
    c. Mark as **mutually exclusive** two **propositions** such that all the ways to achieve the first are incompatible with all the ways to reach the second

Algorithm for the construction of the planning graph

1. **INITIALIZATION: a**ll true propositions in the initial state are included in the first proposition level
2. **CREATING AN ACTION LEVEL**
    a. For each operator and for each way to unify its preconditions to not mutually exclusive propositions in the previous level, **enter an action node**
    b. **For every proposition** in the previous proposition level, enter a **no-op operator**
    c. **Identify the mutual exclusive relationship** between the newly constructed operators
3. **CREATING A PROPOSITION LEVEL**
    a. ∀action node in the previous action level, **add the propositions in the add list by solid arcs**
    b. **For every "no-op"** in the previous level, add the corresponding proposition
    c. ∀ action node in the previous action level, **link propositions in the delete list by dashed arcs**
    d. **Identify incompatible propositions**.

```
function GRAPHPLAN(problem):
         graph = INITIAL_GRAPH (problem)
         targets = GOAL (problem)
         do loop:
            if objectives not mutex last step:
               Sol = EXTRACT_ SOLUTION (graph, objectives)
               if Sol ≠ fail: return Sol
               else if LEVEL_OFF (graph): return fail
            = EXPAND_GRAPH graph (graph, problem)
```

In case there is no solution, the loop is without end, so that's why we set a level_off: we need a stop.

- **The first node contains the initial planning graph.** This contains only one time step (proposition level) with true propositions in the initial state. The initial graph is extracted from INITIAL_GRAPH (problem).
- **The goal to reach is extracted from the function GOAL(problem)**
- **If goals are not mutually exclusive in the last level**, then the **planning graph could include a valid plan**. **The valid plan is extracted through BACKWARD search** EXTRACT_SOLUTION(graph, objectives) that provides either a solution or a failure
    - **Proceed level by level to better exploit the mutual exclusion constraints**
    - **Recursive Method**: given a set of goals at time t the algorithms looks for a set of actions at time t-1 who have such goals as add effects. **The actions should not be mutually exclusive**.
    - **The search is the hybrid breadth/depth first and complete**
- **MEMOIZATION:** If at some step, a **subset of goals is not satisfiable**, graphplan saves this result in a **hash table**. Whenever the same subset of goals is selected in the future will automatically fail.

THIS ALGORITHM IS CORRECT AND COMPLETE.
Complete because if there is a solution, it will find it at some point. I don't have to cut off the number of levels I want to expand. However, since the problem is semi-decidable, I have to go on, go on, go on for looking for these levels. If there's not a plan, I can continue indefinitely, forever.
e.g. I have 10 and after 6 levels we find all the goal, so from the 6 level we start looking backward. If the plan exists, ok, instead we expand the 7 level, etc... When I reach the 10th level, if we find a plan good, ok, else we fail and that's all.
As far as correctness is concerned, if we consider the mutually exclusiveness, we end up having a correct plan.
The problem here is just completeness because we have to stop it.

2.4.1 – EXTRACTION OF A VALID PLAN

Once the planning graph is built, we have to extract a valid-plan, i.e. a connected and consistent subgraph of the planning graph.

FEATURES OF A VALID PLAN

- **Actions** in the same time step can be **performed in any order** (**do not interfere**)
- **Propositions** at the same time step **are not mutually exclusive**
- **The last time step contains all the literals of the goal** and these are **not marked as mutually exclusive**

If we have all these 3 conditions satisfied, we have a valid plan that is also correct and complete.

2.4.2 – THEOREMS

1. **If there is a valid plan** then this is a **subgraph of the planning graph.** This theorem is for **completeness.**
2. In a planning graph **two actions are mutually exclusive** in a time step **if a valid plan that contains both does not exist**. This theorem is for correctness.
3. In a planning graph **two propositions are mutually exclusive** in a time step **if they are inconsistent**, i.e. one of them denies the occurrence of the other.

Important consequence: **the inconsistencies found by the algorithm prune paths in the search tree**.

2.5 - FAST FORWARD

FF is an extremely efficient heuristic planner introduced by Hoffmann in 2000. Heuristic means that **in each state S we compute an estimate of the distance to the goal.**
This algorithm is very smart: we start with easiest possible informed strategies like hill climbing. We have a starting node and here the initial state is very simple. Then, we add all possible successors and these are simply actions that we can do. Then, we evaluate the successors with a heuristic that is created through graph plan. So, we start with 1 state, we open the successors, we analyse all of them and we choose the most promising and we apply A*.

Basic operation: hill climbing + A*

1. From a state S, **examine all successors** S '
2. **If a successor state S* exists better than S, move on it** and go back to **point 1**
3. **If there is no state with a better evaluation**, a **complete A* search is run**, using the same heuristic

2.5.1 – A*

In Informed Search Strategies, we solve the problem by selecting the most promising next successor of a node.
So, for example, if we have to reach a destination, we can consider as a heuristic, a linear distance between a node and a destination. e.g. From Bologna to Rome. So, the next node could be Modena or Venice or Florence, then Rome. Obviously we choose Florence.

In A* we don't just consider the distances, but we consider also the costs. Nobody will go to Venice with the goal of reaching Rome. One important thing is that **in general A* doesn't guarantee to find the optimal solution**.

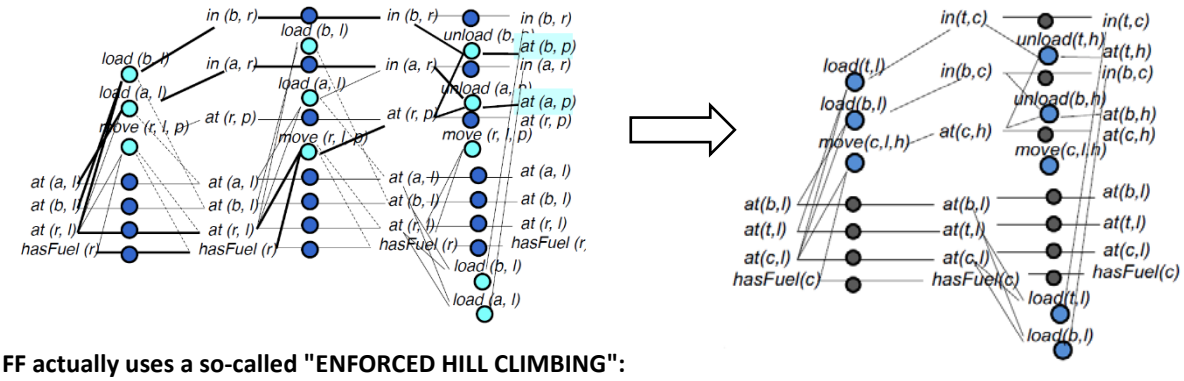
To resume:

- **A* also consider both the distance to the goal and the cost in reaching the node n from the root**
- **We expand nodes for increasing values of f(n) where f(n) = g(n)+h'(n),** g(n) is the depth of the node and h'(n) is the estimated distance from the goal.
- **A* does not guarantee to find the optimal solution** (it depends on the heuristic function).
- Suppose you indicate with **h(n) the true distance between the current node and the goal**: the heuristic function **h'(n) is optimistic that if we always have h'(n)<=h(n). This heuristic function is said to be feasible**.
- **THEOREM**: if h'(n)<=h(n) for each node, A* algorithm always finds the optimal path to the goal.
- Obviously, **the perfect heuristic h'=h is always feasible**. **If h'=0**, we always obtain a feasible heuristic function because it's like we're implementing the **breadth-first search**.

2.5.2 – HEURISTIC FUNCTION

For Fast Forward we use Graph Plan as heuristic. We neglect the delete effect of the actions, so, we will miss many incompatibility constraints and when we'll have found in the **final state the goal**, we'll **proceed backward** very easily because we have **less incompatibilities**. This is not a real plan, so we forget about the delete list and **we consider only the add list**.
Of course, **it is an optimistic heuristic** because Graph Plan could not use less actions if it has more constraints, so **more constraints would lead Graph Plan to add more levels**, not to remove levels.

Given a problem P, a state S and a goal G, FF considers a relaxed problem P+ that you get from P by neglecting delete effects actions. **FF solves P+ with graphplan and the number of actions in the resulting plan it is used as heuristics.**



FF actually uses a so-called "ENFORCED HILL CLIMBING":

```
function FF(problem): Return solution or fails
        S = Initial_state (problem)
        k = 1
        do loop:
           explore all states S 'at k steps
           if a Better state S* is found: S = S *
           else if k can be increased: k = k + 1
           else perform complete A* search
```

So, there is an **initial step of hill climbing**, but the important part is that, then, **we use A* with this heuristic. In practice, it is a complete breadth-first search**. **A solution is always found**, unless the current one is not a dead end.

3 – SWARM INTELLIGENCE

The planner in the previous approaches was a lonely agent with huge reasoning capability and it performed all the tasks by itself.
Now, we move in a **setting called "DISTRIBUTED AI"** (it is a huge research field) where a smart behavior is done in a completely different approach: we have a **population of agents with very limited reasoning capabilities** but **collectively they interact with the environment** and **they obtain a collective behavior that is intelligent**.
These techniques have been built observing natural behaviours and they come from **Social Science** studies that have explained how social intelligence can arise.

Although **there is normally no centralized control structure dictating how individual agents should behave**, local **interactions between such agents often lead to the emergence of global behavior**. Examples of systems like this can be found in nature, including ant colonies, bird flocking, animal herding, bacteria moulding and fish schooling.

Human Intelligence is the result of social interaction. Evaluate, compare and imitate other individuals, learn from experience emulating the successful behavior of others, enables people to adapt to complex environments and situations through the discovery of optimal patterns, beliefs and behaviours (Kennedy & Eberhart, 2001).
Culture and cognition are consequences of human social choices. **Culture emerges when individuals become similar through a process of social learning**. To model human intelligence there is need to model individuals in a social context.

3.1 – FEATURES

If we have to code an algorithm that does a lot of things, it's an effort, while if we have to code simple agents that simply react to something, it's much easier.

SI systems have several **FEATURES**:

- **A set of SIMPLE INDIVIDUALS with LIMITED CAPACITIES**
- **INDIVIDUALS ARE NOT AWARE OF THE SYSTEM IN ITS GLOBAL VIEW**
- **LOCAL COMMUNICATION PATTERNS** (direct or indirect - **STIGMERGY**): in general, these entities/individuals are connected by either proximity constraints or graph connections. e.g. a person could be connected to another person in Facebook but they could not necessarily be close in terms of geographic places.
- **DISTRIBUTED COMPUTATION**: **no centralized coordination** of individual activities (in general it's true, but there are some exceptions)
- **ROBUSTNESS: if we have crashes of computing nodes,** for example**, these systems can still work**. e.g. swarm intelligent system: our brain. In our brain we have neurons that are very simple individuals. One neuron receives some input and it send it to another neuron. Our brain overall has emerging intelligent behavior: if with a surgery we remove a part of the brain, the functionality of this missing part can be devolved to other neurons. Obviously, **if the crashes are too high**, there are no chance and we talk about **graceful degradation**.
- **ADAPTIVITY:** these individuals **interact with environment** and when it changes, they are **resilient** to environmental changes.

3.1.1 - STIGMERCY

STIGMERGY is a **form of indirect communication. Explicit communication**: one individual sends a message to another individual. In these kinds of environment, we do not in general have always these direct communications. In general, what we do is **indirect communicate**: this is obtained **through the modification of the environment**. e.g. ants make a column and this movement is organized as if it was a controller. This

synchronized movement is achieved through a **pheromone** that attracts other ants to follow the same path, so, the pheromone is a modification of the environment.

3.1.2 – SELF-ORGANISATION

To act like a centralized controller, we need 2 ingredients: interactions and feedbacks.
In particular:

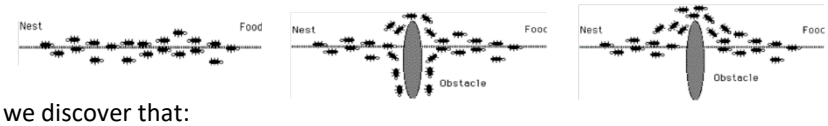
- **Multiple interactions among agents**
  - Simple agents (es. rule based)
  - Multi-agent systems
- **Positive Feedback**
  - Reinforcement of common behaviours
  - Amplification di random fluctuations and structure formation
- **Negative Feedback**
  - Saturation
  - Competition
  - Exhaust resources

3.2 - ALGORITHMS

Many algorithms exist based on the swarm intelligent concept:

- **ANT COLONY OPTIMIZATION - ACO (Dorigo, 1992)**
  Algorithm based on the behaviour of ants. Positive feedback based on pheromone trails that reinforce components that contribute to the problem solution. It was one of the first attempt to study swarm intelligence and we'll find in this algorithm all the features we talked about.
- **ARTIFICIAL BEE COLONY ALGORITHM (Karaboga in 2005)**
  Algorithm based on the behaviour of bees. Population of bees that look for nectar. Not so used, but it is very interesting because it has a particular feature: individuals have different types so they have different behaviours.
- **PARTICLE SWARM OPTIMIZATION - PSO (Kennedy, Eberhart 1995)**
  Algorithm based on the observation of bird flocks or fish shoals. Stigmergy as communication and imitation of neighbourhoods. It has been used a lot with robotic applications.

3.2.1 – ANT COLONY



From the **observation of the ants** we discover that:

- **Ants deposit pheromone trails** while walking from the nest to the food and vice versa
- **Ants tend to choose** (more likely) the **paths marked with higher pheromone concentrations**
- Cooperative interaction leads to the **emergent behavior to find the shortest path**

Scientist observed that there are some ants moving alone going around without a "structure" so that's a sign that there's a probability largen than 0 that some of them would not follow that path.
In ant colonies these interactions through stigmergy always find the shortest path: they simply follow the highest pheromone trail. They don't have any idea that they are following the shortest path.

As shown in the figures, if we put an obstacle, at the start the ants go half to one side and half to the other side. Then, the pheromone evaporates and they all follow the shortest side of the obstacle.

ANT COLONY OPTIMIZATION

Engineers asked themselves if they could mimic this behavior in algorithms that look for the shortest path and other applications. **All these algorithms that are under the umbrella of Swarm Intelligence are PROBABILISTIC**.

In fact, Ant Colony Optimization is a probabilistic parametrized model.
If you want them to be **deterministic algorithm**, the **emergent behavior simply wouldn't arise**. So, **a crucial feature** of the ant colony optimization (but also of all the algorithms of SI) is probability.
The adjective **PARAMETRIZED** is due to the fact that **parameters and their tuning are very important aspects that influence the performance** of the whole algorithm. If we don't tune them in a proper way, our algorithm simply doesn't work.

We'll see 3 algorithms and one of these has been studied by an Italian scientist that had a very prestigious project: he discovered that if the parameters of the project are not properly set, the algorithm doesn't work.
So, probability and parameter tuning are crucial ingredients for make these algorithms working properly.

- **Probabilistic parametrized model** – the pheromone model – **used to model pheromone trails**
- **Ants build solution components in an incremental way**
- **Stochastic steps on a fully connected graph** called **CONSTRUCTION GRAPH**: G = (C,L).
    - Vertexes C are solution components
    - Arcs L are connections
    - States are paths on G
- **Constraints can be represented** to define what is a consistent solution

TSP: Travel Salesman Problem

If the graph is not fully connected, i.e. we can't reach every city from every city, then we can remove arcs and add constraints.
A TSP model in ACO:

- **Nodes of G (solution components) are cities** to be visited;
- **Arcs are connections among cities**
- A solution is a **Hamiltonian path in the graph**
- **Constraints are used to avoid sub-cycles**: each ant can visit a city once

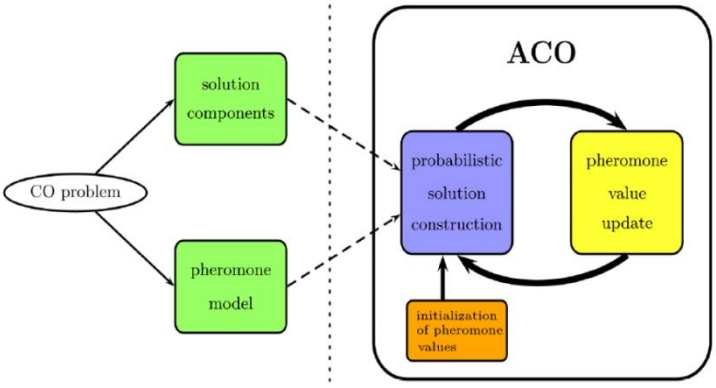
INFORMATION SOURCES

Connections, solution components or both have the following ASSOCIATED INFORMATION:

- **PHEROMONE τ:** the pheromone value abstracts natural pheromone trails and codes the **long-term memory of the global search process**.
- **HEURISTIC VALUE η:** the heuristic value represents the **prior background knowledge** on the problem.

We have two associated information: the pheromone certainly, but we are in a computer science context so we can introduce some knowledge on the problem. Indeed, we can associate to each component also a heuristic value. **I can order them by increasing cost** and we start selecting the closest. In general, a good heuristic for reaching a node is to try to find the shortest distance, so we will first select the value that has the shortest distance between the cities.
LONG-TERM MEMORY: if the pheromone has a specified quantity left on a specific arc by an ant traveller and it lasts for a while until evaporation, then the overall system has to remember what are the most selected path.

A Combinatory Optimization (CO) Problem has the schema on the right and is composed by **solution components** and **pheromone model** that represent the **modelling part**.
The solution components are for example the cities. We have to create a model, so we need to choose which are the constraints. Here we have to decide what



are the components that belong to our solution (arcs and nodes).
In the pheromone model we decide how much pheromone is left in the path and the probability with which we select one solution component based on this pheromone model.
Then, in the right part of the schema, in the orange box we have the **initialization** (not always used). We can initialize the pheromone values of each solution component.
In some cases, we can say that all pheromone values are simply 0, otherwise we can **randomly assign values**, e.g. in neural network we have randomly initialization.
Every time we compute a solution, we update the pheromone value on the network and then we select the next solution.

ANT-SYSTEM ALGORITHM

```
InitializePheromoneValues() → we assign to each arc a value which can be 0 or a random number
while termination conditions not met do
        for all ants a ∈ A do
                Sa ← ConstructSolution(t ,h)
        end for
        ApplyOnlineDelayedPheromoneUpdate()
end while
```

This while cycle is the core of the algorithm: for all ants a belonging to the population A, it builds a construction through a procedure that is a construct solution with 2 parameters, it creates all solutions.
Then, it updates the pheromone and continuing building other solutions.

ANT-SYSTEM

- **Memory** is used to **remind past paths**
- Starting from node i, we have to **probabilistically choose the next consistent node** to visit. Here we tend to select best nodes, like Greedy, but it is not deterministic, it's probabilistic.
- The **probabilistic choice depends on**
  - **Pheromone trail** τij
  - **Heuristics** $\eta ij = 1/dij$

We have two parameters on the arc, particularly the heuristics is the inverse of the distance between node i and node j. Let's consider we have node j, k and i. Their distances are 10, 5 and 2. We have 1/10 probability of choosing the first, 1/5 of choosing the second and 1/2 of choosing the third. The heuristics choose how much we give weight to our nodes. **So, higher is the distance, lower is the heuristic. Lower is the distance, higher is the heuristic.**

$$p_{ij} = \begin{cases} \dfrac{[\tau_{ij}]^{\alpha}\ [\eta_{ij}]^{\beta}}{\sum_{k\ \text{feasible}} [\tau_{ij}]^{\alpha}\ [\eta_{ij}]^{\beta}} & \text{if j consistent} \\ 0 & \text{otherwise} \end{cases}$$

Alfa and beta are weight of the importance of pheromone and the heuristics. If one of the two is 0, we remove one of the parameters. **If we remove the pheromone, we have more or less the hill climbing**. If we put heuristic to 0, we rely only to the pheromone.
The pheromone is updated with the following rule:
Where **ρ is the evaporation coefficient** and **Lk is the length of the path followed by the ant k.**

$$\tau_{ij} \leftarrow (1 - \rho)\,\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k} \qquad \Delta\tau_{ij} = \begin{cases} 1/L_k & \text{if ant k used arc (i,j)} \\ 0 & \text{otherwise} \end{cases}$$

If we have 1 ant that has built a total path of length 100 and another ant that has built a total path of length 10, the ant with the total path shorter, has a higher contribution. If we apply the **offline delay pheromone update**, every ant builds its own solution and the Lk becomes the total length of the path of the ant k.
Now, suppose that we have 100 ants and each of these ants had built a solution. At the end of the solution construction, we have to update the pheromone on all the arcs that have been used by each ant. So, at time

step t, we found X solutions. At time step t+1 we have to update the pheromone trail that has been used in some solutions. We take the previous value and we introduce the 1-ρ with the evaporation coefficient that tells us how fast the pheromone evaporates. Furthermore we have another element: i adds the contribution of Δτ for the m ants that has used the $arc_{i,j}$.

ALGORITHM

```
while termination conditions not met do
        ScheduleActivities
                AntBasedSolutionConstruction()
                PheromoneUpdate()
                DaemonActions() {optional}
        end ScheduleActivities
end while
```

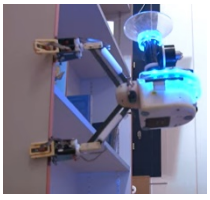
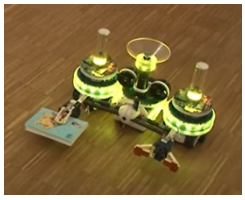
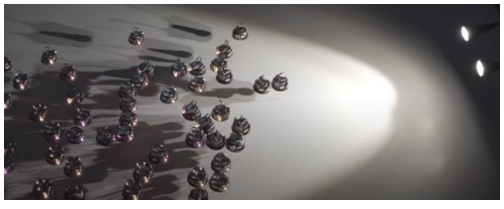
AntBasedSolutionConstruction() → ants move by applying a **stochastic local decision policy** that uses values of pheromone and heuristic on graph components. While moving, **ants take track of the partial solutions** (paths) **that have been built**.

PheromoneUpdate() → ants update the pheromone during the solution construction (**online step-by-step pheromone update**, but they can also update backward the pheromone on components used on the basis of the quality of the overall solution (**online delayed pheromone update**). **Evaporation is applied all time**.

DemonActions() → they are "**centralised" actions** that cannot be executed by the single ants. They can be **local search procedures** applied to each solution built or **collections of global information** to decide whether to leave additional pheromone to guide search from a global perspective.
We said there's no a central unit, but in reality, there are some cases in which it can be useful, e.g. we add pheromone to help ants or something similar. We can add actions that ruin and destroy a little bit the pure natural inspiration of the algorithm. **They're used** for example **for speed up the solution**, **to enforce in a way**, **to improve some local search moves**: in general, they're used to **converge in a faster way to the solution**.

Examples of real applications

- **EU Project: SWARM-BOT, SWARMANOID** (https://www.youtube.com/watch?v=M2nn1X9Xlps)
- **KILOBOTS** (https://www.youtube.com/watch?v=QXNVZJ3KUsA)

3.2.2 – HONEY BEE COLONY

The ARTIFICIAL BEE COLONY ALGORITHM (ABC ALGORITHM) is not so used, but it is an algorithm where agents are not all the same: there are different kinds of robots and each robot has its own capability and ability and he can **coordinate** with other to achieve a goal. **ARTIFICIAL BEES CAN BE OF THREE TYPES:**

- **EMPLOYED BEES:** associated with a specific nectar source
- **ONLOOKERS:** observing the employed bees choose a nectar source
- **SCOUTS:** discover new food sources

Initially, food sources are discovered by scout bees. Then food is consumed and the source exhausted. The employed bees in that source become scout.

Important ingredients for this algorithm are:

- **Food position**: **solution** (we have as many solutions as employed bees)
- **Food quantity**: **fitness**

In these strategies, very different from search tree strategies, we start from an assignment to variables and we try to apply local movement for exploring a neighbourhood of the solution: these are called **LOCAL SEARCH ALGORITHMS** and they use local optimum solutions. Every movement they do when they are in a local optimum, lead to a worst solution, so metaheuristic provides tools and techniques for escaping from local optimum. They move from a solution that cannot be improved trying to find something better. One specific kind of this strategies is, e.g. used in **population problems**.

The honey bee colony has a population based on meta-heuristic. In this case we have a dichotomy between food sources (that are solutions) and food quantity (that is a fitness of these solutions).
The algorithm starts with an initialization phase that can start throwing randomly these bees around the landscape. Then we make three phases, one for every type of bee.

ABC ALGORITHM

```
InitializationPhase()
repeat
        EmployedBeePhase()
        OnlookerBeePhase()
        ScoutBeePhase()
        Sol=BestSolutionSoFar
Until (Cycle=MaxCycleNum or MaxCPUtime)
```

InitializationPhase() → A set of food source positions are randomly selected by the bees and their nectar amounts are determined. Each solution Xm (m=1..Npopol.) is composed of n variables Xmi (i=1..n).
Each variable is subject to a lower and upper bound and initialized to **Xmi = lbi + rand(0,1) (ubi - lbi).**

EmployedBeePhase() → After sharing the information about the nectar amount, every employed bee goes to the food source area visited by herself at the previous cycle since that food source exists in her memory, and then chooses a new food source by means of visual information in the neighbourhood of the present one.
Fitness function of Xm: we use the objective function of the problem to compute the fitness as follows

$$ftn(X_m) = \begin{cases} 1/(1+obj(X_m)) & \text{se } obj(X_m) \geq 0 \\ 1+|obj(X_m)| & \text{se } obj(X_m) < 0 \end{cases}$$

OnlookerBeePhase() → An onlooker bee chooses a food source depending on the probability value associated with that food source.
They provide **positive feedbacks**.

$$p_m = \frac{ftn(X_m)}{\sum_{i=1}^{Npopol.} ftn(X_i)}$$

ScoutPhase() → Scouts chose nectar sources random. The employed bees that cannot improve the solution after a given number of attempts become scout and abandon the food source.
They provide **negative feedbacks**.

Termination condition: in meta-heuristic we have to select a time or a moment where to stop our search because **we don't have any guarantee that our solution is optimal**. We can decide to stop **after X iterations** or we can stop **after a CPUtime**. This is common to all Swarm Intelligence algorithms.

ABC APPLICATIONS
There's a way to design our algorithm having different kind of agents, that's why this algorithm is very important. It's not so efficient, but it is useful.

- **Training neural network**: optimization function that represents the mean squared error
- **Numerical Optimization**
- **Combinatorial Optimization**

3.2.3 – PARTICLE SWARM OPTIMIZATION (PSO)

PARTICLE SWARM OPTIMIZATION (PSO) has been proposed in 1995 by a psyco-social scientist James Kennedy and an electrical engineer Russel Eberart. The research activity starts from the **analysis of interaction mechanisms between individuals that compose the swarm**, particularly **interesting when the whole swarm has a common goal** such as food search.

The observation of rules that guide the **bird flock moves** shows that each individual entity has **driving trends**:

- **Follow neighbours**
- **Stay in the flock**
- **Avoid collisions**

With these rules it is possible to **describe and model the collective move of a flock with NO COMMON OBJECTIVE**. **PSO adds** a common objective: **food search**. With a common objective, a single individual that finds a food source has **two alternatives:**

- **Move away** from the group to **reach the food** (individualistic choice)
- **Stay in the group** (social choice)

If more than one individual entity moves toward the food other flock members do the same. Gradually the whole group changes direction toward promising areas. **The information propagates to all members**.

With PSO we can solve optimization problems with the following analogy:

- **INDIVIDUALS**: tentative configurations that move and sample the solution N-dimensional space
- **SOCIAL INTERACTION**: each individual agent takes advantage from other searches moving toward promising regions (best solution globally found)

Search strategy can be found as a balance between exploration and exploitation:

- **EXPLORATION**: individual agents that search for a solution
- **EXPLOITATION**: social behaviour which is the exploitation of other individual successful behaviour

NEIGHBOURHOOD

A feature that appears to be important is the **CONCEPT OF PROXIMITY**: **individuals are affected by the actions of other individuals** that are closer to them (**sub-groups**) and if they are part of more sub-groups, the spread of information is globally guaranteed.
Sub-groups are not tied to the physical proximity of the configurations in the parameter space but are **defined a priori** and may take into account also considerable shifts between individuals.

ALGORITHM

PSO optimizes a problem by setting a population (Swarm) of candidate solutions (particles) and it moves these particles in the search space through simple mathematical formulas.
The movement of particles is guided by the best position found so far in search space (from individual to population) **and it is updated** when better solutions are discovered.
f: $\mathbb{R}^n \rightarrow \mathbb{R}$ fitness or cost function to minimize: it takes a solution (vector) and produces a fitness value. The gradient of the cost function f is not known.
Goal: find solution "a" such that f(a) ≤ f(b) for all "b" in the search space.

Given:

- **S** number of particles in population
- Each particle has:
  - **a position xi ∈ $\mathbb{R}^n$** in the search space → it's a solution that has a given fitness and it is complete
  - **a speed vi ∈ $\mathbb{R}^n$**
- **p_i is the best solution** found so far **by particle i**
- **g is the best solution** found so far **by the entire swarm**

For each particle i = 1, ..., S do:

Initialize the particle's position with a uniformly distributed **random vector**: xi ~ U(b_{lo}, b_{up}), where b_{lo} and b_{up} are the lower and upper boundaries of the search-space.
Initialize the particle's best-known position to its initial position: pi ← xi
If (f(pi) < f(g)) update the swarm's best-known position: g ← pi
Initialize the particle's velocity: vi ~ U(-|b_{up}-b_{lo}|, |b_{up}-b_{lo}|), so again with a random vector

Until a termination criterion is met (e.g. number of iterations performed, or adequate fitness reached), repeat:
For each particle i = 1, ..., S do:

Pick random numbers: rp, rg ~ U(0,1)
Update the particle's velocity: vi ← ω vi + ϕp rp (pi-xi) + ϕg rg (g-xi)
Update the particle's position: xi ← xi + vi
If (f(xi) < f(pi)) do:

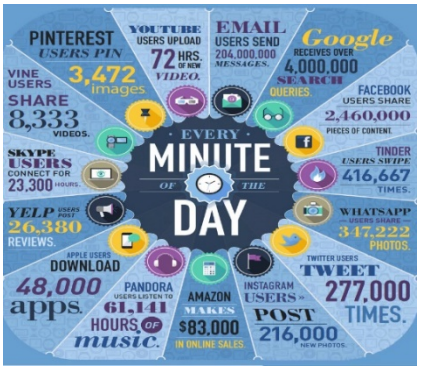
Update the particle's best-known position: pi ← xi
If (f(pi) < f(g)) update the swarm's best-known position: g ← pi

Return g: best found solution.

Parameters ω, ϕp, is ϕg should be carefully selected as they strongly influence the effectiveness and the efficiency of the PSO method.

4 – MACHINE LEARNING

4.1 – INTRODUCTION



4.1.1 – DEFINITIONS

Definition 1
Learning is constructing or modifying representations of what is being experienced [Michalski 1986].
Definition 2
Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task or tasks drawn from the same population more efficiently and more effectively the next time [Simon 1984].

Machine learning is one of the hot topics of Artificial Intelligence. All the most important ICT companies are investing money on ML and hire people with a background in ML: Google, Facebook, IBM, Baidu, Disney, …
Why? Because they have many data and want to extract value from data.

4.1.2 - THE ROLE OF ML

We have now a computing power very larger than 10 years ago, so, we have the possibility to analyse these data and apply different models. We have all the ingredients to exploit the full potential of these data.
So, the role of ML in the two definitions we've seen is for:

- **KNOWLEDGE EXTRACTION**: more related to description
  - For the operation of knowledge-based systems (for example, for classification systems)
  - For scientific purposes, for the discovery of new facts and theories through observation and experimentation
- **IMPROVEMENT OF PERFORMANCE ON A SPECIFIC TASK**: more related to actions
  - For example, improving the motion and cognitive abilities of a robot
  - For game playing

We have different techniques that can be roughly classified in three big classes:

- **SYMBOLIC TECHNIQUES:** different types of representation
  - **Value attribute representation**
  - **Representation of the first order**
- **STATISTIC TECHNIQUES**
- **SUB-SYMBOLIC TECHNIQUES/Neural Networks**

Symbolic techniques: it means that when we learn something from data, when we extract some knowledge, this **knowledge** is symbolic so it can be **understood by human expert**. This is a very important feature because in general a human expert that interacts with a system, wants to understand what have been extracted and wants to check if what extracted is meaningful or not.

Statistic techniques: this **relies on statistics method** for extracting figures from data, for example **regression** can be created through statistics.

Sub-symbolic techniques: here we can find **neural network**. They are **extremely effective**, but they have a **drawback**: the **model that we extract with these techniques is NOT understandable for a human expert**. So, a human cannot understand why it works so well and why it gives you that answer.

WE NEED TO LEARN A MODEL FROM DATA AND THEN GENERALIZE.

The generalization is fundamental because if we don't use it, we don't extract something new anymore from our data. **Our model should be applicable to data** that we've not seen so far, so this generalization capability is the core of machine learning.
e.g. kid that after having seen 3 dogs with 4 paws, he creates a mental model where all dogs have 4 paws.

4.1.3 - APPLICATIONS

- **Classify incoming e-mails as spam or not** → classification problem
- **Apple shares price prediction** → regression, because we're trying to predict a number
- **Diagnosis based on exams of a patient (IBM Watson Health)** → classification because we want to try to select the most probable diagnosis, we select a class that maps the symptoms
- **Understand VAT numbers** → which digits are those handwritten. This is a classification problem because we have to map the windows with a single digit to a class (e.g. 10 possible classes for numbers)
- **Text translation (Google translate)** → we take a word and we map it to a word in another language, so it's a classification problem
- **Teach a robot to grab a cup** → reinforcement learning
- **Design a molecule with specific properties** → related to a data set where the input variables are structured. There's a structure underline the molecules as a graph: **deep networks**. This is a **classification problem**
- **Convert a voice in text** → classification problem
- **Automatically write the caption to a figure** → we have an image and we have to understand what is in the image. An output is a class, so this is a classification problem
- **Beat the world champion in a game (AlphaGo)**

4.2 - ML TASKS

4.2.1 - CLASSIFICATION

Classification is the task of approximating a mapping function (f) from input variables (X) to DISCRETE/CATEGORICAL OUTPUT VARIABLES (y). Here we want to start from examples with faulty states and normal states and we have to learn a mapping function, a model that maps the input variable into the output variables.

Since we have to provide to the system both inputs and outputs, this is called **SUPERVISED LEARNING TASK**.

- The **output variables** are often **called labels or categories or classes**
- The **mapping function predicts the class or category for a given observation**

A classification problem requires that examples be classified into one of two or more classes:

- **Two classes** → two-class or binary classification problem
- **More than two classes** → multi-class classification problem

MULTI-LABEL CLASSIFICATION PROBLEM: an example is assigned to multiple classes.

A classification can have:

- **Real-valued input variables**
- **Discrete input variables**
- **Categorical input values** such as yes/no, low/high, red/blue/green, …

The **classification accuracy** is the percentage of correctly classified examples out of all predictions made.

4.2.2 - REGRESSION

Regression is the task of approximating a mapping function (f) from input variables (X) to a CONTINUOUS OUTPUT VARIABLE (y). A continuous output variable is a **real-value**, such as an integer or floating-point value. These are often quantities, such as amounts and sizes.

Here the output is a number and we don't have to select the class faulty or not faulty, but we can e.g. start from input variables that are numbers coming from sensors and have as an output the percentage of fault, so it's a number or e.g. we can predict the time to fault.
We have ALWAYS to predict a number, this is the main difference. This is again a **supervised learning task** because also in this case we have data that should be labelled, but we don't have classes, we have numbers.

Regression is a statistic problem with a real value as output. Again, we start from experience and we create a model on something that we have not yet seen.
A regression problem **requires the prediction of a quantity** and can have **real valued or discrete input variables**.
A problem with **multiple input variables** is often called a **MULTIVARIATE REGRESSION PROBLEM**, and if input variables are ordered by time is called a **TIME SERIES FORECASTING** problem.
Since a regression predictive model predicts a quantity, **the skill of the model must be reported as an error in those predictions**, like the RMSE.

4.2.3 - CLUSTERING

Clustering is the problem of organizing objects into groups whose members are similar in some way. It determines the intrinsic grouping in a set of unlabelled data and is the **main unsupervised learning task**.
Here we don't need to have labelled data, we don't need an expert human, but we need to group these data with **similarity measures**.

Clustering types:

- **Hard clustering**: each element can be **assigned to one and only one group**
- **Soft or fuzzy clustering**: each element can **belong to more than one group**

The most widely used algorithm for **hard clustering** is the **K-MEANS ALGORITHM [MacQueen 1967].** Often this is used as a baseline and it works pretty well.

K-MEANS

K-means defines a cluster by minimising the intra cluster variance. **Each cluster is identified by a centroid**.

1. **At the beginning** we create **k partitions** and we assign input data to one set of the partition randomly
2. Then the **centroid of each group is computed**
3. **A new partition is built** using the computed centroids
4. **New centroids are computed** on the basis of the partition **and so on until convergence**

4.4 - TYPES OF LEARNING

4.4.1 - SUPERVISED LEARNING

Supervised machine learning problem: find a mapping between input and output, from observations (data).
There are **different algorithms**, methods and problems that have different:

- **Input Space** X
- **Output Space** Y
- **Function Definition** f
- **Choices for computing** f

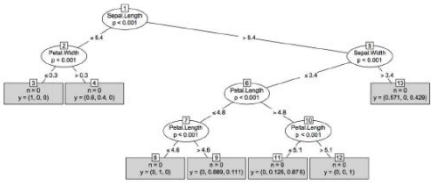
Schema:
Given **observations** $x \in X$
Prediction target $y \in Y$
data set $D = \{(x_i, y_i)\}_{i=1}^{N}$
Build a **function** $f : X \rightarrow Y$
f should predict the value of $\hat{y}$, given a new observation $\hat{x}$

Input space X:

- **Numeric/Symbolic Attributed**
- **Logic Representations**
- **Structured space** (trees, graphs, etc.) → e.g. molecules

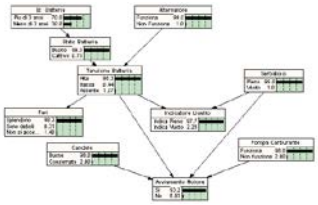
Output space Y:

- {±1} → **Binary Classification**
- {1, …, M} → **Multi-Class**
- R → **Regression**
- Structured space → **Structured Output**



SYMBOLIC TECHNIQUES

- **Extract logic rules**, e.g. father(X,Y):- parent(X,Y), male(X)
- **Extract decision trees**
- **Variants**



STATISTICAL TECHNIQUES

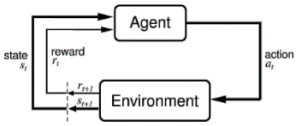
- **Bayesian technique**
- **Linear regression**
- **Non-linear regression**

4.4.2 - UNSUPERVISED LEARNING

In unsupervised learning we have only data and **no trainer**, we don't know classes. This approach is useful because **clustering data means:**

- **Finding frequent patters**
- **Identifying most important features**
- **Reduce data dimensionality (**encoding/compression)

4.4.3 - REINFORCEMENT LEARNING



This is a different paradigm: **teacher gives rewards/punishments**. **The agent learns a strategy that minimizes** (delays) **the punishment** as much as possible and **maximises the reward**.
A great example: computers playing Atari (by Google DeepMind).

4.4.4 - SUPERVISED VS UNSUPERVISED

SUPERVISED	UNSUPERVISED
Labelled data are very useful	Unlabelled data are everywhere
Function to be learned is well defined	Interesting info can be extracted from data
Label data is very expensive, long and boring	Worse performance on specific data

4.5 - QUALITY OF A ML SYSTEM

How to measure the quality of a ML system?

- **Performance measure**
- **Overfitting and cross-validation**

If we ask to ourselves the following 2 questions, we can notice that they're related to generalization:

- How the system be haves on example that it has seen?
- How the system behaves on example that it has not seen?

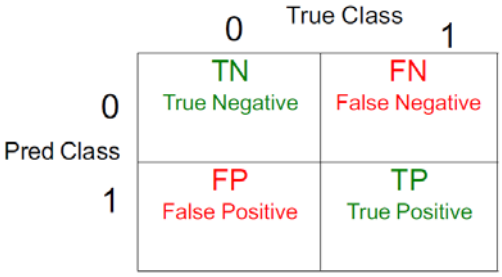
4.5.1 - PERFORMANCE MEASURES

Given a problem, how to measure performance of a ML system? We can't count the errors, it's too simplistic.

e.g. We are building a system that diagnoses a very rare pathology. Our classifier provides the right classification in the 99.9% of the cases. Are we happy with this classifier? No.
If data are not balanced, we need to know it because in this case we would have a problem because we're never foresee the proper class.
In the example we have 1000 people and 99.9% are not affected by the pathology, while 1 of them is affected.
In our data set we have many examples whose class is "no" and one of them that says "yes". If we create a classificatory predictor that always says "no" and we count the number of errors, the number of errors is 1/1000 which is very low. But we want to identify the 1/1000 person, so we're not happy about the result! **A classifier that always says "no" has certainly a low number of error but is completely useless.**

CLASSIFICATION

This performance measure of classification is easily **extendable to more than 2 classes** and is based on four parameters: **Accuracy, Precision, Recall and F-Measure**.
The False Negative FN is the worst because it seems that we don't have any problem, but we have of course.



$$A = \frac{TP+TN}{TP+TN+FP+FN} \qquad P = \frac{TP}{TP+FP}$$

$$R = \frac{TP}{TP+FN} \qquad F1 = \frac{2PR}{P+R}$$

REGRESSION

In order to measure the quality of regression, the most used performance measures are:

- **MSE: Mean Squared Error**
- **RMSE: Root Mean Squared Error**
- **MAD: Mean Absolute Deviation**

$$MSE = \sum_{i=1}^{N} \frac{(\hat{y}_i - y_i)^2}{N} \qquad RMSE = \sqrt{MSE} \qquad MAD = \frac{1}{N}\sum_{i=1}^{N} | \hat{y}_i - y_i |$$

4.6 - MODEL SELECTION

The performance of a machine learning system depends on the training phase that defines the parameter of a model. To select the best parameters **data are divided into:**

- **TRAINING SET** (o learning set): used to learn the parameters and the function f
- **VALIDATION SET:** used to measure performances during training
- **TEST SET:** used to measure performances after training on unseen data

In the **validation set and test set,** we have examples data that have exactly the same structure of the training set and they measure the performance of the system, but they are **unknown to the system**; we only know how the system should classify them.

4.7 – MODEL VALIDATION

The training of a ML system has the aim of minimizing a cost function.

- **POOR GENERALIZATION**: pay attention, because we can perfectly learn how to classify the training set, but perform trivial error on the validation/test set. In fact, if we have a perfect classifier on the training set but on other set has many errors, we're not happy with it. **We want a system is able to generalize!**
- **OVERFITTING**: generally, a learning algorithm is said to overfit relative to a simpler one if it is more accurate in fitting known data, but less accurate in predicting new data. In our domain, the cost function would be perfect for the training and the validation set, while on the test set would miserably fail.

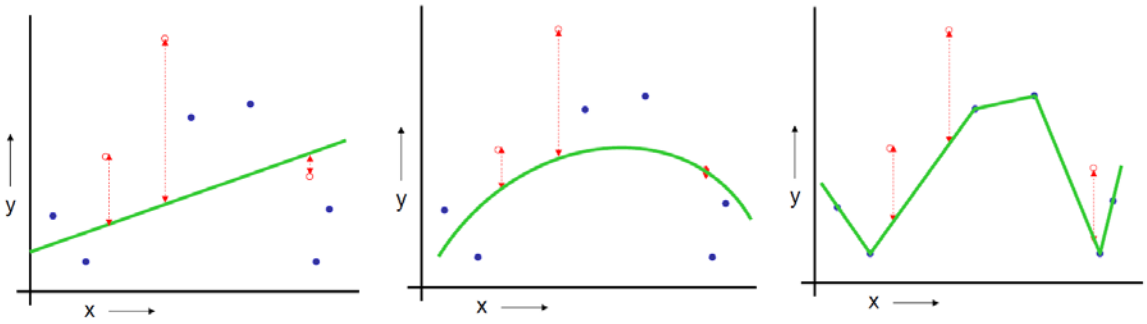
4.8 - ESTIMATING PERFORMANCES

4.8.1 – TEST SET METHOD

Let's consider the following regression problem: **y = f(x) + noise**. In order to learn f from the data in the graph, there are three methods:

- **Linear regression**
- **Quadratic regression**
- **Join the dots**: piecewise linear non parametric regression

Obviously, the function that best fits the data is the last one: it is perfect on the validation set. But now, let's try to use those functions with the test set:



Now, we make these steps:

- Randomly choose **30% of the data to be in a test set**
- The remainder is a training set
- Perform your regression on the training set
- Estimate your future performance with the test set

Consider that the **blue dots are the ones of the training set**, if we measure our future performance on the red points (the ones of the test set), we compute the errors with MSE and we obtain a score of:

- MSE = 2.4 for the Linear Regression
- MSE = 0.9 for the **Quadratic Regression** → this is the best cost function
- MSE = 2.2 for the Join the Dots

PROS	CONS
• Very simple • Choose the method **with the best test-score**	• **It wastes data**: you have to choose the best method to apply by using 30% less data • **If you don't have much data your test set might be lucky or unlucky** • **The test set estimator has a high variance**

4.8.2 – LOOCV METHOD

LOOCV means Leave One Out Cross Validation and is a particular case of leave-p-out cross-validation with p=1.

For k=1 to R
    Let (xk,yk) be the kth record
    Temporarily remove (xk,yk) from the dataset
    Train on the remaining R-1 datapoints
    Note your error (xk,yk).
When you've done all points, report the mean error.

In the previous example:

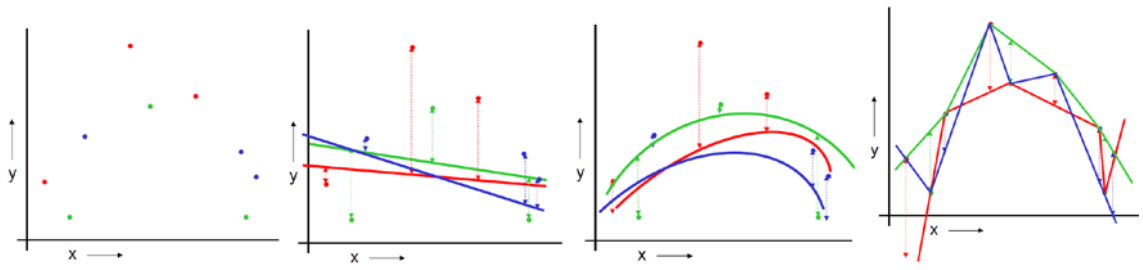
- Using linear regression leads to MSE=2.12
- Using quadratic regression implies MSE=0.962
- MSE=3.33 for the Join the Dots method

PROS	CONS
• **Doesn't waste data**	• **Expensive** • **Sometimes has a weird behaviour**

4.8.3 – K-FOLD CROSS VALIDATION METHOD

The K-FOLD CROSS VALIDATION method randomly breaks the dataset into k partitions (in this case k = 3):

- **For the red partition**: Train on all the points not in the red partition. Find the test-set sum of errors on the red points
- **For the green partition**: Train on all the points not in the green partition. Find the test-set sum of errors on the green points
- **For the blue partition**: Train on all the points not in the blue partition. Find the test-set sum of errors on the blue points
-



With this method, we obtain:

- MSE=2.05 for the linear regression
- MSE=1.11 for the quadratic regression
- MSE=2.93 for the Join the dots

	CONS	PROS
TEST-SET	Variance: unreliable estimate of future performance	Cheap
LOOCV	Expensive. Has some weird behaviour	Doesn't waste data
10-FOLD	Wastes 10% of the data. 10 times more expensive than test set	Only wastes 10% of data. 10 times more expensive instead of R times.
3-FOLD	More waste than 10-fold. More expensive than test set	Slightly better than test-set
R-FOLD	Identical to LOOCV	

4.9 – INDUCTIVE LEARNING

Inductive learning: the system **starts from observations** coming from a trainer or from the surrounding environment **and generalizes, gaining knowledge** that, hopefully, is also **valid for not yet observed cases** (induction). There are **two types of inductive learning:**

- **LEARNING FROM EXAMPLES**: the **knowledge is gained from a set of positive examples** that are instances of the concept to be learnt and negative examples which are non-instances of the concept
- **LEARNING REGULARITY**: there is not a specific concept to be learned, **the goal is to find regularities** (i.e. common features, **patterns**) **in data**

Examples of "learning from examples": e.g. the kid and the dog, e.g. I have data coming from sensors that represent a situation of faults or not faults. We have a concept on the normal behavior of the machine.
In general, we can have positive and negative examples of the concept.

Example of "learning regularity": e.g. clustering. We don't have any specific concept to learn, but we have to find common features in data.

4.9.1 – LEARNING FROM EXAMPLES

Given:

- **Universe U**: set of all domain objects
- **Concept C**: subset of objects in the domain $C \subseteq U$
- **A description language for objects Lo**
- **A description language for concepts Lc**
- **A procedure that interprets both languages and checks** whether the description Dc of the concept C is satisfied by the description Dx of an object x (Dc covers Dx).

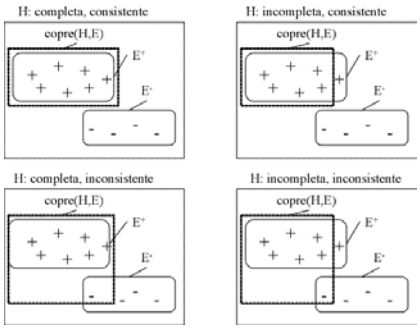
Learning a concept C means finding a description of C that allows to tell if an object $x \in U$ is an instance of C, i,e. if $x \in C$.

FACT: description of an object.
Example of a concept C: labelled fact. The label is:

- "+" if the object is an instance of C
- "–" if the object is not an instance of C

TRAINING SET: set E of examples (labelled facts). E is composed by all the positive examples E+ and all the negative examples E-.

Hypothesis: we have a description of the concept to be learned. If a fact satisfies the hypothesis, we say that the hypothesis covers the fact.
The function for the test coverage "covers (H, e)" returns true if "e" is covered by H, false otherwise.
Extension to sets of examples:
covers (H, E) = { e ∈ E | covers (H, e) = true }



LEARNING PROBLEM DEFINITION

Given a set E of positive and negative examples of a concept C, expressed in an object description language Lo, **find a hypothesis H**, expressed in a given concept description language Lc, **such that:**

- **Every positive example** e+ ∈ E is **covered** by H
- **No negative example** e- ∈ E- is **covered** by H

COMPLETENESS: A **hypothesis H** is complete if it **covers all positive examples** → covers (H, E+) = E+
CONSISTENCY: A **hypothesis H** is consistent if **does not cover any negative example** → covers (H, E-) = ∅

4.9.2 – KNOWLEDGE REPRESENTATION

Languages used for the representation of examples and concepts:

- **Attribute-value** languages
- **Relational** languages
- **First order** languages

The richer the language, the more complex the learning problem.

ATTRIBUTE-VALUE LANGUAGES

OBJECT DESCRIPTION LANGUAGE

Fixed number of attributes for each instance, each instance is described by values assumed by the set of attributes. Those attributed are predefined and we'll use them to represent our instance of the object. Attributes can be:

- Boolean or binary
- Nominal (high, medium, low)
- Ordinal (first, second, third…)
- Numeric (35.6, 7, 8.4)

If k are the attributes, each instance can be represented as a point in a k-dimensional space.

Equivalent to propositional logic:

- Any **equality or inequality** can be seen as a **proposition** (no variables and no terms)
- **There are no variables, quantifiers and predicates with arity > 1**

Example: universe of athletes. Instances described by attributes: height, weight, body-type.
A possible instance is: height = 1.85m, weight = 110kg, body-type = robust
So, height, weight and body-type can be seen as predicates, and "robust" and the other values are the parameters.

CONCEPT DESCRIPTION LANGUAGE

We can learn production rules or decision trees**. In production rules we have antecedent and consequent and the consequent is the concept we want to learn**.

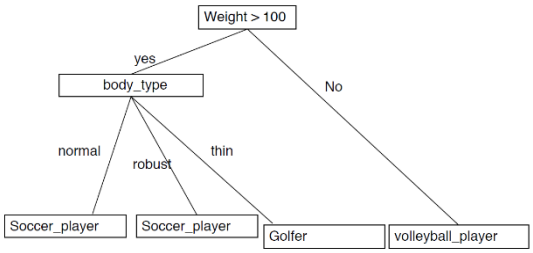
PRODUCTION RULES:

- **Antecedent**: conjunctions and disjunctions of equalities/inequalities between an attribute and a value
- **Consequent**: the concept (class)

Example with concept "soccer player". Example description of the concept:
weight > 100 and (body-type= normal or body type = robust) → soccer_player
We are trying to understand with ML the conjunctions and the disjunctions that define a soccer player.

DECISION TREES are fully equivalent to production rules:

- **Each node corresponds to a test on an attribute** (equality, inequality)
- **Each branch** that starts from the node is labelled with the test result
- **Each leaf corresponds to a class (concept)**



RELATIONAL LANGUAGES

OBJECT DESCRIPTION LANGUAGE

Attribute-value languages are not suited to represent instances composed of subparts.

Example: jones family, components:	name: dave,	son: mike,	father: ron
	name: mike,	son: junior,	father: dave
	name: junior,		father: mike

The description of the family could be transformed into a list of attribute value pairs providing a number of attributes equal to the product of the maximum number of components for the maximum number of attributes per component: **waste of memory!**

OBJECT AND CONCEPT DESCRIPTION LANGUAGE

The attribute-value languages are therefore inefficient to represent these universes. **Relational languages** are used instead, because they **allow descriptions of concepts that may contain variables and quantifiers** (non-propositional languages).

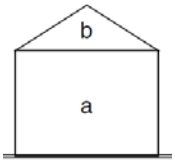
The description of the family concept with a grandparent is: ∀x,y,z (son(x) = y and son (y) = z)

FIRST ORDER LANGUAGES

OBJECT DESCRIPTION LANGUAGE

First-order logic languages are:

- **Very rich**
- **Objects** are described as **ground facts**
- **Each attribute** of a component **becomes a predicate**.
- We can express also **relations between components** of the same object



Example of "object":

Instance in attribute-value form: components: name: a, shape = square, size = large, on-table = yes
name: b, shape = triangle, size = small, on-table = no

Instance in first-order logic: object(s, a), object(s, b), square(a), triangle(b), large(a), small(b), on-table(a) on(b,a) → relation between parts

CONCEPT DESCRIPTION LANGUAGE

Concepts are described through logic clauses and if compared to relational languages, **they also allow**:

to define the concepts recursively: ancestor(X, Y):- father(X, Z), ancestor(Z, Y). This is a recursive clause, but starting from production rules and ending in first order clauses, there's a **huge gap**. We have many problems when we are studying these clauses.

They are a **deeply studied and formal language**. Use of logic programming for the representation of objects and concepts: **INDUCTIVE LOGIC PROGRAMMING.**

4.9.3 – LEARNING TECHNIQUES

- **Learning from value-attribute objects**:
  - Decision trees
  - Production rules
- **Learning from first order logic facts:**
  - Inductive Logic Programming

DECISION TREES LEARNING

They are **really intuitive** and **extremely powerful**. They are at the **basis of the large majority of symbolic ML**.

Systems that learn decision trees: CLS, IDR, C4, ASSISTANT, ID5, **C4.5** etc.

Appropriate problems:

- **Instances are represented by attribute value pairs**
- **The target function has discrete values**
- **Disjunctive descriptions of concepts** may be required
- **The set of training data may contain errors** and/or **missing data**

Suppose that the sensor from which we are taking data is disconnected or broken:

- If it is broken, we have errors
- If it is disconnected, we don't have data at all

I don't want to spare data, I want to keep them all and **distinguish the ones that are wrong and the ones that are missing.**

Another important feature is that **decision trees learning can help us to identify the attributes that are representative, significant, for our classification tasks.**
E.g. Suppose that in our 100 sensors values, only 10 of them are significant for a given kind of fault. Decision trees identify and select among our attributes, only the ones that are needed for the classification. All the others, are simply useless. This is an extremely useful capability of the decision tree.

TREE GENERATION ALGORITHM

Given a training set T (set of examples) and a set of classes {C1, C2, ..., Ck}, consider the set T:

- If T contains one or more examples, all belonging to the same class → **single leaf labelled with the class**
- T contains examples that belong to multiple classes → **partition T in subsets** according to a test on an attribute. We have a **node associated with the test, with a sub-tree for each possible test result**. **Recursively call the algorithm** on each node created by the partition.
- If T contains no example (**empty set**) → **single leaf labelled the class more frequently in both fathers**

TERMINATION CONDITION
In principle, the **algorithm stops when all leaves contain homogeneous examples**. Indeed, we have less tight termination conditions.
C4.5 does stops even if:

- **No test exists** such that at least two subsets contain a minimum number of cases
- **By default, the number of cases is 2**

ATTRIBUTE CHOICE
How to choose the test (attribute) at every step? Generation of the smallest possible trees is **computationally intractable**. It is then **chosen based on a greedy heuristic** (**non backtrackable**).
The choice is designed to **minimize the depth of the final tree**:

- **The perfect attribute divides examples into sets that are all positive or all negative**
- **A useless attribute** leaves the example set with roughly the **same proportion of positive and negative examples** as the original set

A small decision tree avoids overfitting and generalizes most, that's why we want smallest trees. We don't want to classify the training set, the real aim of the decision tree is to classify other real examples, not the training set. So, **THE SMALLER THE DECISION TREE IS, THE HIGHER IS THE CAPABILITY OF GENERALIZATION.**

We need a **formal measure of the attribute "performance":**

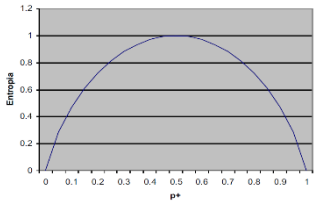
- The measure should have **its max with perfect attributed** and **min with useless ones**
- Takes **inspiration from information theory**, i.e. the amount of **information provided by an attribute**
- The **proportion of positive and negative example** in a set T is a **good estimation** of this:
    - How much information we still need after the attribute test (the lower the better)
    - Based on the **concept of Entropy**

ENTROPY OF A SET OF EXAMPLES
Given the set of samples S and the class Cj the entropy of the training set info(T) is:

$$info(S) = -\sum_{j=1}^{k} \frac{freq(C_j, S)}{|S|} \times \log_2 \left(\frac{freq(C_j, S)}{|S|} \right)$$

It can also be seen as a **measure of non-uniformity of an arbitrary collection of examples: the lower the better.**

Example: in the case of only two classes, p+ and p- are the proportion of positive and negative examples in the training set (p- = 1-p+). So, the entropy is given by:



$$info(T) = -p^{+} \times \log_2 p^{+} - p^{-} \times \log_2 p^{-}$$

The entropy is minimum when all the examples in T belong to the same class:

p+= 0 (or p+= 1) → info (T) = 0 because 0 * log2 0 = 0

The entropy is maximum when half of the examples belong to a class, and the other half to the other class:

p+= 0.5 → info (T) = 1

GAIN OF AN ATTRIBUTE CHOICE

The gain of an attribute X is given by the **difference between the original information and the one after the partition on X**. Entropy after partitioning (according to the X test): **weighted average of the entropies of the individual sub-units.** The information gain is gain(X).

$$info_X(T) = \sum_{j=1}^{n} \frac{|T_i|}{|T|} \times info(T_i) \qquad gain\ (X) = info\ (T) - info_X(T)$$

TEST GENERATION

Problem: generation of the set of possible tests. A priori definition of the allowed forms.

Three possible types of tests:

- **Discrete attribute**: a result for each **value**
- **Discrete attribute**: a result for each **group** of values
- **Continuous attribute**: two possible outcomes (binary test).

Additional constraint: at least two of T1, T2, ..., Tn must contain a minimum number of examples.

TEST ON DISCRETE ATTRIBUTES

When testing on discrete attributes, **we can have:**

- **A result for each value** found in the training set
- **A result for each group of values**: we must determine how many and which groups to consider
  - **Domain knowledge**: a priori identify significant groups (new values)
  - **Testing of all possible partitions** of the set of values

TEST ON CONTINUOUS ATTRIBUTES

A continuous attribute A takes m distinct values in T {V1, V2...., Vm} ordered from the smallest to the largest.

Select as **threshold Z = Vk** that splits the m values into two groups:

- **A ≤ Z** {V1, ..., Vk}
- **A > Z** {Vi + 1, ..., Vm}

So, we have **m-1 candidates: V1, ..., Vm-1.** We evaluate each of the m-1 candidates in terms of entropy.

e.g. if for the temperature we have 75, 80, 85, 64, 71, 81, ... these are the distinct numbers. For humidity we can have other different values. We have to collect all the different values, then we check whether each of these values is between V1 and Vm-1, setting the proper threshold.

We have to find examples that are homogeneous in terms of the class they belong to, it is not related to dimensions. **In general, we prefer unbalanced trees but more accurate.**

ATTRIBUTES WITH MISSING VALUES

In many domains not all attribute values are known for every example. How to compute the gain? We cannot compute the entropy based on the distribution of the values, because we don't have those values.

Gain: we call **F⊆T the subset of T for which A is known** and **X is a test on A**: info(T) = info(F), infoX(T) = infoX(F). The test of A for examples in T \ F does not provide any information on the class.

Gain (X) = (Probability that A is known) x (info (T) - infoX(T)) + (Probability that A is unknown) x 0 =

$$= \frac{|F|}{|T|} \times (info(F) - info_X(F))$$

PARTITIONING

Probabilistically generalized: each instance is associated with a weight w initially set to 1

Consider a test on A:

- **If an example e, with weight w in T, has A = Vi (known)**, assign it to Ti with weight w and to Tj (j≠i) with weight 0
- **If an example e, with weight w in T, has A unknown** we partition it in each Tj with a weight of w*Pvj where Pvj is the probability of the value Vj in T

Pvj can be estimated as: the sum of the weights of the instances in T that have A = Vj divided by the sum of all the weights of the instances in T that have A known.

OUTPUT INTERPRETATION

Numbers (A, B) associated with each leaf:

- **A** = total number of **training examples sets associated to the leaf**
- **B** = number of **misclassified examples** of the training sets associated to the leaf

e.g. N (3.4 / 0.4) means that 3.4 cases belong to the leaf of which 0.4 does not belong to the class N.

CLASSIFICATION OF NEW CASES

The main purpose of any ML model is to be able to classify unseen examples. This is easy for examples with all attributes known. **Classification of an example e with A unknown:**

- **Initially assign e weight w=1**
- **If a node with a test on A is found**, **we partition** it in all possible sub-branches. In each sub-branch for Vj assigning e the weight Pvj
- **We eventually reach more leaves:**
    - **We obtain a distribution of classes instead of a single class**. The probability of each class is the weight of the examples that reached the leaf.
    - **If two leaves are associated to the same class C, the probability of C is given by the sum of the probabilities of the example weights in the two leaves.**

EVALUATION OF A DECISION TREE

A learning algorithm is good if the produced hypotheses are good at predicting the classification of unseen cases. The steps we have to do are:

1. **Divide the example set** into two **disjoint sets**: the **training set** and the **test set**.
2. **Design the decision tree on the training set**
3. **Measure the percentage of examples in the test set that are correctly classifies**
4. **Repeat the above steps** with different sizes of training sets that are randomly selected

NOISE AND OVERFITTING

In overfitting, a learnt model describes random error or noise instead of the underlying relationship. Overfitting occurs when a **model is excessively complex**, such as having too many parameters relative to the number of observations. **A model that has been overfit has poor predictive performance** because it overreacts to minor fluctuations in the training data.

Decision trees suffer a lot of overfitting. They are extremely good at predicting and classifying but they have poor predictive performances on the test sets because in the test set there are new examples.

Two ways to prevent overfitting:

- **Tree pruning**: removes a posteriori splits on irrelevant attributes
- **Cross-validation**: repeatedly split the known data in training and test set with the results averaged. This approach is a little more tricky if we do classification

C4.5 ALGORITHM

c4.5 [Qui93b, Qui96], designed by Quinlan is an **algorithm for learning decision tree**s, ranked #1 in the Top 10 Algorithms in Data Mining preeminent paper published by Springer LNCS 2008.

- **Evolution of ID3** by the same author
- **Inspired by one of the first systems: CLS** (Concept Learning Systems) by E.B. Hunt
- **J48 is an open source Java implementation of the C4.5 algorithm** in the Weka data mining tool

Remarks:

- C4.5 performs a **hill-climbing search** in the **space of all possible decision trees.**
- **The space** of possible decision trees is **equivalent to powersets of all possible examples:** too large
- **C4.5 only keeps a single case when searching.** It performs a commitment each time it selects a test attribute.
- **C4.5 does not perform backtracking**. Therefore, it runs the **risk of incurring a locally optimal solutions**
- **C4.5 uses all training examples at every step** to decide how to refine the tree. On the contrary, other methods make decisions incrementally based on individual examples.
- **The search of C4.5 is less sensitive to errors in the individual examples**.
- **C4.5 selects informative features out of the training set**

DECISION TREES VARIANTS

Based on outcome:

- **Classification trees**: predicted outcome is a class (like the example of Saturday morning)
- **Regression trees**: predicted outcome is a real number. This is the same as classification trees, but the heuristic is different. **In the leaves we don't have a class, but we have a probability, a real number.**
- Some similarities and differences (ex. procedure to determine where to split)

Ensemble methods: construct more than one tree. We build multiple decision trees with separating training sets, then we keep them all and we use them all for example to compute the majority or we build an aggregate result of these trees. **The most known algorithms are:**

- **Bagging trees**: builds multiple decision trees by repeatedly resampling training data with replacement, and voting the trees for a consensus prediction
- **Random Forests**: uses a number of decision trees in order to improve the classification rate.
- **Boosted trees**: used for classification and regression

Random Forests and Boosted Trees are used, but we lose something: we have some results, like "yes you can go playing tennis", but there's no the raison why. **We lose the information about which are the most important features** that enable us to make a classification.
They largely overcome the problem of overfitting, but **we lose the explainability** and the **selection of the most salient features** for classification.

5 – BAYESIAN CLASSIFICATION

The Bayesian learning is a statistical technique of supervised learning (chapter 4.4.1). Select, given a certain pattern x, the class ci which most likely predicts the pattern:

$$P(C = c_i \mid x) > P(C = c_j \mid x) \qquad \forall j \neq i$$

BAYES THEOREM (BT):

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

Bayesian learning provides a method to calculate the probability of a random event A, while a priori knowing the occurrence of an event B. In classification:

$$P(c_i|x) = \frac{P(x|x_j)\,P(c_i)}{P(x)}$$

Since we want to know the class cj that maximizes P(cj|x), we need to find the class cj that maximizes P(x|cj)P(cj).

Example
Instances described by one attribute, two classes: "the patient has hepatitis" and "the patient has not hepatitis". Attribute: result of a exam to the liver. Two possible values: + (hepatitis) and - (no hepatitis).
From the lab results we know:
P(hepatitis) = 0.008 P(no hepatitis) = 0.992
P(+ | hepatitis) = 0.98 P(- | hepatitis) = 0.02
P(+ | no hepatitis) = 0.03 P(- | no hepatitis) = 0.97
We observe that, according to a lab exam, a new patient's result is + and we want to classify it (has he hepatitis or not?). Since for BT (leaving out the denominator) we have:
P (+ | hepatitis) * P (hepatitis) = 0.078
P (+ | no hepatitis) * P (no hepatitis) = 0.298
We conclude that it is more likely that the patient has not the hepatitis.

In case where the instance x is described by more than one attributes, we find the class Cj such that
P (cj | X1=x1, X2=x2, ..., Xn=xn) is maximum, i.e we want to make a belief revision.
For brevity we write that P (ci| x1, x2, ..., xn). By applying BT, this is equal to find the class that maximizes:
P (x1, x2, ..., xn | ci) x P (ci).

In the case of n attributes, the problem is to estimate P (x1, x2, ..., xn | ci) from the available data.
Direct approach: P (x1, x2, ..., xn | ci) is given by the number of instances of the class ci equal to x1, x2, ..., xn that appear in the data divided by the number of instances of the class ci.

Problems:

- In order to compute P (x1, x2, ..., xn | ci), the instance x1, x2, ..., xn should appear more times in the data → **we need a lot of data**
- **we cannot classify instances not seen** or not present in the training data → **no generalization**!

In fact, if we have multiple attributes, and every variable has a value, we need to find in the table the tuples that have exactly those values.

To overcome these problems, we have to use the **simplifying assumptions** that the **observed attributes are independent (Naive Bayes assumptions) given the class.**
If a and b are independent, given the class, the likelihood that a and b occur simultaneously, given c, is
P(a,b|c) → P(a|c) x P(b|a,c) = P(a|c) x P(b|c).

CLASSIFICATION NAÏVE BAYES

Naive Bayes Method: using the Bayes theorem making the assumption of independence of the attributes.
In our case P (x1, x2, ..., xn | ci)= P (x1| ci) x P (x2| ci) x ... x P (xn | ci).
So, with the Naive Bayes method the **highest probability is assigned to the class c** obtained by the following formula:

$$c = \underset{c_j}{\operatorname{argmax}} P(c_j) \prod_{k=1}^{n} P(x_k \mid c_j)$$

The parameters are calculated by the relative frequency:
P (ci) = proportion of examples of the training set that belong to ci
P (xk| ci) = number of examples in the training set belonging to class ci and that have the k-th attribute equal to xk divided by the number of examples in the training set that belong to the class ci

No	Outlook	Temp	Humid	Windy	Class
D1	sunny	mild	normal	T	P
D2	sunny	hot	high	T	N
D3	sunny	hot	high	F	N
D4	sunny	mild	high	F	N
D5	sunny	cool	normal	F	P
D6	overcast	mild	high	T	P
D7	overcast	hot	high	F	P
D8	overcast	cool	normal	T	P
D9	overcast	hot	normal	F	P
D10	rain	mild	high	T	N
D11	rain	cool	normal	T	N
D12	rain	mild	normal	F	P
D13	rain	cool	normal	F	P
D14	rain	mild	high	F	P

Example
Given a day with the following characteristics:
G=<Outlook=sunny, Temp=cool, Humid=high, Windy=T>
we want to know whether or not to play tennis.
Here we don't have numbers, we don't have numerical values. So, the first thing that we have to do is to **remove numbers and group them in pre-processing into as many groups as we wish**, but they should be non-numerical or, at least, non-continuous numbers.
We do not calculate the entire matrix, we calculate only the probabilities that we need:
P (Class=P) = 9/14 = 0.64 P (Class=N) = 5/14 = 0.36
P (Outlook=sunny | Class=P) = 2/9 = 0,4 P (Outlook=sunny | Class=N) = 3/5 = 0.6
P (Temp =cool | Class=P) = 3/9 = 0.333 P (Temp =cool | Class=N) = 1/5 = 0.2
P (Humid =high | Class=P) = 3/9 = 0.333 P (Humid =high | Class=N) = 4/5 = 0.8
P (Windy T = | Class=P) = 3/9 = 0.33 P (Windy T = | Class=N) = 3/5 = 0.6
P (Class=P) *P (Outlook=sunny|P) *P (Temp = cool |P) * P (Humid =high |P) *P (Windy T =|P) = **0.0052**
P (Class=N) *P (Outlook=sunny | No) * P (temp =cool | No) * P (Humid =high | No) * P (Windy T =| No) = **0.0207**

The more 'probable class' is N, so, with the new instance G, we calculate the probability of classes:
P (Class= P | G) = 0.0053/ (0.0053+ 0.0206) **= 0.205**
P (Class= N | G) = 0.0206/ (0.0053+ 0.0206) = 1-0.205 **= 0.795**

THINGS TO UNDERLINE:

- We need to have a **training set very similar to the one of decision trees but with no numerical values**. This is our universe of observation.
- This is a batch method; **we have to look to the overall training set** because we have to count the frequencies.
- **We can build a table with all these probabilities** that are precomputed and use this table for classifying our new instances
- It is always worth trying to **protect our training set** because it can contain sensitive data, so we need to find a way of classifying new sets without accessing our training set. **We can throw away our training set and we can keep the tree**: in this way, nobody can reconstruct the exact original data
- If the data are not very sensitive, we can also use the training set but it is not a very nice way to proceeding

6 – INDUCTIVE LOGIC PROGRAMMING (ILP)

6.1 – TERMINOLOGY

We've seen machine learning that starts with an initial training set. With the training set that we've seen, objects are described as sets of attributes values. Then, we saw the decision tree, so now we should have understood that this is the concept language that we use: this is our way for describe objects and to classify them in specific classes. If the outlook is sunny and humidity is less than 75, then the class is P and so on... Now, we move to a **completely different framework that is much more powerful in term of expressiveness**.

With **ILP** we **start from a knowledge**, something that we know before starting our ML algorithm. In ILP we have all the **power of the first order logic** and we have **more expressivity than decision trees**.
But the **disadvantage** is that this **approach is more complicated** because **we need more specific information**. Furthermore, here **we deal with recursive loops** and we have to take care.

ILP is the learning sector employing logic programming as a representation language of the examples and concepts.

Formally, given:

- a set $\mathbb{P}$ of possible programs
- a set E+ of good examples
- a set E- of Negative examples
- a consistent logic program B, such that B |= e+, for at least one and^{+} $\in IS^{+}$

Find: a logic program P $\in \mathbb{P}$ such that

- $\forall$ e+ $\in$ E+ $\rightarrow$ B, P |= e+ **(P is complete)**
- $\forall$ e- $\in$ E- $\rightarrow$ B, P |= e- **(P is consistent)**

- **B: background knowledge** is a priori knowledge, predicates of which we already know the definition
- **E+ E- constitute the training set**
- **P target program**
- **B, P is the program** obtained by adding the P clauses after those of B
- **$\mathbb{P}$: hypothesis space,** defines the **search space**. The **description of such hypothesis space** takes the name of **LANGUAGE BIAS**.
- **"B, P |= E"** means "P covers e"

With decision trees we search for all possible decision trees, **we do not explore the set of the decision trees exhaustively because it's too large**, but we use the proper decision tree (reasonably small) that covers some examples. **Here, we work with ALL possible programs and the set of this possible programs is not finite.**

Consistent and complete logic problem: we want to find a new P such that covers all positive examples and don't cover any negative example.

Example: Learning of the predicate father.
Given:

- **$\mathbb{P}$: Logic programs** containing **clauses father (X, Y): - τo** with τo literal conjunction chosen from
- parent(X, Y), parent(Y, X), male(X), male(Y), female (X), female (Y)
- B = {parent (john,mary), male(john), parent(david,steve), male(david), parent(kathy,ellen), female(kathy)}

This is our background knowledge. There's a relation between john and mary, and we know that john is parent of mary. We know also that john is a male and so and so forth. **Here we have a CWA,** but we have to relax this CWA because we want to enlarge the B with a P.

Father(john, mary) is a consequence, so is a good example to add in E+. Let's consider father(david, steve): this is a goal we have to prove, we unify parent(david, steve), than we see that this is true in the background knowledge, same thing for the male, so this is another example to add in the E+ set.

Parent(Kathy, ellen) → true, male(Kathy) → false. So, in this case we apply the CWA: if male(Kathy) doesn't appear, we know that she's not a male, so this is a negative example.
For father(john, steve) we fail because we don't have parent (john, steve), another negative example.

So the two sets are:
E+ = {father (john, mary), father (david, steve)}
E- = {father (kathy, ellen), father (john, steve)}

Find a program to compute the predicate father that is consistent and complete with respect to E+, E-
Possible solution: father (X, Y): - parent (X, Y), male(X).
This can be translated: father(X,Y) OR not parent(X,Y) OR not male(X) → **the set of literals that compose our clause are father, parent and male.**

6.1.1 - GENERALITY RELATION

The systems learn a theory by searching in the space of clauses generated using the language bias. In ILP the space is subject to a **generality relation θ-subsumption** [Plo69]: the clause C1 θ-subsumes C2 if there exists a substitution θ such that C1θ ⊆ C2.

- **A clause C1 is at least as general as clause C2** (and we write C1 ≥ C2) if C1 θ-subsumes C2.
- **C1 is more general than C2** (C1 > C2) if C1 ≥ C2, but not C2 ≥ C1.

We really want to start from the examples and then we want to infer a description of the concept that is generalized. If we don't generalize, machine learning is useless.
So, we need to add to our background knowledge our examples, but this make us not able to generalize! The thing that is important here is **trying to find the most general clause and for doing that we have to understand what does it mean being more general.**

Example
C1 = grandfather (X, Y) ← father (X, Z).
C2 = grandfather (X, Y) ← father (X, Z), parent (Z, Y).
C3 = grandfather (john, steve) ← father (john, mary), parent (mary, steve).

C1 = not father OR grandfather
C2 = not father OR not parent OR grandfather
C3 = not father OR not parent OR grandfather
"grandfather" and "not father" are included in the set of C2, and C1 is smaller than C2, so it is more general.

C1 θ-subsumes C2 with empty substitution θ = ∅
C1 θ-subsumes C3 with the substitution θ = {X / john, Y / steve, Z / mary}.
C2 θ-subsumes C3 with the substitution θ = {X / john, Y / steve, Z / mary}.

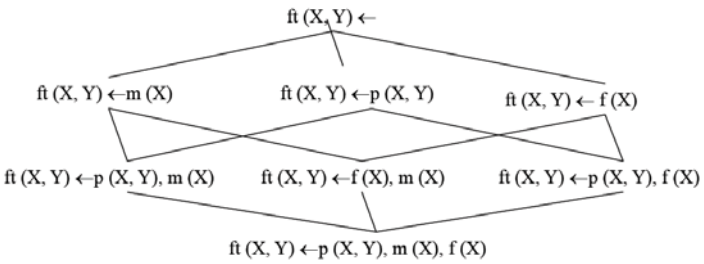
PROPERTY OF θ-SUBSUMPTION

- **If C1 θ-subsumes C2, then C1 | = C2. Vice versa is not true.**
- **It induces a lattice in the set of clauses**

θ-subsumption has the important property that: **if C1 θ-subsumes C2, then C1 | = C2. Vice versa is not true.**
For this reason, θ-subsumption is used to represent the generality relation.

There θ-subsumption has another important property: **it induces a lattice in the set of clauses.** This means that **each pair of clauses has a least upper bound (lub) and a greatest lower bound (glb).**

θ-SUBSUMPTION LATTICE



The top of the lattice is the most general clause.
Looking at the second level, we have to remove everything from the body to obtain a fact. If we have an empty body, this is always true.
In the **bottom** of the lattice, we have a **lower bound** for all the members of the lattice itself, and it's **the most specific clause**.

6.2 – ILP ALGORITHM

The algorithms for ILP are divided into bottom-up and Top-down approaches [Ber96] **depending on whether they use generalization or specialization operators:**

- **BOTTOM-UP ALGORITHMS start from the examples** (specific or bottom) **and generalize them getting a theory that covers them**. They use of generalization operators obtained by reversing the **deductive rules of unification, resolution and implication**
- **TOP-DOWN ALGORITHM** learns programs by **generating the clauses sequentially** (one after the other). The generation of a clause starts **from the most general clause** (with empty body) **and specializing it** (by applying a specialization operator or refinement) **until it no longer covers any negative example**

6.2.1 – TOP-DOWN ALGORITHM

TopDown algorithm (E, B, H): given the training set E and the background knowledge B, it returns H.

```
H: = 0    Ecur: = E
repeat / * covering loop * /
        generateClause (Ecur, B, C)
        Add the clause to the theory H: H=∪ {C}
        Ecur: = Ecur- covers (B, H, Ecur)
        // Removes from Ecur positive examples covered by H
until no positive examples are left
```

SPECIALIZATION LOOP

GenerateClause(E, B, C): given the training set E and the background B, it returns the best clause C.

```
Choose a predicate to learn
C: = P (X): - true.
repeat / * specialization loop* /
        Find refining Cbest Îr(C) using a heuristic
        C: = Cbest
until no negative examples are covered
```

HEURISTICS

The simplest heuristic is accuracy: A (C) = n^+(C) / (n^+(C) + n^-(C)) where n^+(C) n^-(C) are, respectively, the positive and negative examples covered by the clause C.
Another heuristic and informativeness, defined as: **I(C) = - log2(n^+(C) / (n^+(C) + n^-(C)))**
Other heuristics are:

- **The accuracy of gain: GA (C', C) = A (C') - A (C)**
- **The information gain: GI (C', C) = I (C') - I (C)**

Example
$\mathbb{P}$: Father (X, Y): - α with α ={Parent (X, Y), parent (Y, X),male(X), male (Y), female (X), female (Y)}
B = {parent (john, mary), male (john), parent (david, steve), male(david), parent (kathy, ellen), female (kathy)}
E+= {father (john, mary), father (david, steve)}
E-= {father (kathy, ellen), father (john, steve)}

The P with parent(X,Y) covers E+ but also the first negative example of E-, so we need to add another literal!

- 1st specialization step**: father (X, Y): - true**. It covers E+ but also E-.
- 2nd specialization step: **father (X, Y): - parent (X, Y).** It covers E+ but also one E-: father (kathy, ellen).
- 3rd specialization step: **father (X, Y): - parent (X, Y), male(X).** It covers E+ and any E-.

The covered positive examples are removed, E+ becomes empty and the algorithm terminates generating the theory: father (X, Y): - parent (X, Y), male(X).

6.2.2 – BOTTOM-UP ALGORITHM

The bottom-up algorithms are **based on the concept of least general generalization (lgg).**
The lgg of two clauses C1 and C2 is a clause C that:

- **Is more general than C1 and C2**
- **Is the most specific among the most general clauses C1 and C2**

In the bottom-up methods, **the clauses are generated starting from the most specific clause covering one or more positive examples and no negative example and repeatedly applying generalization operators** to clause until it cannot be further generalized without covering negative examples.
Typically, there is also a further step in which you eliminate literals in the body of the generated clause until the clause does not cover the negative examples.

LEAST GENERAL GENERALIZATION (lgg)

θ-subsumption has the important property that introduces a lattice in the set of clauses. This means that each pair of clauses has a least upper bound (lub) and a greatest lower bound (glb).
DEFINITION OF THE LGG - LEAST GENERAL GENERALIZATION [Plo69]: The least general generalization of two clauses C1 and C2, denoted by **lgg (C1, C2), is the least upper bound of C1 and C2 in the lattice of the θ-subsumption.**

ALGORITHM FOR IGG

TWO TERMS
The lgg of two terms f1(s1,..., sn) and f2(t1, ..., tn) is:

- **If f1 = f2: f1(lgg (s1, t1), ..., lgg (sn, tn))**
- **Else: a new variable V**

Sa same variable is used to generalize the same terms.

Examples:
lgg (f (a, b, c), f(a, c, d)) = f(a, X, Y)
lgg(f(a,a), f(b,b)) = f (lgg(a,b), lgg (a, b)) = f (X, X)

TWO LITERALS
The lgg of two literals L1 = (~) p (s1, ..., sn) and L2 = (~) q (t1, ..., tn)

- **is undefined** if L1 and L2 do not have the same predicate symbol and the same sign
- **otherwise it is defined as** lgg (L1, L2) = (~) p (lgg (s1, t1), ... lgg (sn, tn))

If p=q we use the sign of both, the predicate symbol of both and they have the same arity.

Examples:
lgg (parent (john, mary), parent (john, steve)) = parent (john, X)
lgg (parent (john, mary), ~parent (john, steve)) = undefined
lgg (parent (john, mary), father (john, steve)) = undefined

TWO CLAUSES
The lgg of two clauses C1 = {L1, ..., Ln} and C2 = {K1, ..., Km} is defined as:
lgg (C1, C2) = {lgg (Li, Kj) | Li ∈ C1, Kj ∈ C2
and lgg (Li, Kj) is defined}

Examples:
C1 = father (john, mary) ← parent (john, mary), male (john)
C2 = father (david, steve) ← parent (david, steve), male(david)

In practice I do the permutation: father(john, mary) with father(david, steve), then with parent(David, steve), then with male(David).
Then i permutate parent(john, mary) with the ones below, then male(john) with the ones below
Tot. 9 couples

lgg (C1, C2) = father (X, Y) ← parent (X, Y), male(X)
C1 = win (conf1) ← occ (place1, x, conf1), occ(Place2, o, conf1)
C2 = win (conf2) ← occ (place1, x, conf2), occ(Place2, x, conf2)

If we want to generalize: p(a,b) with p(b,a) we have p(X,Y). p(X,X) is wrong!

lgg (C1, C2) = win (Conf) ← occ (place1, x, Conf), occ(L, x, Conf), occ(M, Y, Conf), occ(Place2, Y, Conf)

In this last lgg, we have only 2 useful literals. The last 2 are useless! Why?

- If I have no variable that appears in the head, I should prefer a literal where there's one variable that appears in the head: all of them have Conf, so it's ok.
- We are looking for the LEAST general generalization so we should keep all the Conf that satisfies the 4 properties.

The 1st can be generalized by only the 1st literal and the 1st can make true the 2nd, the 3rd and the 4th. So, we can say that the only useful is the first! The second one, if it was different from the 1st we should keep it, but it is more general so we can remove it.

6.3 – CLASSIFICATION SYSTEMS

There are different dimensions of ILP systems [Lav94].
One classification is the following:

- **BATCH LEARNERS**: they may require that all the examples are used to learn the model (decision trees)
- **INCREMENTAL LEARNERS**: they can accept them one by one

Another classification is:

- **SINGLE PREDICATE LEARNERS:** They can learn one predicate at a time
- **MULTIPLE PREDICATE LEARNERS:** they can learn more than one predicate at a time

Again, another classification is:

- **INTERACTIVE LEARNERS**: During learning they can ask questions to the user to check the validity of generalizations and/or to classify examples generated by the system
- **NON-INTERACTIVE LEARNERS**: have no interaction with the user

Finally, a system can **learn a theory from scratch** or from a pre-existing incomplete and/or inconsistent theory (**theory revisors**). If we have an initial theory that is consistent, this is not so difficult, but if the initial background is inconsistent, it's very difficult!

Although these are independent dimensions, **existing systems belong to two opposite groups:**

- **EMPIRICAL SYSTEMS**: non-interactive batch systems who learn from scratch one concept at a time
- **INTERACTIVE OR INCREMENTAL SYSTEMS**: incremental, interactive systems that make theory revision and learn more predicates at a time

7 – NEURAL NETWORKS

Until now **we have seen symbolic reasoning approaches based on symbols and syntactic rules** for their manipulation. **Connectionist believe that the symbolic manipulation is a very poor mechanism.**
The **connectionist approaches** are **based on simulation of the mechanisms in the human brain**. **Models resembling neurological structures** have therefore been developed.

Computers are excellent in computing, but **fail** when trying to play typically **human activities**:

- **Sensor perception**
- **Sensor-motion coordination**
- **Image recognition**
- **Adaptability**

Although a computer can beat the world chess champion, it is not able to compete with a 3 years-old child in building a Lego car, recognizing the face of a person or recognizing the voice of parents.
These **complex actions depend on many factors**, which **cannot be precisely predicted** by a program. These **factors** have to be **acquired with the experience**, in a **learning phase** → mind needs a body.
e.g. the success of gripping an object is determined by several factors: the object position, our posture, the size and shape of the object, the expected weight, any interposed obstacles, …

SPEECH RECOGNITION
It requires a learning phase necessary to adapt to the person who speaks (dialect, context), filter out external noise (e.g. during a concert) and separate any other items.
It takes place in **presence of noise** or **strong distortions**. It is **independent of the subject who speaks**. It is **independent of accents, volume, intonation**.

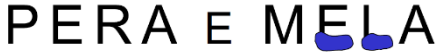
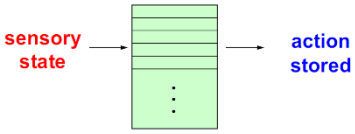


IMAGE RECOGNITION
Depending on the context, we can extract meaning by the symbols even if they're hidden or semi hidden.

When we recognize a face or grasp an object, we do not solve equations. **The brain works in an associative fashion**. **Each sensory state evokes a brain state** (electro-chemical activity) **that is stored according to need.**

e.g. Tennis match. The trajectory depends on several factors: stepping force, initial angle, effect, wind speed;

Forecasting trajectory requires: the precise measurement of the variables and the simultaneous solution of complex equations, to be recomputed at each data acquisition. How does a player do that?
In a learning phase the player tries actions and records the good ones: if the ball is passed in this visual field area, take a step back; if the ball ...
In the operational phase, once trained, the brain performs actions without thinking, on the basis of learned associations. A similar mechanism is used while driving.



ASSOCIATIVE CALCULATION: A set of complex equations are solved by means of a look-up table. It is built on the basis of experience and is refined during training.

It is extremely difficult to treat these problems with a computer. There is need to **study new methods of computation**, **inspired by the neuronal networks**. Neurophysiologists study the brain, while engineers implement the code.

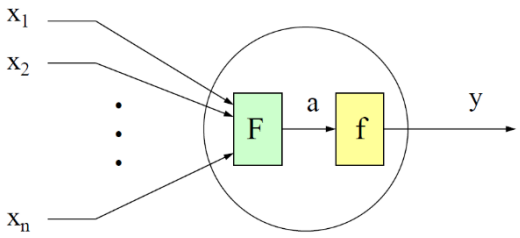
HISTORY

1943 - McCulloch and Pitts: defined the first binary threshold neuron model
1949 - Hebb: from studies on the brain, he showed that learning is not a neuron property, but it is due to a modification of synapses.
1962 - Rosenblatt: he proposes a neuron model that can learn by examples: the perceptron
1969 - Minsky and Papert showed the limitations of the perceptron: diminished enthusiasm on neural networks.
1982 – Hopfield proposed a network model to create associative memories.
1982 – Kohonen proposed a type of self-organizing network (receptive maps).
1985 - Rumelhart, Hinton and Williams: formalize supervised learning (Back-Propagation).
2006 - Yoshua Bengio deep networks

7.1 – HOW TO BUILD A NEURAL NETWORK

To build a neural network we have to define:

- The **neuron model**
- The **network architecture**
- The **neuron activation mode**
- The **learning paradigm**
- The **learning law**



$$a(t) = F(x_1, x_2, \dots, x_n)$$

$$y(t) = f(a)$$

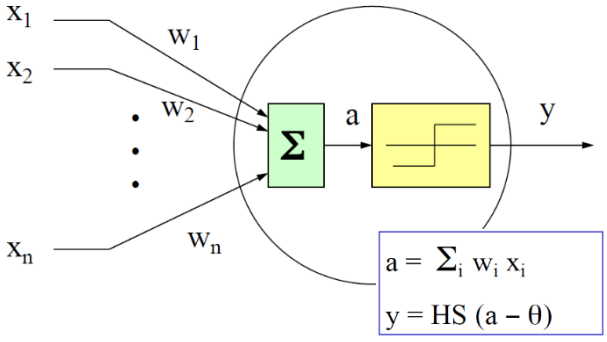
7.1.1 – PROPERTIES OF THE BRAIN

- **Speed of neurons**: few ms
- **Number of neurons**: 1011 ÷ 1012
- **Connections**: 103 ÷ 104 per neuron
- **Operations**: activation / inhibition
- **Distributed control**: lacks of a CPU
- **Fault tolerance**: graceful degradation

7.1.2 – NEURAL MODELS

To model a neuron, we have to define:

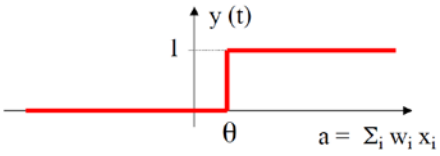
- **Number of input channels**: N
- **Type of input signals**: xi
- **Connection weights**: wi
- **Activation function**: F
- **Output function**: f



7.1.3 – ACTIVATION FUNCTIONS

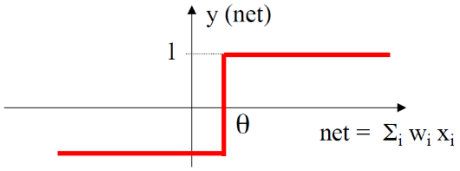
If we set a threshold, the activation function F could be, for example a **Step function**.
Neuronal functioning: the pulses received from the dendrites increase the electric potential in the neuron up to a certain threshold.

HEAVISIDE FUNCTION



$$y(t) = \begin{cases} 0 & \text{se } \sum_i w_i x_i < \theta \\ 1 & \text{otherwise} \end{cases}$$

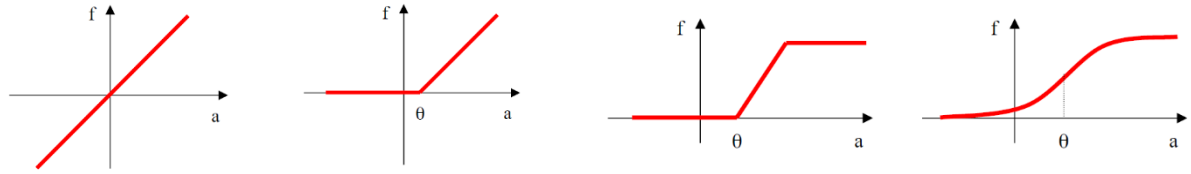
HOPFIELD MODEL



Pay attention: the interval is not from 0 to 1 but from -1 to +1.

$$y = \mathrm{sgn}\left(\sum_{i=1}^{n} w_i x_i - \theta\right)$$

OTHERS



7.1.4 - STABILITY PROPERTIES (Hopfield '82)

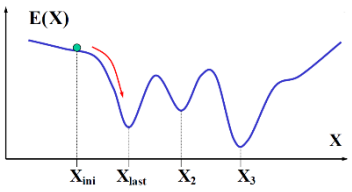
A fully connected neural network is **globally stable if:**

- **The matrix of weights is symmetric**
- **The activation is asynchronous**

This is theorem that allows to build a neural network in which states are stable and in these states we can store something that we want to record. **Asynchronous** means that **states change one after the other and not in the same time.** (e.g. synchro swimming)

7.1.5 - THE ENERGY FUNCTION

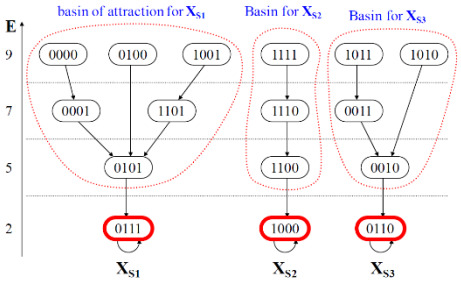
Each state is characterized by an energy: $E(X) = -\frac{1}{2}X^T W X$



If the matrix of weights is symmetric and the **activation is asynchronous**, then E (X) is non-increasing monotonic in the state evolution: **E [X (t + 1)] ≤ E [X (t)].**

The network is traversing a space, an energy space, and evolves towards a stable state: **in these minima, we can store the memory**.

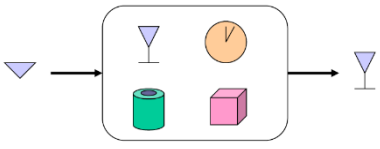
(For the proof, see the slides 42-43, pack 10_NeuralNetworks)



Basin of attraction: set of states such that all trajectories that start from them, end in the same stable state.

7.1.6 - ASSOCIATIVE MEMORIES

They are **memories whose contents can be retrieved on the basis of partial or distorted information on the content itself.** We are talking about how to shape a neural network to store some memory, for now we are not talking about learning. **By hands we need to create a number of minima for storing a set of objects.**



Suppose that we have a network with 4 stable states and in each stable state we have a representation of an object. If I have this for minimum, if I give to my network something that resemble these objects, then the network provides me, as an output, the memory that it has stored and that is closed to the input we gave.

RULE OF STORAGE (Hopfield '82)

e.g. storing pictures in a fully connected network

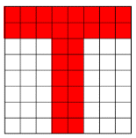
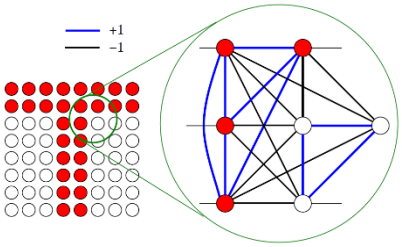
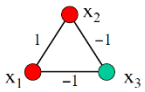
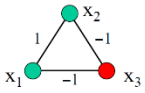


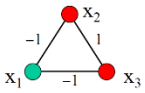
Image: n*m pixel = 8*8 = 64
Neurons: N = n*m = 64
Connections: C = N^2 = 4096
States: S = 2^N = $2*10^{19}$



Here we have an image with n*m pixels and we have 1 neuron/pixel. Of course, the **number of states** of this **fully connected network** is 2^N, so this number is **pretty large**.

M1: (+ + −)  $W_1 = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$

M2: (− − +)  $W_2 = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$

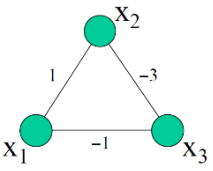
M3: (− + +)  $W_3 = \begin{bmatrix} 0 & -1 & -1 \\ -1 & 0 & 1 \\ -1 & 1 & 0 \end{bmatrix}$

Up to now there was nothing related to learning, we were using something by hands. The idea was: I have a neuron per pixel, so I can create the network in this way.

If we zoom the circle: some neurons are activated (red) and the ones that are off are related to white pixels.

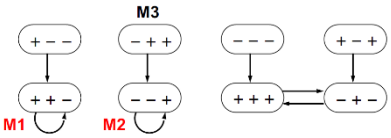
We want a weight matrix that enables me to store the network with the minimum energy. Now, we create the weight matrix and it is the following: **neuron both 1 or both 0 have a weight of +1**. **Different neurons instead have weight = -1.** This is a very easy way to build the matrix.

$$W = \sum_{k=1}^{m} W_k = \begin{bmatrix} 0 & 1 & -3 \\ 1 & 0 & -1 \\ -3 & -1 & 0 \end{bmatrix}$$



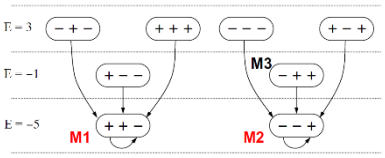
(Synchronous activation)

$M = \{(+ + -), (- - +), (- + +)\}$



(Asynchronous Activation)

$M = \{(+ + -), (- - +), (- + +)\}$



Looking at the Asynchronous activation, we have 3 neurons and we want to store 3 configurations: it's too much! If we don't have enough neurons, we cannot save too many information. M3 is not a stable state, so I can store anything in this state.

When we overlap too many memories:

- **Not always all memories are stable:** creating of a local minimum could remove another one
- **Spurious memories can appear:** The surface energy can have complex shapes. We have some stable states that are more than the one we want to create, so we have some spurious memories that we didn't want

STORAGE CAPACITY

In order to store something, we can use:

- **Rule of thumb (Hopfield):** A network of N neurons can accommodate at most size M=N/7 memories (M≅0,15N). This is more or less the 25% of the neurons we have.
- **Statistical analysis**: given β the probability of stability of the memories,

  $M = \frac{N}{2\ln\left(\frac{N}{a}\right)}$ where $a = -ln\beta$ e.g. N=1000 and β=0.9 → M=54

7.1.7 - ACTIVATION MODE

There two types of activation mode:

- **Synchronous** (parallel): The neurons change their state all together, synchronized by a clock.
- **Asynchronous** (sequential): The neurons change state one at a time. We must define a selection criterion.

Only fully-connected networks have both types of activation.

7.2 - LEARNING

We want that the training process changes the weight so that the neural network behaves in a way that we are happy with: **we want to classify states that are more general that one that we've seen in the training part. The goal is generalization.**
Hebbs (neurophysiologist) found that learning occurs by changing synapses. Network capacity to change behavior in a desired direction by changing synaptic connections (weights). **The learning paradigms can be divided into three basic classes:**

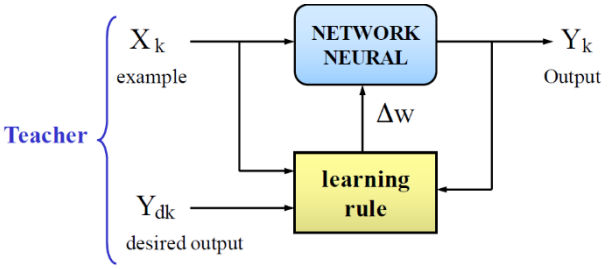
- **Supervised**
- **Competitive**
- **Reinforcement**

7.2.1 - SUPERVISED LEARNING

It is the **most widely used**. **The network learns to recognize a set of desired input configurations.**
The network learns to associate a set of given pairs (Xk, Ydk). The network operates in two distinct phases:

- **Learning phase**: it stores the desired information via examples
- **Evolution phase**: it retrieves the stored information

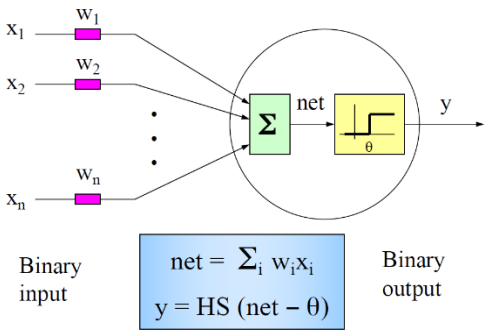
There's a trainer that tells you that the example 1 is associated to class1, the example2 is associated to class2, etc. etc. This trainer is the supervisor.
During the learning phase the network has to store the desired information: we provide the **examples** and the right classification and the network changes weights accordingly. **Then, we use the test set** (same as the training set) **to see if the network has generalized enough**.
In the first block of slides of Machine Learning we talked about **overfitting**: overfitting means that we are very good at classifying but our generalization capabilities are very poor.

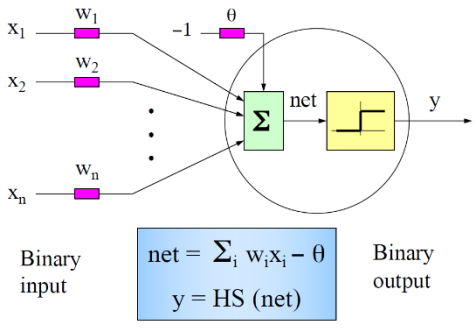


The NN checks the example, provides an output and compares the output of the network with the desired output: if they are the same, ok, if they're not the same we have to change the weights.

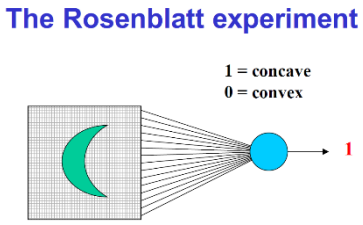
THE PERCEPTRON (Rosenblatt '58)

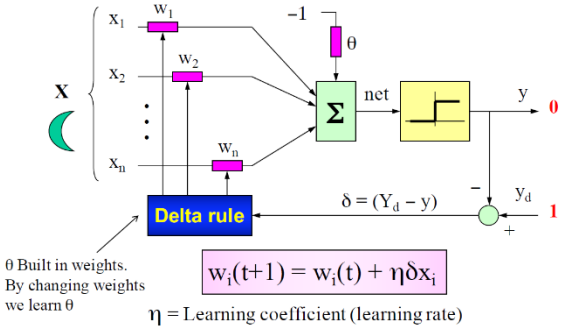
The perceptron is an **algorithm for supervised learning of binary classifiers**. A binary classifier is a function which can decide whether or not an input, represented by a vector of numbers, belongs to some specific class.





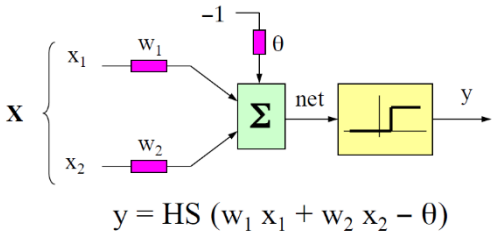
CLASSIFICATION
A perceptron can be trained to **recognize whether an input pattern X belongs or not to a class C,** e.g. cube (1) or not cube (0).





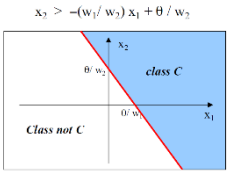
LEARNING ALGORITHM

1. Given a training set of m examples: TS = {(Xk, ydk), k = 1..m}
2. Initialize weights with random values wi;
3. Give as input a pair (Xk, ydk);
4. Compute the answer yk the network;
5. Update weights with the **delta rule: (Δw = ηδx)**
6. Repeat from step 3 until all answers are correct: yk = ydk ∀k ∈ [1..M]



INPUTS PERCEPTRON

The patterns belonging to the class will be those such that: **w1x1 + w2x2 − θ > 0.**
This equation defines a semi-plane.



LIMITATIONS

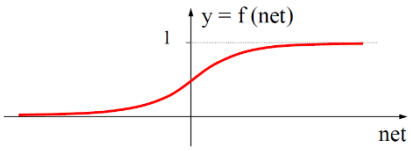
To learn a classification, the **problem must be linearly separable**:

- **The patterns belonging to the class C must be contained in a semi-plane of the input space**
- **With n inputs**, the **input space** becomes **n-dimensional** and **classes** are **separated by a hyperplane**.

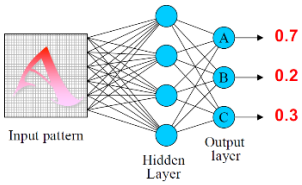
THE PROBLEM OF XOR: it is not linearly separable.
Possible solutions:

- Use **neurons with appropriate output functions**
- Combine neuron answer in **multilayer architectures**

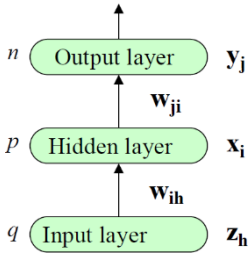
BACK PROPAGATION (Rumelhart-Hinton-Williams, '85)



It is a widely used algorithm with **LAYERED NEURAL NETWORKS** for **supervised learning**. Backpropagation **generalizes the gradient computation in the delta rule**, which is the single-layer version of backpropagation, and **is in turn generalized by automatic differentiation**, where backpropagation is a special case of reverse accumulation.



Here we use **real-valued inputs ∈ [0,1]** and **neurons have a non-linear sigmoid output function** that must be differentiable.
e.g. LETTER RECOGNIZER



DEFINITIONS

TS is the training set and is **TS = { (Xk, Ydk), K=1, …, M}** where:

- Xk = input, one at a time
- Ydk = desired output (also defined as tj)

The error has a quadratic form in the space of weights, distorted by the non-linear output function.

error on example k

$$E_k = \sum_{j=1}^{n} (t_{kj} - y_{kj})^2$$

Global error

$$E = \sum_{k=1}^{M} E_k$$

BACK PROPAGATION GOALS

We can divide the Back-Propagation's goals in:

1. **LEARNING**

   Train the network on a set of desired associations (Xk, tk): **Training Set (TS).** To favour learning, **the weights must be initialized with smaller values.**

   Indeed, net small → f'(net) great → Δw great.

   $$\Delta w = \eta \delta x \propto f'(net)$$

2. **CONVERGENCE**

   Reduce the global error E to variation of weights, so that **E < ε.** In order to do this, we adopt a **GRADIENT DESCENT METHOD** and the weights are changed according to the following law, called **GRADIENT RULE** (η = learning coefficient/learning rate. Remember that **η too small → slow learning**, **while η too big → fluctuations**)

   $$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = \eta \, \delta_j \, x_i$$

   with the error $\delta_j = -\frac{\partial E_k}{\partial net_j}$ where $net_j = \sum_{i=1}^{p} w_{ji} x_i$

   For the output layer $\delta_j = (t_j - y_j) f'(net_j)$ For the hidden layer $\delta_i = f'(net_i) \sum_{j=1}^{n} \delta_j w_{ji}$

   For the hidden layer, t_{kj} is unknown, so **we propagate back the error** because we know the error on the output and the weight is that one that we find on the synapse.

   We can vary η in function of error, in order to accelerate the convergence in the beginning and reduce the oscillations in the end. We obtain **smooth oscillations with a low pass filter on weights:**

   $$\Delta w_{ji}(t) = \eta \, \delta_j \, x_i + \alpha \, \Delta w_{ji}(t-1) \qquad \text{where } \alpha = \text{momentum}$$

3. **GENERALIZATION**

   Ensure that the network behaves well on **unseen examples.**

   To assess the network's ability to generalize the examples of TS, it defines another set of examples, said **Validation Set (VS)** and evaluate the error on the VS.

   The number of parameters to be adjusted depends on the number of hidden neurons in the network. A **few hidden neurons** may **not** be **sufficient to reduce the global error**, while **too many hidden neurons** could **overfit the network** on the TS specific examples. The network would respond well on TS, but the error would be high on other examples (**overtraining**).

Having an input at a time means that we cannot guarantee convergence because if there are many errors in the training set and we start with the wrong example, it takes longer for the network to use the reshaped weights for the new correct example.

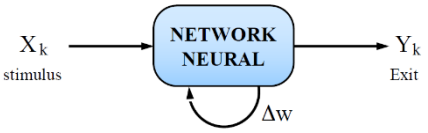
Neural Networks should always have **inputs** that are **defined**: we always have to give a value augmenting the set by filling the gaps in cells that are empty, so **NN are not very robust to errors and missing values.**

BACKPROPAGATION ALGORITHM → nowadays used in all deep learning techniques

```
randomly initialize the weights;
do {
        initializes the global error E = 0;
        for each (Xk, tk) ∈ TS {
                compute yk and error Ek;
                compute δj on the output layer;
                compute δthe on the hidden layer;
                update weights of the network: Δw = ηδx;
                updates the global error: E = E + Ek;
        }
} while (E > ε);
```

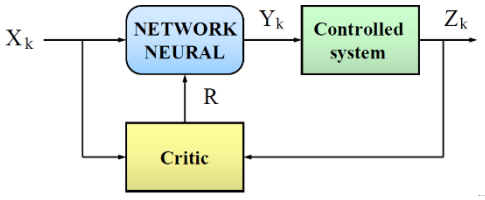
NEURAL NETWORKS	DECISION TREES
• When the weights change, we have **no clue about what NN has learned**	• The **model learning is comprehensive** for a human
• **NN don't provide information about classification**: techniques that observe the weights exist, but they are not so intuitive	• Able to discern and classify informative features that enable us to create the classification and **know which are the most salient features**

7.2.2 - COMPETITIVE LEARNING



Neurons compete for specializing in the recognition of a particular stimulus. **Similar stimuli end up in the same class**. In the end, each neuron is activated by a given stimulus **(isomorphism between stimuli and output neurons).**
This way of learning is called competitive learning or **UNSUPERVISED learning**: there's no trainer. We have to use other methods to train the system, in particular there's a framework proposed by **Kohonen (Self-organising maps)** that basically tells us that the neurons compete for specializing in recognising stimulus and this is more **similar to our real brain**. In this way, **each area is specialized in a specific task**.
The neuron changes the weight in relation to the neighbours when a stimulus occurs.

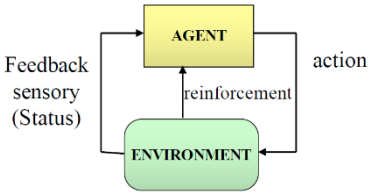
7.2.3 - REINFORCEMENT LEARNING



Reinforcement learning simulates the learning mechanism in animals based on reward and punishment: used for control systems applications.

It is very used in **games**, but for example **for autonomous drive it is not the best choice**. If we have simulator accurate enough and safe enough, we can train these systems, otherwise not because these systems **learn by doing errors**. They can be used also in **real-time applications**.

Xk is the initial state, and due to the action Yk the control system moves to Zk and then decides if Zk is a good state or not.
R is a signal called Reinforcement. It is "null" if the state Zk is a successful state, while **it is a penalty if the NN proposes an action that involves a dangerous state for the system.**

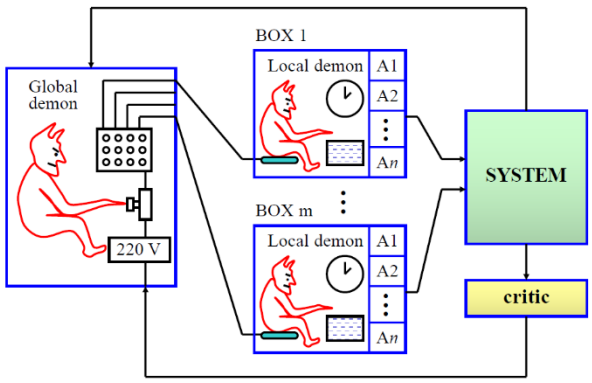


REWARDS AND PUNISHMENTS

An **agent** operates in the environment and **adapts the actions on the basis of the produced consequences**.

BOXES MODEL

- **Learning based on punishment**
- It partitions the state space into **N disjoint regions (box)**
- **Whenever the system enters a state (box) a control action is selected**
- **The controller has to learn to perform the actions which delay the punishment** as much as possible

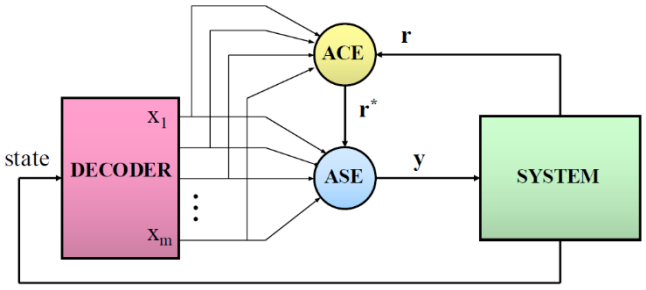


The voltage is the punishment. If the system tells you that the state has some problems, the global demon punishes all the local demons. One local demon per time is activated and the local demons select the action and then, the system performs the actions and has some changes. If the system enters a failure state, so the action that caused this failure is the last performed one and not the action performed 1 hour ago. That's why the clock is fundamental: the longer is the time between the action performed and the failure state, the smaller is the probability that the bad action was that one.

ASE-ACE

ASE: Associative/Adaptive Search Element
ACE: Adaptive Critic Element

After a while the system minimizes the number of punishments, so it works pretty well and doesn't learn anything else. For this reason, we add the ACE and r* that is the additional reinforce.



ASE: in each state we have **an input that is 1 and all the others are 0,** so the ASE has some weights that are on the input. They are initialized randomly, as usual, and it has an output computed in this way: sign of the weighted sum of the input (we have only one input = 1) plus a gaussian variable.

n(t) is the gaussian variable with zero mean and variance σ^2.

ASE can only generate two actions:

- **a+** (Y = 1)
- **a−** (Y = 0)

When the system is in the state xi (xi = 1) we have that:

- **If wi = 0**, actions a+ and a− have the same probability
- **If wi > 0**, the a+ choice is more likely to be performed
- **If wi < 0**, the a− choice is more likely to be performed

$$y(t) = \operatorname{sgn}\left[\sum_{i=1}^{n} w_i x_i + n(t)\right]$$

$$\Delta w_i(t) = \alpha\, r(t)\, e_i(t)$$

$$e_i(t+1) = \delta\, e_i(t) + (1 - \delta)\, y(t)\, x_i(t)$$

The weights of ASE are updated with the following law: **Δwi(t) = α r(t) ei(t)**

- α is a positive constant (learning rate)
- r(t) is the reinforcement signal and its value is r(t)=-1 in case of failure, r(t)=0 otherwise
- ei(t) is a signal (eligibility) introducing a short-term memory on synapses (the clock):

ei(t+1) = δ ei(t) + (1 − δ) y(t) xi (t)

A failure (r <0) reduces the probability of choosing recent actions that have caused it:

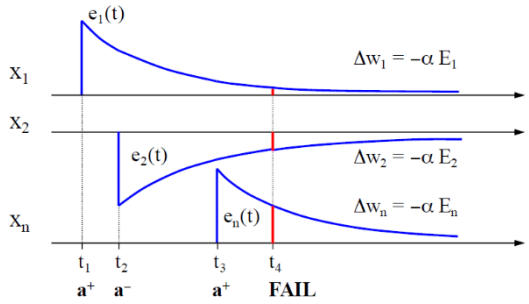
- a+ → ei(t) > 0 → Δwi < 0
- a− → ei(t) < 0 → Δwi > 0

A success (r> 0) increases the probability of selecting the recent actions that have caused it:

- a+ → ei (t) > 0 → Δwi > 0
- a− → ei(t) < 0 → Δwi<0

ELIGIBILITY

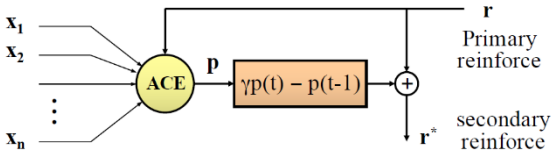
Time t1: at this point the eligibility of the system jumps up.
Time t2: here the action proposed in this state is e-, so the eligibility jumps down and start increasing.
Time t3: here the last state is encountered so we have that the action taken is a3.



Time t4: the system enters in a failure state and we have to update the weights, that are $-\alpha$ multiplied by the eligibility at time t4. So, we cut at time t4 the red line and we encounter all the eligibility of the neurons before the failure state.

The red vertical segment is the eligibility of the reinforce that I applied. Looking at x1, the red line is very little because this neuron took a decision a long time ago, while in x2 it is a little bigger. In the last neuron, we have Δwn and the eligibility is the higher because it is the last one that took a decision.

ACE



As the number of failures is reduced, the system tends to learn more slowly. The adaptive critic element (ACE) generates a **more informative secondary reinforcement**.
Observing the system status and failures, the **ACE learns to predict dangerous conditions**. It generates:

- **An award (r* > 0)** if the system moves away from a dangerous state
- **A punishment (r* <0) otherwise**

The ACE is trained so that **p(t) converges towards one of the primary reinforcement predictions**. Then, r* is generated according to the **change of the forecast.**

SECONDARY REINFORCEMENT

$$r^*(t) = r(t) + \gamma p(t) - p(t-1)$$

- **If (r = 0)**
  - **Forecasting increases** translate into awards (r* > 0)
  - **Forecasting decreases** in punishment (r* <0).
- **If (r = −1), Xi = 0 ∀i, then p (t) = 0. Therefore:**
  - **If the failure has been predicted** [p (t−1) = r (t)] **then r* = 0.** However recent actions have already been punished in previous time steps.
  - **If the failure has not been predicted** [p (t−1) = 0] **we have r* <0,** which still generates punishment.

e.g. of reinforcement learning is the simulator with the car on the road that is learning the path.

7.3 - NETWORK ARCHITECTURES

Wji = weight on neuron j on the connection coming from neuron i.
There are **two types of network architectures**:

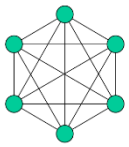
- Fully connected
- Multi-layer

TRANSFORMATION: Function T: S → S, which transforms a state X(t) in the following X(t + 1).
TRAJECTORY: Sequence of states traversed by the network, starting from an initial state X0:
X(0) = X0, X(t + 1) = T [X(t)]
LIMIT CYCLE OF ORDER K: Closed trajectory in phase space traversing Xi every k steps.
STABLE STATE: State that generates a constant trajectory: X (t + 1) = X (t) = Xs
REACHABLE STATE: A state XF It is said to be reachable from Xi if there exists a trajectory from Xi to XF.
GLOBAL STABILITY: A network is said globally stable if for every initial state X, the trajectory from X reaches a stable state.

7.3.1 - FULLY CONNECTED NETWORKS

They represent **states** that **evolve over time**. Weights of the network are specified through a connection matrix.

- **Binary neurons with threshold**
- **Parallel activation**
- **State transition:** $\boldsymbol{X_i(t+1) = HS(\sum_i w_i\, x_i(t))}$



$$\mathbf{W} = \begin{pmatrix} \text{neuron 1 weights} \\ \text{neuron 2 weights} \\ \vdots \\ \text{neuron n weights} \end{pmatrix}$$

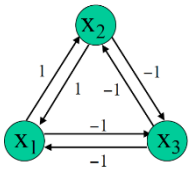
$$x_i(t+1) = \begin{cases} 1 \text{ if } \Sigma_i w_i x_i(t) \geq 0 \\ 0 \text{ otherwise} \end{cases}$$

In matrix form: $\boldsymbol{X(t+1) = HS(W \cdot X(t))}$

Where:

- **X(t)** is the **state** of the network at time t
- **W** is the **weight matrix**
- **WX(t) = vector of the state of the network at time t**

Example with **W symmetric matrix**

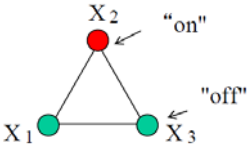


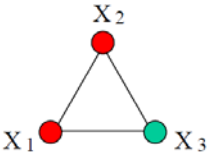
$$\mathbf{W} = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{pmatrix}$$

Here we have 0 on the diagonal because there are no self-arcs, but the arcs that starts from x1 to x2 and the same vice versa are present in the matrix.
So, no self-connections → 0 on the diagonal.
When we have 1, it means that the neuron is on.

if the Initial state is X(t) = (0,1,0). The **STATE TRANSITION** to the next state is X(t+1) = HS(WX(t)) =

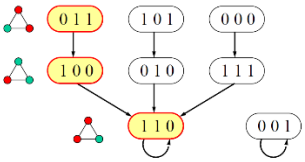
$$\text{HS}\left(\begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}\right) = (1,1,0)$$





TRANSITION DIAGRAM

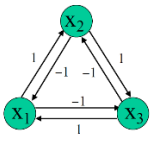
Since we have 3 neurons, we can build the transition diagram that says that: if we start from 011 and we apply the state transition, we go down in 110. When we reach 110, this is a stable state because we cannot do others transitions. So, **with this diagram, we can know the states and the transitions of our system.**



If we end up in the yellow "1 1 0", nobody can take us away from this ending point. So, if we are able to store something in this minimum here, we have achieved what is called **associative memory**. The state on the right "0 0 1" is incorrect.

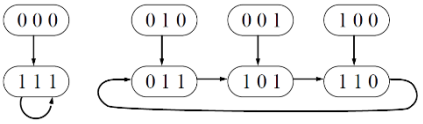
Example with **W antisymmetric matrix**

Here the weight between x1 and x2 is different from the weight between x2 and x1, so the matrix is not symmetric.



$$\mathbf{W} = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

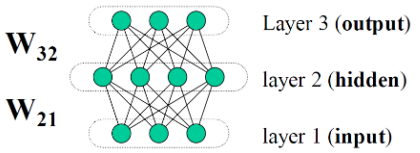
In the transition diagram we have 8 states because there are 3 neurons, but 111 is reachable only with 000 and on the right, we **don't have a stable state, but we have a cycle.** In order to try to create an associative memory, is better to use the previous example, the one with the W symmetric, because we are trying to link each stable state with a memory, so only if for example we are able to store in the stable state an object, then we are able to create an associative memory.



To resume, **here we don't have a stable state** so we're not happy about it because **we cannot create an associative memory**.

7.3.2 - MULTI-LAYER NETWORKS

Each neuron of a layer is connected with all neurons of the nearby layer. There are **no connections between neurons of the same layer**.

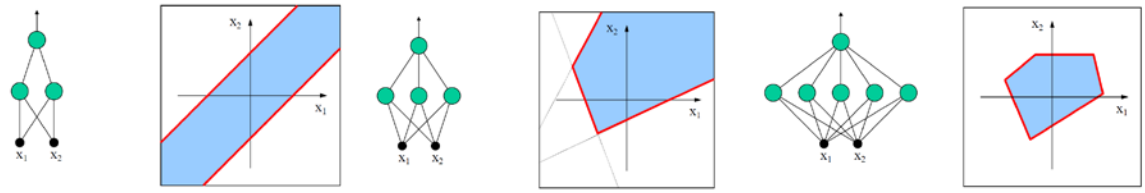


The **weights** of a network composed by n layers can be specified through **n−1 connection matrices**.

We can combine hyperplanes and exploit them for having a simpler classification problem.

3-LAYER NETWORKS

They are able to **separate convex regions with number of edges ≤ number hidden neurons**.



4-LAYER NETWORKS

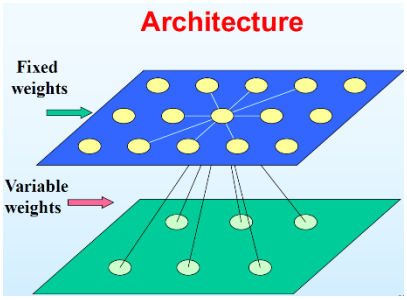
They are **able to separate regions of every shape**, but **the addition of other layers does not improve the classification ability.** (In deep networks, on the contrary, the more layers we have the more the classification ability is improved)

IMPORTANCE OF THE NON-LINEARITY

If the output functions were linear, a network N layers would always be reduced to 2 layers. To perform **complex classifications**, **neurons must be non-linear** and be **organized on multiple layers**.

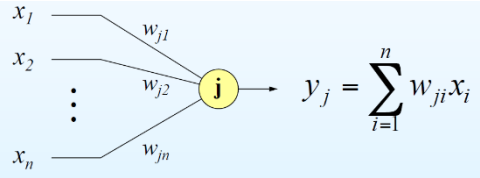
Problems: how do you train a multilayer network? What is the desired output of the hidden neurons?

7.4 - KOHONEN NETWORKS



In 1983, Teuvo Kohonen managed to build a **neural model that replicates the process of formation of the sensory maps in the cerebral cortex:**

- **Layered network**
- **Unsupervised learning** based on the **competition** between neurons



$$y_j = \sum_{i=1}^{n} w_{ji} x_i = W_j \bullet X = |W_j| \bullet |X| \cos\theta$$

The **fixed weights** depend on the distance of the neurons:

- **Neighbouring neurons** ⇒ positive weights
- **Distant neurons** ⇒ negative weights

Neurons compete to respond to a stimulus. The **neuron with greater output wins** the competition and **specializes** to recognize the stimulus. Thanks to the **excitatory connections**, the **neurons close to the winner** are **sensitive to similar inputs**.

There's the **isomorphism between the input space and output space.**

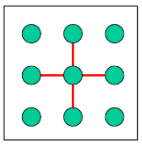
7.4.1 - IMPLEMENTATION

For **efficiency reasons**, **output neurons are not connected together** and the **winner neuron is chosen with an overall strategy** comparing the outputs of all neurons. In order to do that, we can use **two techniques**:

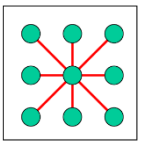
- **FIRST METHOD: Choose the neuron with maximum output**
- **SECOND METHOD: Choose the neuron whose weight vector is the most similar to the stimulus**

To simulate the radial connections, neurons close to the winner have an updated weight.
The **neighbourhood** is the set of neurons **within a given distance R from the winning neuron**, where R is the **radius of interaction**.

proximity 4



proximity 8



To allow the formation of the map, it is necessary to **define a topology on the output layer**: each **neuron** has to have a **position** identified by a **vector of coordinates**. **The output map is usually defined as a one- or two-dimensional space.**

7.4.2 - DISTANCES

The weights of the neurons are varied according to their distance from the winner neuron.

Euclidean distance

$$DIS(X,Y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Manhattan distance

$$DIS(X,Y) = \sum_{i=1}^{n}|x_i - y_i|$$

Hamming distance
(for binary vectors only)

$$DIS(X,Y) = \sum_{i=1}^{n}(x_i != y_i)$$

Given j0 the index of the winner neuron, we have a weight change also in its neighbourhood.
We use the formulas on the right, where quantities r and alfa decrease over time.

$$\forall j \in \text{neighborhood } (j_0, r)$$
$$d = DIS(j, j_0)$$
$$\Delta w_j = \alpha\, \Phi(D)\, (X - W_j)$$

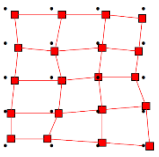
7.4.3 – LEARNING ALGORITHM

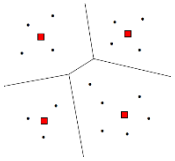
Firstly, we initialize the weights randomly. Then, we initialize the parameters: alfa for updating, r for the radius of the influence of neighbouring neurons and, after that, we start proposing to the algorithm a training set of examples, so for each of the examples we compute the output and we choose the winner neuron.
This choice is developed by a **centralized unit** that **looks at all the outputs of the neurons and choose the best.**

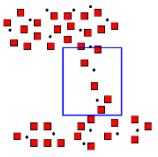
```
Randomly initialize the weights;
Initialize the parameters: α = A; r = R;
do {
        for each (Xk ∈ TS) {
                Compute all the outputs yj;
                Chose the winner neuron j0;
                update the weights of the neighbourhood;
        }
        reduce α and r;
} while (α > α min);
```

MAIN SCENARIOS

1. **M inputs = N neurons**
2. **M inputs > N neurons**
3. **M inputs < N neurons**







In case N=M, we have an isomorphism between weights and the grid. It will not be exactly the same, but alfa become ≤ $alfa_{min}$, so maybe not all the neurons are specifically covering the input, but they are very close to it.

In case the neurons are less than the inputs (N<M), a neuron specializes in recognising a number of inputs, so we are in a way **clustering the data space** in relation to **the distance** they represent. Neurons will converge toward a point that is the **centroid of the cluster**, it is basically the minimum of the data points.
So, the centroid of the cluster is the neuron with its coordinates and we obtain a **partition of the space**.

In case of N>M, so more neurons that the inputs, there will be a number of neurons where data are denser and less neurons in **areas** that are **less populated** with data.

VORONOI PARTITION

It is a **space partition** of S in n subspaces Si, such that:

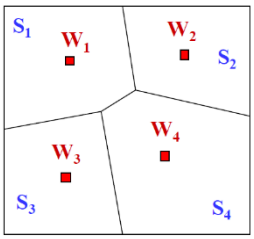
- The union of the subspaces = the initial space
- These subspaces don't intersect to each other
- The distance of points in the same subspace is always < the distance of points out of this region

W_i = centroid of S_i

$$\bigcup_{i=1}^{n} S_i = S$$

$$S_i \cap S_j = \phi \quad \forall i \neq j$$

$$DIST(X_k, W_i) < DIST(X_k, W_j) \quad \forall i \neq j, X_k \in S_i$$



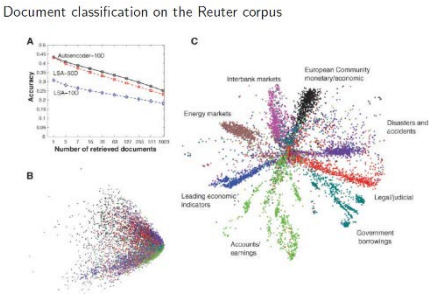
7.4.4 - APPLICATIONS

- **Clustering**
  - **Group a huge set of data in a limited number of classes**, based on the similarity of the data
- **Compression**
  - **Convert an image** with millions of colors in a compressed image to 256 levels (not fixed)
- **Classification**
  - **Classify a set of sentences** in a set of separate classes for related topics
  - Used by some search engines to **rank the preferences of connected users**

8 – DEEP NETWORKS

Deep Learning has generated much excitement in the scientific community and in industry in the last few years thanks to many ground-breaking results in:

- Speech recognition
- Machine translation
- Computer vision
  - Object recognition in images
  - Video understanding
- Natural language understanding
- Game playing



Examples

- Document classification on the **Reuter corpus**
- **ImageNet Challenge**: Universal image classifier
- **Google DeepMind (alphaGo)**

In the 4-layer networks we said that **the addition of other layers doesn't improve the classification ability**, but is it really true? **With traditional NN it is true.**

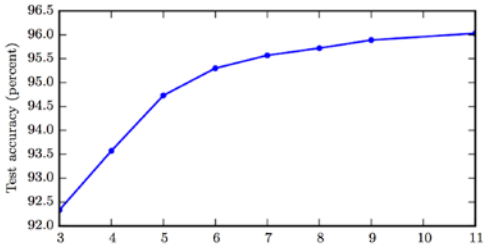
UNIVERSAL APPROXIMATION PROPERTIES AND DEPTH: Feed forward network with at least one hidden layer provide a universal approximation framework.
Regardless of what **function** we are trying to learn, we know that **a large multi-layer NN is able to represent i**t. We are **not guaranteed** that the **training algorithm will be able to learn** that **function** for two reasons:

- **Not able to find appropriate values for weights**
- **Overfitting**

Universal approximation theorem states that there is a **network large enough to achieve any degree of accuracy** we desire, but the theorem does not say how large this network will be. In the worst case, an **exponential number of hidden units may be required**. **The hidden layer might be infeasibly large and may fail to learn to generalize correctly.**

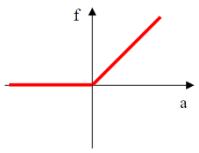
So, **adding more layers might be useful** but we have to **change few things**:

- **Activation function** (from sigmoid to **relu**)
- **Semi-supervised learning**
  - Unsupervised pre-training
  - Backpropagation for fine tuning
- **Exploit the structure of data (e.g. Locality in CNN):** Many specialized networks have fewer connections reducing the number of parameters to learn: **problem dependent**



If we add layers in the training set, we reach 96%, so we have like 3.6% improvement in percentage and this is a really good result. In the figure we see the results showing that **deeper network generalizes better when used to transcribe multi-digit numbers from photographs of addresses.**

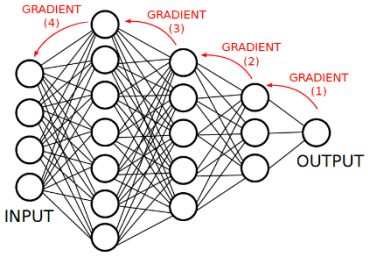
8.1 – ACTIVATION FUNCTION: RELU



Activation functions can be linear and non-linear. If we have a linea r function, every layer would be combined with other layers, so we don't exploit them in a proper way. That's why we prefer having a non-linear function. The **Rectified linear activation function (RELU): f(a) = max(0,a)** is recommended with most feed forward NN. The function remains very close to linear thus preserving many properties of linear models.

Starting from positive "a" this function is exactly "a", while before positive values, it is zero. Why RELU is better than sigmoid? It is much **easier to treat, much faster and has some nice properties**.

8.2 – SEMI-SUPERVISED LEARNING



Traditional NN are trained using **supervised back propagation**:

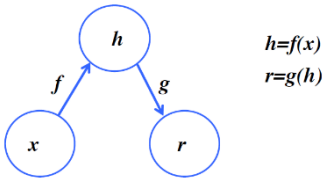
- They rely only on **labelled data**
  - Labelled data are **difficult to find**
  - **Unlabelled data are everywhere** and are cheaper
- **Back-propagation has the problem of vanishing gradients** because we compute the error with derivatives. The gradient of 1 or 2 levels is enough for changing the weights, but with more levels the gradient disappears:
  - Difficult to use when training a NN from scratch
  - Good for fine tuning

Innovation in 2006 with **Deep Belief Networks (DBN)** (Hinton et. al. 2006):

- **Train one layer at a time**
- Use **non-supervised learning**
- **Back-propagation for fine tuning** of the network

SUPERVISED LEARNING	UNSUPERVISED LEARNING
Data consist in observations X with a label Y	Data consist in observations X
Given an observation x in X, find a prediction y in Y	Find patterns and regularities in data
Learn a function f: X → Y from data	**Find most important features representing data**

8.3 – AUTOENCODERS



Autoencoders are NN that learn a representation of the input and are able to reconstruct it. They are trained with **unsupervised learning** and **minimize the reconstruction error**.

We do not want to have a perfect copy, but an approximation that prioritizes which aspects should be copied. For example, if we want to **minimize a loss function**:

$$L(x, g(f(x)))$$

with f=encoder and g=decoder, it penalizes g(f(x)) to be dissimilar to x (e.g. MSE).

8.3.1 – UNDERCOMPLETE AUTOENCODERS

In undercomplete autoencoders, h has a smaller dimension than x.

- **Feature selection**
- **Dimensionality reduction** similar to PCA
- **Capture the most salient features** of the training data
- **If the decoder is linear and L is the mean square error: Principal Component Analysis** (that enables to do a dimensionality reduction)
- **Non-linear f and g learn more powerful generalizations of PCA**

8.3.2 – OVERCOMPLETE AUTOENCODERS

Overcomplete autoencoders have an h larger than x.

- Induce **parameter sparsity**: used for classification
- **Discover non-linear relations** that are hardly discovered via statistical methods

8.3.3 – REGULARIZED AUTOENCODERS

When we have a overcomplete autoencoder, we have to impose some soft internal constraints and this is what we call regularization.
Regularization prevents the autoencoder to learn useless mapping, e.g. the identity function. Regularized autoencoders use a **loss function that encourages the model to have other properties besides the ability to copy its input to its output:**

- **Sparsity** of the **representation**
- **Small derivatives** of the **representation**
- **Robustness to noise and missing input**

8.3.4 – SPARSE AUTOENCODERS

They have a similar behavior of the overcomplete autoencoder because they concentrate on the really important features. In this case the loss function is a composition of two terms: the first is the reconstruction error L and the second is the sparsity term/penalty that is applied to the internal state of the autoencoder.
In fact, **a sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty Ω(h) on the code layer h, in addition to the reconstruction error**.

$$L(x, g(f(x))) + \Omega(h)$$

Sparse autoencoders are typically used to learn features for another task such as classification:

$$\Omega(h) = \lambda \Sigma_i |h_i|$$

There are different possible loss terms for sparse autoencoder and the most common is this one. We have a sum of all the neurons in the hidden layer hi and we use a lambda coefficient to decide which neuron we want to weight more. If we plugin this formula in the final loss of the autoencoder, the network will be balanced in 2 different terms: **the autoencoders tries to minimize the difference between the input and the output (reconstruction error), but on the other hand it tries to activate the minimum number of neurons any time.**

8.3.5 - CONTRACTIVE AUTOENCODERS

Penalizing derivatives forces the model to learn a function that does not change much when x changes slightly:

$$L(x, g(f(x))) + \Omega(h,x)$$

Here, h is the internal neuron of the autoencoder and x is the input: **we don't sum the output of each internal neuron as before, but we take the derivative**: in order to compute this, we need also x.
This is also called **contractive autoencoder** and its **used for minimize the derivative.** Before, with sparse autoencoders we wanted to minimize the number of active neurons in the layer of the neuron network, here in the contrary, we want to minimize the derivative.
Because this penalty is applied only at training examples, it forces the autoencoder to learn features that capture information about the training distribution.

$$\Omega(h, x) = \lambda \Sigma_i || \nabla_x h_i ||$$

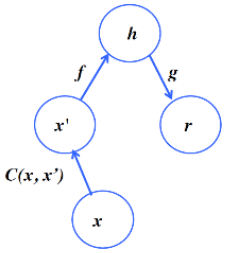
8.3.6 – DENOISING AUTOENCODERS

Rather than adding a penalty, **a denoising autoencoder has a different loss function computed on a corrupted input vector x':**

$$L(x, g(f(x')))$$

The denoising autoencoder (DAE) receives a **corrupted data point as input** and is trained to predict the original, **uncorrupted data point as its output.**
This is most capable to generalize because it's not so sensible to the input: it will not be surprise when facing different inputs. It has an entirely different loss function, not computed on the original input, but on a corrupted version of the input vector. The L is original to the autoencoder, but we don't have only x: we have also x'. So, we compare the difference of the input and the output but with this corrupted input vector.

C(x,x') is a corruption process representing the conditional distribution over corrupted samples x' given x.



The autoencoder then learns a reconstruction distribution $p_{reconstruct}(x \mid x')$ estimated from training pairs(x,x') (pairs is our training set) with the following steps:

- **Sample a training example x from the training data**
- **Sample a corrupted version x' from C(x | x')**
- **Use (x,x') as a training example for estimating the autoencoder reconstruction distribution** $p_{reconstruct}(x \mid x') = p_{decoder}(x \mid h)$ with $h=f(x')$

8.4 – LAYER SIZE AND DEPTH

Autoencoders are often trained with a single layer encoder/decoder. Each layer of the encoder and the decoder encapsulates better the features from the previous layer and if we reduce the compressing operations, we introduce multiple weight, so we increase the representational ability but we permit each layer to focus on a particular feature to be extracted.
Deep encoders have **advantages**:

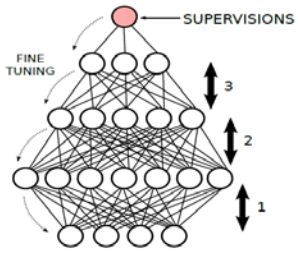
- **Depth can exponentially reduce the computational cost** of representing some functions
- **Depth can exponentially reduce the amount of training data** needed
- **Deep autoencoders yield a much better compression**

8.4.1 – DEEP NETWORKS GENERAL SCHEME

Layer-wise unsupervised pre-training is this technique, now a bit obsolete, but it's one of the techniques really most used because it works very well on preparing the NN.

- **We train a level at a time: unsupervised learning**
- **Each level can be an autoencoder**

If we consider one layer at once, each layer works as an autoencoder and can be trained in an unsupervised way:

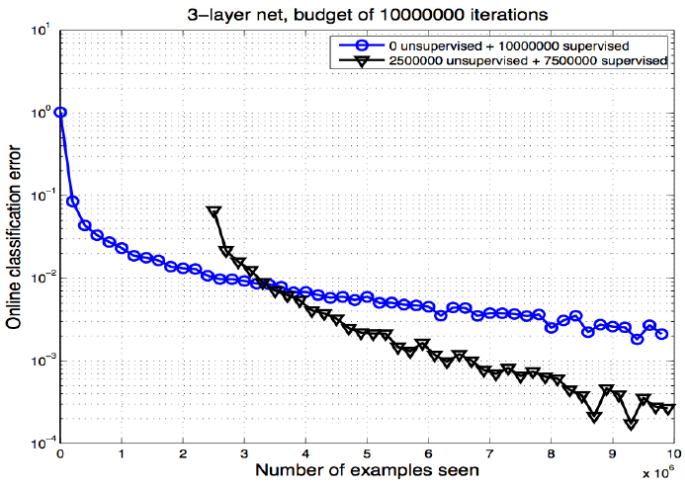


it only needs an input. The second layer in the figure can be seen as an overspending autoencoder because the number of neurons is larger than the previous level.

This NN was used for classification, the most common supervised task that we can think of. There are some layers that extract some features. **At the end we need a single neuron who represents a class** and **we use a supervised learning for propagate.**
We don't consider the labels in the step 1, 2, 3, and at the end we train again our network weights, but **we don't start from random initialization:** the initial space is very huge so it's critical to find initial weights.
Using this process allows us to **converge to global optimum in a much faster way** without stay stuck in local minima. What has been shown is that **LAYER-WISE UNSUPERVISED PRE-TRAINING** is an effective weight initialization phase. In order to do that, each layer needs to be an independent autoencoder and then, we apply a **fine-tuning process backward with supervised learning and back propagation,** probably not in one iteration.

DEEP LEARNING IS ESSENTIALLY REPRESENTATION LEARNING. Most machine learning approaches work well for specific tasks owing to a great effort in the design of good features.
Finding appropriate features can be a time-consuming but also subtle task even for specialists. So, **representation learning tries to automatically learn good features from raw input data.** Deep learning tries to learn a **hierarchy of multiple levels of representation** having **increasing complexity.**

For the same training error (at different points during training), the test error is systematically lower with unsupervised pre-training.
The blue line represents the training without a supervised learning, the black one uses unsupervised pre-training layer-wise and supervised training: in the black line the classification error decrease by order of magnitude. This is actually true: at the beginning is a little unstable, we have less benefits of unsupervised learning, but after enough supervised samples, the black line clearly outperforms the blue one.



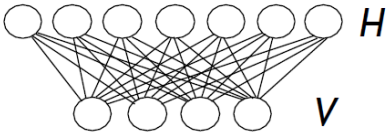
Unsupervised pre-training can be seen as a **form of regularizer** (or prior)

- It amounts to a **constraint on the region in the parameter space** where a solution is allowed
- Experiments show that its **effect is most marked for the lower layers of a deep architecture.** We are pruning out a big region in the parameter space and that's why it's so used.

8.5 – DEEP BELIEF NETWORKS (DBN)

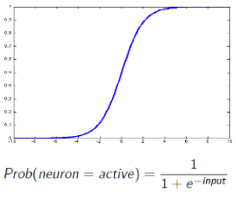
DBN are composed by multiple levels where each level is a Restricted Boltzmann Machine (RBM): each level is composed by **two sets of random variables**, V and H.

- **V = V1, ..., Vm (visible)**
- **H = H1, ..., Hn (hidden)**

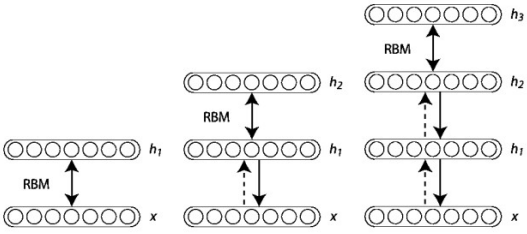


The key part is that **every Hi variable is independent from all the variables on V and vice versa for Vj and H.**

all H_i are independent when conditioning on V
all V_j are independent when conditioning on H



DBN induce a probability factorization. In the original DBN, the activation function was the sigmoid, that induces a probability factorization. The activation function is non-linear because this is the key of the expressive power of NN: they can represent any linear and non-linear function thanks to the non-linearity of the activation function. Now, for efficiency reasons, the most common non-linear Activation Functions are others, not the sigmoid.



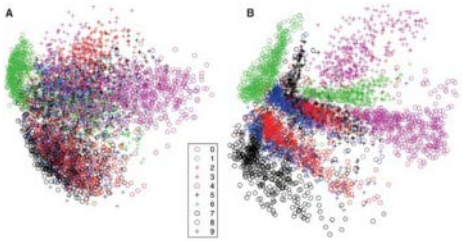
Experiments performed on:

- Image classification (Digits, Faces)
- Document classification (Reuters corpus)

Image classification is the most convincing experiment which showed that DBN could be extremely effective for this task. Now, image classification is not considered anymore a human only skill because there are extremely powerful techniques and DBN was the first showing good results.
In both cases, a DBN was trained with 4 layers, with the final feature encoder consisting in only 2 dimensions:
784 – 1000 – 500 – 250 – 2 for images 2000 – 500 – 250 – 125 – 2 for text

MNIST is a typical benchmark for image classification. Each image corresponds to a digit and the task is to teach a model how to classify them. On the left we have PCA (principal component analysis) that is a statistical approach before the DNN, on the right we DBN. Every combination of color and marker corresponds to different

classes, different digits. So, the image of the right is a better model because the different classes are much separated: since these classes are so clearly separable, it means that our model is working very well. The results are extremely good because the destroy the state of the art, that was the PCA.
Same thing for the documents (see the image on top of this chapter): the PCA works very poorly, whereas the DNN works very well and groups the documents depending on the area and the field of the document.



DBN are used not only to classify images and documents, but also to reconstruct digits, faces and other physical characteristics.

8.5.1 – RESTRICTED BOLTZMANN MACHINES (RBMs)

Since there are **no connections among units of the same layer**, the **structure of the RBM induces a probability factorization**, so the probability distribution over such units is computed as the following formulas.

$$P(H = h|V = v) = \prod_i P(h_i|v) \qquad P(h_j = 1|v) = \sigma(c_j + \sum_i v_i W_{ij})$$

$$\Longrightarrow$$

$$P(V = v|H = h) = \prod_i P(v_i|h) \qquad P(v_i = 1|h) = \sigma(b_j + \sum_i h_j W_{ij})$$

Classic learning method for RBMs: Maximum Likelihood it is whole class of methods and its approach is the one of estimating the parameters of a statistical model given observations, to define an estimator. The likelihood of the visible data v can be written as **a sum over all possible configurations of hidden states.**
This is a statistical approach with which we want to use the observation to find the optimal parameters for our DBM or in general for our Deep Learning model. **I**t finds the parameters given some observations and we can exploit those results for improving our model.

$$L = P_{model}(v) = \sum_h p(v, h) = 1/Z \ \exp(\sum_{ij} v_i w_{ij} h_j)$$

Key point = thanks to ML, given the training data, we can find the correct parameters.

8.5.2 – DROPOUT

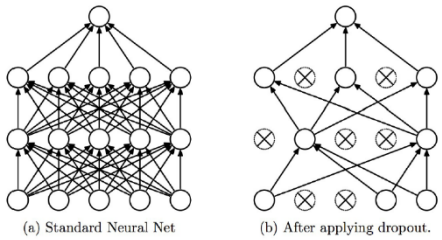
This is another trick for having a really effective NN: it means "droping out" some parameters, so we transform our wide network (high number of neurons in each layer) in a thin network. We're not talking about depth!
In general, if we combine multiple classifiers, we have a better classification performance (or in general more efficient machine learning techniques). This the reason why we use so much ADA boosting, Forests, etc.

To resume:

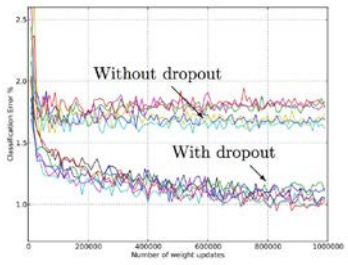
- Dropout is the **mechanism that transforms a deep network in a thinned one**
- **A neural network with n units can be seen as the combination of 2^n possible thinned neural networks**
- **Combining classifiers almost always improves performance**
- **Training different architectures with different parameters and/or inputs would be too expensive**
- The key idea of dropout is to **randomly drop units and their connections** during training

Initially every neuron is connecting to the neurons in the adjacent layers, but after having applied dropout, we have on the right some neurons dropped from the network and obviously the related weights. So, we have a simpler network: but why should we do that? **We reduce the computational load reducing the number of connections**, so the number of weights, so we simplify our hyperparameter space.
Furthermore, we have a **more generalized model**: we have fewer weights for representing the feature.

If we don't have enough examples, but we have an extremely rich internal space with many hyper parameters, is very easy to learn the identity function or something similar.

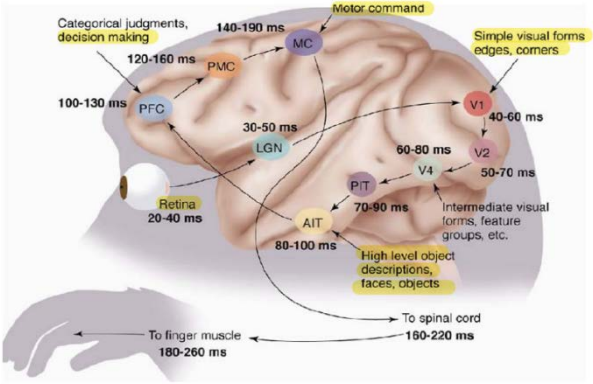


In a NN with multiple layers is easy to have a higher number of weights: if it is comparable to the number of examples, our NN will have less capacity of generalize, so the NN is going to overfit a lot. **If we apply dropout, our risk of overfit is reduced a lot.**



If we use dropout the capability of classification increases and the error decreases.

8.6 – CONVOLUTIONAL NN (CNN)

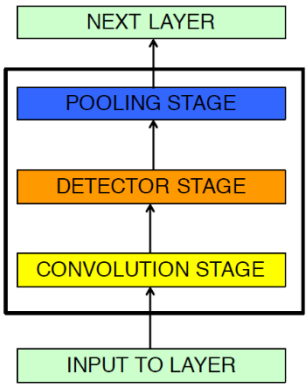


CNN are a specialised kind of NN for processing data that have a known grid-like topology:

- **Time series data (1D grid)**
- **Image data (2D grid)**

CNN employ a **mathematical operation called convolution** and basically, they are NN that **use convolution in place of general matrix multiplication** in at least one of their layers.

The architecture is inspired by biological processes, **focused on vision**: neurons respond to overlapping regions in a visual field. They are **extremely fast computation** (especially now with GPUs) **pioneering models** back from the 80s-90s. **The human visual cortex is hierarchical.**



Convolutional networks **leverage three important ideas** that improve a machine learning system:

- **Sparse interactions**
- **Parameter sharing**
- **Equivariant representation**

TYPICAL CONVOLUTIONAL LAYER

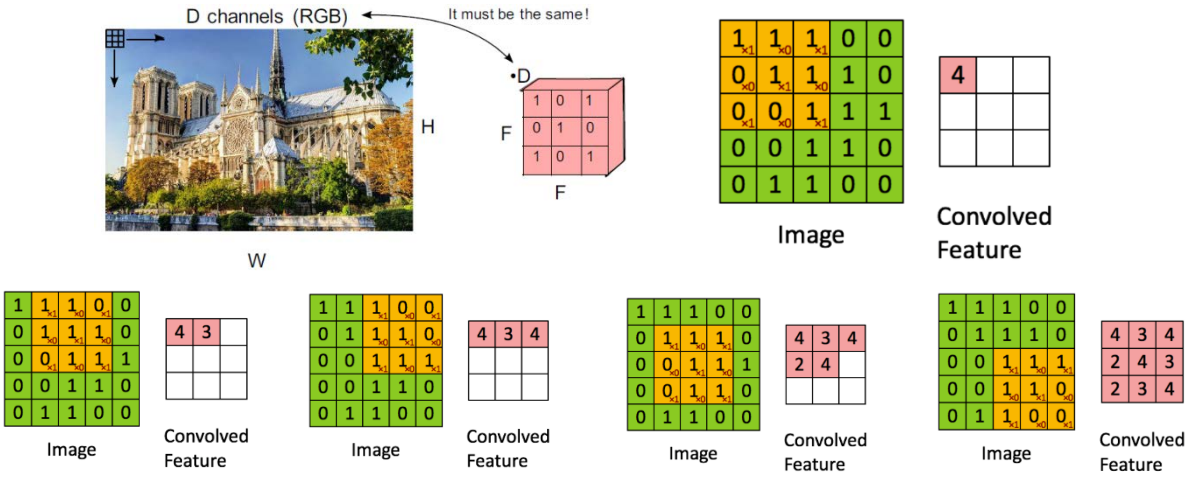
The pooling function replaces the output of the net at a certain location with a summary statistic of nearby outputs. **In the detector stage**, each activation is run through a non-linear activation function and the **Convolution Stage** performs several convolutions in parallel to produce a set of linear activations.

Pooling aggregates the output at a certain location with a summary statistic of the nearby output.
Common pooling functions:

- **Max Pooling Operation** reports the max output within a rectangular neighbourhood
- **Average** of a rectangular neighbourhood
- **L2 norm** of a rectangular neighbourhood
- **Weighted average** based on the distance from the central pixel

Pooling helps to make a representation become approximatively invariant to small translations of the input: if we translate the input by a small amount, the values of most of the pooled outputs do not change.

8.6.1 – RECEPTIVE FIELDS AND CONVOLUTIONAL FILTERS



Convolutional filter: a matrix (tensor) of weights to be applied on the image to perform convolutions.
The first metric is 111 011 001. We multiply the 3x3 matrix for the filter that was 101 010 101. Then we move the convolutional filter by one position, to the second column, and we do the same operation.

STRIDE

Hyper-parameter S indicating the "step" to be used when moving the filter on the image. Given a W x H image and a F x F filter, **(W – F)/S and (H – F)/S must be integers.**

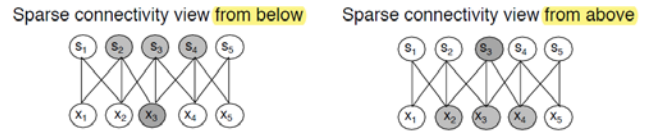
ZERO PADDING

Adding zeros along the border to allow convolutions on all pixels, so **if S = 1 → zero padding with (F –1)/2**

8.6.2 – SPARSE INTERACTION

Traditional networks connect each neuron of a layer with ALL neurons of adjacent layers, so **each output is influenced by all inputs**. **In CNN, instead, there is a significantly lower number of connections**, e.g. in an image with thousand or millions of pixels, we might detect meaningful features with kernels that occupy tens or hundreds of pixels:

- **Less parameters**
- **Reduced memory requirement**
- **Improves efficiency**



8.6.3 – PARAMETER SHARING

Parameter sharing refers to use the same parameter for more than one function in a model.

In **traditional networks each element** of the **weight matrix is used exactly once** when computing the output of a layer: it is multiplied by the input and then never revisited. **In a CNN each member of the kernel is used at every position of the input** (except for the boundaries). Rather than learning a separate set of parameters for every location, we learn one set.

- **Forward propagation runtime**, O(kxn)
- **Storage requirement**: k parameters
- **k several orders of magnitude smaller than m and n**

Convolution is thus dramatically more efficient than dense matrix multiplication in terms of memory requirements and statistical efficiency.

e.g. in Edge detection we obtain an image from the original by taking each pixel and subtracting the value of its neighbouring pixel on the left. The transformation can be described by a convolution kernel containing two elements. Floating point operations with convolution: 319 x 280 x 3 = **267960 versus** Floating point operations with matrix multiplication: 320 x 280 x 319 x 280 **> 8 billions**

8.6.4 - EQUIVARIANCE

In case of convolution, the particular form of parameter sharing causes the layer to have a property called EQUIVARIANCE TO TRANSLATION: f(x) is equivariant to a function g if f(g(x)) = g(f(x)). If the input changes, the output changes the same way. **Convolution is not equivariant to rotations, changes in scale etc.**

8.7 – ImageNet CHALLENGE

The dream: **build a computer vision system capable of recognizing thousands of object categories:**

- Over 14 million of images
- Over 20 thousand of categories tagged via crowd-sourcing
- Organized according to the WordNet noun hierarchy

In 2012 **AlexNet**: first deep learning method capable to beat human accuracy in image recognition.

- 60,000,000 parameters
- 650,000 neurons
- trained for 5-6 days with 2 GPUs in parallel
- achieved a top-5 error rate of 18.2 % (second best 26.2 %), reduced to 15.4 % with multiple models and pre-training (top 5 error rate: capability of identify the five more probable class for an object)
- achieved a top-1 error rate 40.7 %, reduced to 36.7 % with multiple models and pre-training

Features of the 2014 edition:

- 1.2 million images for training, 50k images for validation, 100k images for test
- 1,000 semantic categories

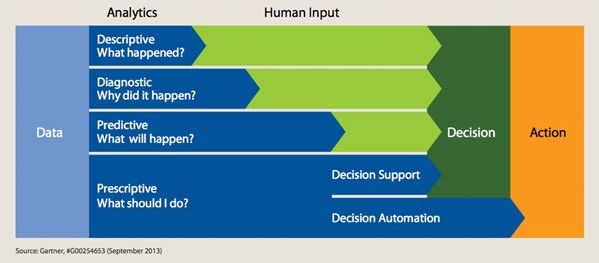
Winner: **GoogLeNet** (**22 layers**) → 6.67 % Top-5 error

In 2015 **ResNet** (Microsoft Research) → 3.6 % top-5 error
They employed a **152-layer net** and they won all the tasks (localization, detection, segmentation)

9 – CONSTRAINTS

We have **two ways for building decision systems:**

- **Data driven**
- **Knowledge based**

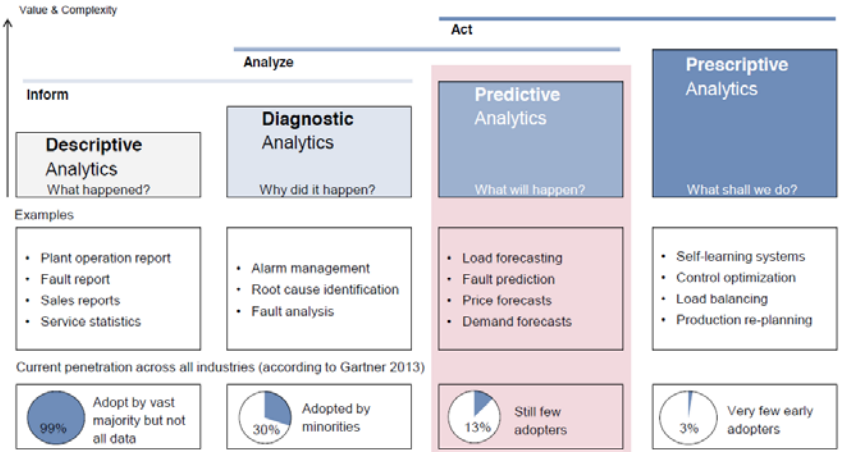


In the **DATA DRIVEN MODEL**, we have several steps:

- **DESCRIPTIVE ANALYTICS**: suppose that we have data that come from a machinery in an industrial plant, the descriptive model will describe the components of the plant starting from the data. So, we have a lot to do for reaching a decision.
- **DIAGNOSTIC ANALYTICS**: here we try to find the causes of specific faults or behaviours. It goes a little bit further than the descriptive part.
- **PREDICTIVE ANALYTICS**: during the ML part we've seen that using regression we can predict the price of sum stocks, or the energy produced by a photovoltaic system. We can foresee the future dynamics of our system. **We can also predict the failure,** because we are able to anticipate it just using some measures. For example, a simulator is a predictive model.
- **PRESCRIPTIVE ANALYTICS**: we can use the **decision support** or **decision automation**. In decision support a human expert selects a decision and apply it to a system.

WHAT IF ANALYSIS

With a predictive model we can use the "what if analysis": **we can define different decision for our system.** If it is in line, we can perform a decision. This approach has a **pitfall**: if the number of decisions is limited, ok, otherwise **if the number grows exponentially with the size of the system, it is not possible to apply it.**



99% of industries adopt descriptive analytics, but this percentage falls down to **30%** when we talk about **diagnostic analytics**: when an alarm is raised, we need to find the root code specification, we need to find the reason and make an analysis of fault.
This 30% goes down to **13%** if we deal with **predictive analytics**: these tools try to forecast the load of their plans, the time or the probability of fault of specific components, the demand of goods and forecast the prices.
That 13% became **3%** if we talk about **prescriptive analytics**. As we can see, AI has a major role in the first two steps: descriptive and diagnostic analytics.

9.1 – CONSTRAINT PROGRAMMING (CP)

CP is a technique to solve Constraint Satisfaction Problem (CSP) and Constraint Optimization Problem (COP).
These problems are very similar: in CSP we look for a solution that satisfies the constraints, while in COP we find the optimal solution.
CP has a **declarative approach** in which **we model the problem and we solve it** (more or less like MIP). **The Model is composed by variables and a rich set of constraints** (much more than linear in-equalities).

Examples of CSP in the real world are:

- Netherlands railways
- Elderly care in Sweden
- Delaware river water management
- Management of UPS delivers
- Dams management in the Netherlands
- Procter & Gamble purchase management

A few reasons for using CP:

- **Rich modelling language**
- **Fast prototyping**
- **Easy to maintain** (modifications are simple)
- **Extensible** (new constraints, customizable search...)
- **Very good framework for hybrid approaches**

It's a **good solution technique, especially for messy, real-world, problems**

The declarative framework is useful for integrating approaches coming from other domains.
During the course of Operational Research, we've studied a number of optimization problems that have been studied very deeply. e.g. one technique used is the one of creating cutting planes for reaching a solution quickly. In a real system is not so easy to find a so clean solution: we have some constraints derived by guidelines and this messy real problem doesn't have a real structure. So, they are really good candidates for using this constraint programming system.

Perfect examples for CSP:

- Map Colouring
- PLS = Partial Latin Square

In all problems formulated like the previous two, **there is a pattern.** We reason on the constraints to narrow down the possible choice and we make (and unmake) some choices.

CP = MODEL + CONSTRAINT REASONING + SEARCH

CP is a declarative approach, so it starts compulsory with a declarative model. The model is the only part that we need to define because the constraints and the search are already defined.

9.2 – MODEL

9.2.1 – CSP

First, we need to define the kind of problem we are interested in: **CSP = <X, D, C> is a Constraint Satisfaction Problem** where:

- **X is a set of variables** in which xi is a single variable
  - **I**n principle: any kind of variable: colors, numbers, objects, sets, graphs…
- **D is a set of domains** and xi takes values in D(xi)
- **C is a set of constraints**

CONSTRAINTS

$$c_j \subseteq \prod_{x_i \in X(c_j)} D_i$$

A constraint cj is defined over a subset X(cj) of the variables that is the scope of the constraint. **A tuple τ of arity n is a sequence of n values** and defines which of the tuples are consistent with the constraint itself, in fact **a constraint on X(cj) defines which tuples are compatible with cj.**
A constraint **defines a subset of the cartesian product of the domains of X(cj).**

SOLUTION

A solution for a CSP is an assignment of all the variables that is feasible for all constraints. A CSP with no solution is called INFEASIBLE.

Formally:

$$\tau \in \prod_{x_i \in X} D(x_i) \ : \ \tau[X(c_j)] \in c_j \qquad \forall c_j \in C$$

where

- $\tau[X(c_j)]$ is the projection of τ over $X(c_j)$
- $\tau[X(c_j)]$ = sequence of values assigned by τ to variables in $X(c_j)$

Example of CSP

- **Variables X={x0,x1}**
- **Domains: D(x0)={0..2}, D(x1)={0..2}**
  The cartesian product of the values are 9 values, one with each other: 00, 01, 02, 10, 11, 12, 20, 21, 22.
- **Constraints: c0={(0,0),(0,1),(0,2),(1,1),(1,2),(2,2)}, c1={(0,0),(0,1),(0,2),(1,0),(1,1),(2,0)}.**
  They are a subset of the cartesian product

The solution is (0,0),(0,1),(0,2),(1,1).

There are two kinds of representation of constraints:

- **EXTENSIONAL REPRESENTATION**
  - Ok with finite domain variables
  - We can actually list all the feasible assignments
  - **Very general**
  - Possibly **inconvenient** and **inefficient (for large domains**)
- **SYMBOLIC/INTENSIONAL REPRESENTATION**
  - We may prefer this one because it is **more compact**
  - **More clear**
  - **Less general**

DOMAIN

In practice, domains can be:

- Integer variables
- Real variables
- Set variables
- Graph variables

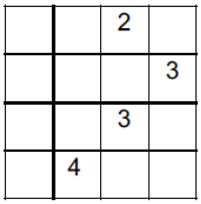
In this course we strongly focus on finite, integer domains that are the most studied cases, so we talk about **Constraint programming on finite domains.**

9.2.2 – CONSTRAINT LIBRARIES

INTENSIONAL FORMS ARE MADE POSSIBLE BY CONSTRAINT LIBRARIES. A constraint library is a collection of constraint types. Our first constraint library is composed by equality and disequality.

- **Equality**:
  - Notation: x = y
  - Semantic: satisfied if x is equal to y
- **Disequality**:
  - Notation: x ≠ y
  - Semantic: satisfied if x is not equal to y

e.g. Partial Latin Square



Variables and domains:

- $x_{i,j} \in \{1..4\}$, for each cell i, j
- $i \in \{0..3\}$ = row index, $j \in \{0..3\}$ = column index

Constraints:

- $x_{i,j} \neq x_{i,h} \quad \forall i, \forall j < h$
- $x_{i,j} \neq x_{h,j} \quad \forall j, \forall i < h$
- $x_{i,j} = v$ is cell i, j is pre-filled with value v

9.3 – CONSTRAINT REASONING

We can reason on the constraints and find some techniques to prune our possible solutions a priori, for example with the forward checking: we check what we've already done and we remove the values not compatible with the values already assigned.

9.3.1 – FILTERING

Filtering for cj means removing probably infeasible values from the domain of the variables in the constraint scope. We can also say that we are pruning or propagating the values.
Convention:

- **Pruning** = the act of **removing a single value**
- **Filtering** = the **process that guides pruning**
- **Propagation** = the **process by which filtering from one constraint may enable filtering from another constraint**

Each constraint cj should be able to remove the values that are probably infeasible in the constraint scope.
If a constraint is a software component that embeds an algorithm based on its semantic and it's able to prune the values incompatible with the values that are already assigned, this is called FILTERING ALGORITHM and we have one for each constraint.

- **For constraints in extensional form:**
  - **General methods**
- **For constraints in intensional form:**
  - Specialized **filtering algorithms** specified in the **constraint library**
  - Alternative names: **filtering rules, propagators**

Example for the equality constraint x=y.

$$\text{Rule 1:} \quad v \in D(y) \wedge v \notin D(x) \longrightarrow v \notin D(y)$$
$$\text{Rule 2:} \quad v \in D(x) \wedge v \notin D(y) \longrightarrow v \notin D(x)$$

Rule 1: if v belongs to the domain of y and not to the domain of x, the value is removed from the domain of y
Rule 2: if v belongs to the domain of x and not to the domain of y, the value is removed from the domain of x
e.g. before filtering: x∈{0,1}, y∈{1,2} → after filtering: x∈{1}, y∈{1}

9.3.2 – PROPAGATION

Propagation is the process by which filtering from one constraint may enable filtering from another constraint.
Propagation is controlled by a **propagation algorithm**.

A first propagation algorithm that can be used is the AC1.

ALGORITHM AC1

```
dirty = true
while dirty:
        dirty = false
        for cj ∈ C:                    → it considers each constraint separately
                D' = filtercj(D)
                if D' != D: dirty = true
```

Where **filtercj(D) is a procedure** that, given the current variable domains, applies the filtering algorithm of cj and then returns the updated domains.

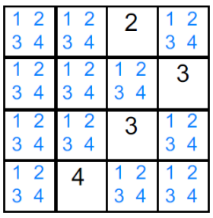
FIX POINT THEOREM

AC1 always converges to a fix point, which is independent on the constraint processing order.
The result can be proved in two steps:

- **AC1 always terminates** when no more pruning can be done
- **The processing order does not matter**

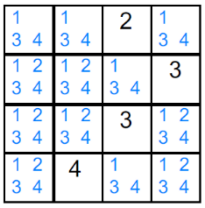
The proof relies on some properties of $filter_c(D)$. **In AC1 domains are always reduced** and the order does not matter but **some orders might be more efficient than others.**

e.g. PLS
Applying the constraints, we can prune the solutions obtaining the second figure. Applying then the AC1, we can reduce again the possible feasible values, as it is shown in the 3rd figure.

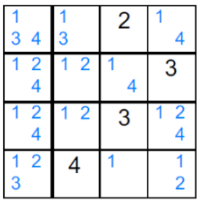


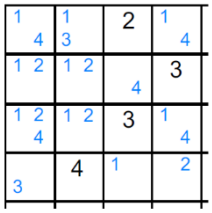
$x_{0,0} \neq x_{0,2}$ prunes 2 from $D(x_{0,0})$

$x_{0,1} \neq x_{0,2}$ prunes 2 from $D(x_{0,1})$



$x_{0,2} \neq x_{0,j}$ and $x_{0,2} \neq x_{i,2}$ prune a lot





We apply AC1

BINARY CONSTRAINTS

Each constraint requires a filtering algorithm. If we want the situation to stay manageable, **we need a systematic approach to design filtering algorithms.**
We will start from the simplest case, i.e. binary constraints. We have seen equality and Disequality.
Now we will add a few more: <, <=, > and >=.

Informally, we remove a value v from domain D(x) if it cannot be extended to a feasible assignment for cj by using only the values in D(y). Formally, for a binary constraint c:

A value v∈D(x) has a support iff there exists a value w∈D(y) such that (v,w) ∈c.

9.3.4 – ARC CONSISTENCY

We can prune all and only the values that lack a support. Once we are done, we say that Arc Consistency holds for c. Arc consistency takes its origin from a graph that represents the CSP.
Formally:

A binary constraint c with X(c)={x,y} is arc consistent iff every v∈D(x) and every w∈D(y) has a support.

A CSP IS CONSISTENT IFF ALL THE CONSTRAINTS ARE ARC CONSISTENT.

LOCAL VS GLOBAL CONSISTENCY

AC provides guarantees only for a single constraint, so **Arc Consistency is a form of local consistency**.
Can we enforce global consistency on a problem? Yes, but it is as complex as solving the problem (in general).

In general, any variable is involved in many constraints. Consequently, **any change in the variable domain can propagate to other values** from the domains of the other variables. **Agents' perspective:** during their life time, **constraints alternate their state of suspended and active** (triggered by events).
If we arrive in an empty domain, we fail, we are in a faulty state: **our problem is infeasible**, it doesn't admit any feasible solution. The order in which the constraints are considered (suspended/awakened) does not affect the result, but can significantly influence computational performance.

The complexity of a filtering algorithm is very important because there are many executions in each search node and the search strategy explores an exponential number of nodes. Consequently**, the search strategy has a strong impact on the performance.**

9.3.5 – BOUND CONSISTENCY

Example: the complexity of enforcing AC on the Disequality constraint is O(|D(x)|*|D(y)|): very inefficient!
If we consider x∈{400 000..1 000 000} and y∈{0..500 000}, even assuming that max(D(y)) has already been computed, we need to iterate 600 000 times obtaining a set of {400 000..500 000}!
Nonetheless, we can consider the maximum possible value for x as max(D(y)), i.e. 500 000, therefore every v∈D(x)>500 000 lacks a support. So, if we store the min/max of each domain explicitly, we just need $\bar{x}$ = 500 000, so O(1)!

We can formalize this idea:

- For each variable x, we store the domain min/max $\bar{x}$
- Based on the variable x and the min/max $\bar{x}$, we compute the min/max for the other variable y
- We prune by forcing $\underline{y}$=lb$_y$ (lowe bound) and $\bar{y}$=ub$_y$ (upper bound)

Formally:

A constraint c on x and y is bound consistent iff $\underline{x}$ and $\bar{x}$ have a support in {$\underline{y}$.. $\bar{y}$} and vice versa.

So, overall, we have that:

- **AC** is the best that we can do, but **may be inefficient**
- **BC is much more efficient, but may prune less**

Both AC and BC are incomplete: we still need search:

- With **AC**, we spend **more time on propagation**, **less time in search**
- With **BC**, we have **faster propagation**, but we need to **search more**

In some cases, one approach is usually better, e.g. BC for ≤, = or arithmetic constraints. In other cases, the answer is not trivial. That's why many CP solvers allow to choose which filtering algorithm to use.

9.4 – SEARCH

In general, filtering and propagation are not enough to solve a CSP, we still need to search for a solution. In fact, **solving a CSP is NP-hard**: no surprise, since the definition is so general, but don't forget that most of the interesting problems out there are NP-hard.

9.4.1 – DEPTH FIRST SEARCH (DFS)

The simplest search approach in CP is **Depth First Search:**

function DFS(CSP):
if a solution has been found: return true
if the CSP is infeasible: return false
for d in decisions(CSP): → otherwise, build a set of decisions and try to apply them one at a time
    if DFS(apply(d,CSP)): return true → if the decision can be applied, ok, else backtrack and unmake the last decision. Then, try the next one.
return false → if no decision Is successful, return false, so the problem is infeasible

A problem is infeasible if we have a **DOMAIN WIPEOUT, so** an **empty domain**, because of propagation.
This is a sufficient condition, but is not a necessary condition, i.e. **if there is wipeout, then the problem is infeasible, but infeasibility may hold even if there is not wipeout.**

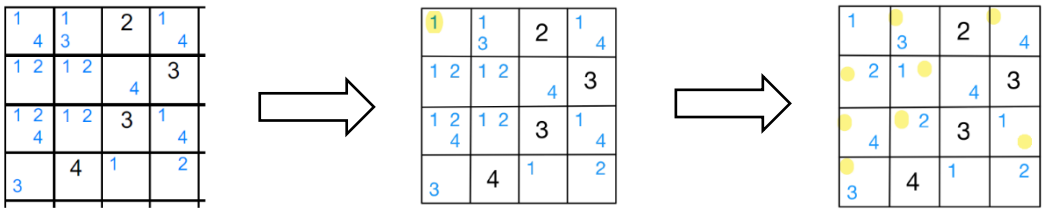
Decisions returned by decision(CSP) are constraints. For example, we pick the first unbound variable xi (variable with non-singleton domain), we pick the smallest value v in D(xi) and our decisions are xi=v and xi≠v.
There are many more options, but for now let us stick to this.

Applying a decision means:

- **Adding (posting) the constraint to the problem**
- **Triggering its filtering algorithm for the first time**

Remember: on backtrack, the constraint must be removed.

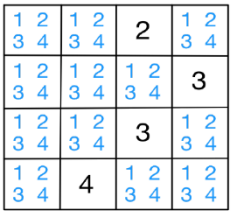
e.g. after having applied AC1, we decide that x00=1. Then, we propagate and we obtain a solution with just one search decision. This result is enable by the use of constraint reasoning.



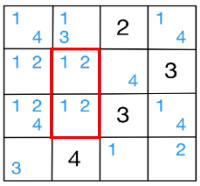
Depth First Search is the easiest strategy and some heuristic are based on Depth First and they try to select variables with the smallest domains. **In general, we consider the most constrained principles.**
For the values, instead, we can have another heuristic which is the Most Promising Value: as soon as we have defined these decisions, the process converges to a failure or a solution.

9.5 – GLOBAL CONSTRAINTS

Every time we impose different constraint between two variables, it starts propagating its effects on values that don't have any support in the other domain. Now, we can propagate the constraint that involves the variables that are singletons. In the example, we can delete "3" in the last column, for example.



We obtain, looking at the second figure, an arc consistent network because the rest of the cells cannot be propagated anymore. This is an impressive reduction in relation to the first image because we had a lot of possible values and each value represent a subtree in the tree search. So, removing one value is not just remove one value, but it means to remove a WHOLE SUBTREE.



Here we have a huge reduction because we have 21 values and many of them are already propagated, so we have just to search for 8 different cells. It means 16 values instead of 48!
All these steps concern local constraints, but **if we use global constraints, we can do even better.**

For example, in this case, looking at the cells rounded in red, 1 and 2 have to be chosen in these 2 cells because if we choose 1 in the cell up, only 2 is left in the orange cells and it's not correct! So, having 2 variables equals in those cells, means that we can prune the 1 on the top cell that has 1,3: this is called **GLOBAL REASONING**.

9.5.1 – ALLDIFFERENT

We can introduce a new global constraint: **ALLDIFFERENT(X), where X is a vector of variables.**
Semantically, it is equivalent to xi ≠ xj, ∀i ≠ j, but the filtering algorithm can be written ad hoc using virtually any algorithmic technique. **In the case of ALLDIFFERENT, polynomial GAC (Generalized Arc Consistency) propagators do exist.**

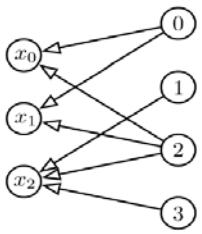
Global constraints enable to have a much powerful filtering: **ALLDIFFERENT** is the most used. We take a vector of variables and we suppose that they should be all different. This is semantically equivalent to a set of pairwise constraint of different, but instead of having an arc consistent algorithm that is applied to each constraint, we have a single algorithm working on the whole set of variables containing the vector. **THIS IS A CORRECT AND COMPLETE FILTERING ALGORITHM.**

Problem of assignment: linear programming provides an integer solution in a polynomial time relaxing the integrality constraints. This is a polynomial time algorithm, so we can create a filtering algorithm that achieves consistency for this constraint, or generalized arc consistency (GAC).
For achieving GAC we have polynomial propagator that enables us to remove values not belonging to any feasible solution of this constraint.

PROPAGATOR

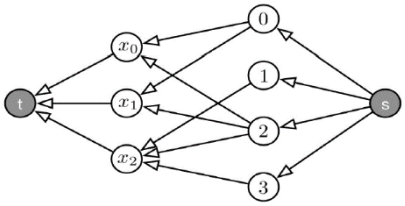
We will see an ALLDIFFERENT **propagator based on network flows**, while the **classical propagator** is instead **based on graph matchings.**
We consider **two distinct problems**:

- **Checking the constraint feasibility**
- **Performing filtering**



FLOW BASED REPRESENTATION

Example instance: ALLDIFFERENT(X), with x0∈{0,2}, x1∈{0,2}, x2∈{1,2,3}
The circles on the left are the variables, while the circles on the right are values.



The arrows are the arcs, so the possible assignments of values to variables. **The value graph is bipartite by construction.**
Then, we add an additional source node s, connected to all values, and an additional sink note t, to which all vars are connected.
This structure can be seen as a **pipe network**: **each arc is a tube with capacity equal to 1**. **Flow can originate from s and move toward t.**
The important thing to keep in mind is the **capacity conservation**: if x1 is assigned to variable 2, the flow starting from value 2 has to be of entity 1, because x1 has a capacity =1. So, if the capacity of an arc is equal to 1, the value cannot be assigned twice to the same var.
If the arc of capacity = 1 is the one that starts from s, it means that each variable can be used at most once.

If the arc of capacity = 1 is the one that ends in t, it means that each xi can be assigned at most once.
FEASIBILITY: A solution exists iff all variables are assigned, i.e. if there exists a **flow with total value equal to |X|,** in this case 3.

Hence, we have our feasibility check:

- Route the maximum possible flow from s to t
- Check if the total flow value is equal to X

If the total flow value is | X | the constraint is feasible.

Some practical examples:

- **Partial Latin Square with colors** → one ALLDIFFERENT per row and one ALLDIFFERENT per column. e.g. with a 10x10 matrix, we have 20 ALLDIFFERENT
- Fiber optical networks → Routin in fiber optic networks is a NP-hard problem

9.5.2 – GLOBAL CARDINALITY CONSTRAINT (GCC)

If we have a problem in which we are interested in:

- **Counting the occurrences** (i.e. cardinality) of specific values
- **Restricting the maximum/minimum cardinality**

So, we could define a **Global Cardinality Constraint: GCC(X, V, L, U)**
Where:

- **X is a vector of variables xi**
- **V is a vector of values vj**
- **L is a vector of cardinality lower bounds lj for vj**
- **U is a vector of cardinality upper bounds uj for vj**

The Global Cardinality Constraint is very important in practice:

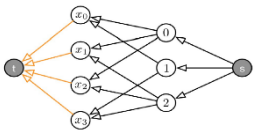
- **Restriction on cardinalities** appear in many models
- **Shift-scheduling**
- **Timetabling problems**
- **Sport and tournament scheduling**
- **Capacity constraints** (for identical demands)

Even ALLDIFFERENT(X) could be encoded as GCC(X, V, [0..0], [1..1]), although it is more efficient to use ALLDIFFERENT, when possible.

PROPAGATOR

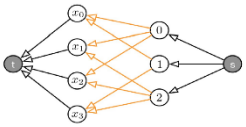
In order to perform filtering for GCC, we exploit the similarity with the ALLDIFFERENT:

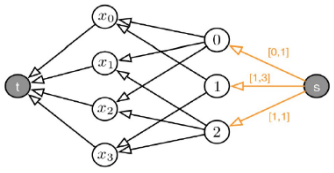
- **We write a consistency checker based on networkflows** (with the same flow interpretation as in the ALLDIFFERENT)
- **We define flow-based filtering rules**



Actually, our ALLDIFFERENT propagator was originally designed for GCC.

- **1st figure**: these arcs have capacity 1, as for the ALLDIFFERENT: **each variable cannot be assigned more than once**
- **2nd figure**: these arcs have capacity 1, as for the ALLDIFFERENT: each variable cannot be assigned more than once and **each value cannot be assigned twice to the same variable.**
- **3rd figure**: these arcs have a **capacity = Ui**, so **they cannot be used more than Ui times.** They have a **demand = Li**, so **they must be used at least Li times**.





e.g. in the arc <s, 0> we have [0,1], so it means that 0 should be assigned minimum 0 and maximum 1 time.
In the arc below, we have [1,3] (where 1 is the demand and 3 is the capacity), so 1 can be assigned 1-3 times.
In the last arc with [1,1], 2 has to be assigned exactly 1 time.

The constraint is feasible iff we can find a flow such that:

- **There is flow on all xi →t arcs** (i.e the max flow value is | X |)
- **The capacity and demand constraints on all arcs are satisfied**

FILTERING

Filtering can be performed by reasoning on the residual graph: the **value-variable arcs are identical** to those in the **ALLDIFFERENT**, hence, **we can filter based on (lack of) cycles** and **use strongly connected components** to **speed up the process**.

9.5.3 – DISTRIBUTE

In solvers the GCC is sometimes called DISTRIBUTE.
Actually, DISTRIBUTE is a more general constraint, with signature **DISTRIBUTE(X, V, N)**
Where:

- **X is a vector of variables xi**
- **V is a vector of values vj**
- **N is a vector of cardinality variables nj**

Two important differences in relation to our definition:

- The **cardinality bounds are specified via D(nj)**
- The **employed propagator can filter the nj variables**

Which means that **we can use DISTRIBUTE for counting.**

9.5.4 – OTHERS

Many solvers provide other similar constraints:

- **COUNT(X, v, c)**: If we simply need to count the occurrences of a value. **The constraint is satisfied if value v is taken c times in X.**
    - X is a vector of integer variables
    - v is an integer value
    - **c is either an integer or a variable**
- **ATMOST(X, v, c):** If we need to limit the occurrences of a single value. **The constraint is satisfied if value v is taken less than c times in X.** The case with c variable is subsumed by COUNT(X, v, c).
    - X is a vector of integer variables
    - v is an integer value
    - **c is an integer**

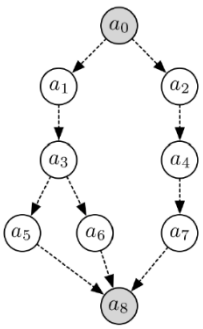
10 – SCHEDULING

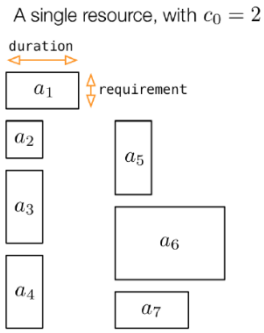
10.1 - CONSTRAINT BASED SCHEDULING

10.1.1 - RCPSP

In the **Resource Constrained Project Scheduling Problem (RCPSP):**

- We have a **set of n activities**
- **Each activity I has fixed duration di**
- **Activities are connected by end-to-start precedence relations**
- **There are m resources**
- **Each resource k has fixed capacity ck**
- **Each activity requires an amount rik of resource k:** requires means that rik resource units are locked while the activity runs





In the example we can see that a3 should be executed after a1 but before a5 and a6, and a3 has no relations with a4.

PROJECT GRAPH

The network of activities/precedences is called **Project Graph:**

- Typically: **fake start/end activities**
- **Fake = 0 duration, 0 requirements**
- **They can be disregarded in CP models**

Goal:

- **Build a schedule**
  - Assign a start time to all activities
  - Satisfy all constraints
- **Minimize the project completion time (makespan)**

PRACTICAL APPLICATIONS

- **Large scale construction projects**
- **Research/development projects**
- **Production planning**
- **Parallel software optimization**
- **Code optimization** (compile time optimization)

10.1.2 - CP MODEL FOR RCPSP

$$eoh = \sum_{i=0}^{n-1} d_i$$

Using a **start time variable for each activity si ∈ {0..eoh},** we can tackle the problem using CP. **eoh** is a safe **End Of Horizon.** There is always a schedule with **makespan ≤ eoh, unless the resource constraints are trivially infeasible**.
Problem objective Model → **Makespan = project completion time = largest end time**

$$\min z = \max_{i=0..n-1} (s_i + d_i)$$

Precedence constraints Model → **if there is a precedence between activities I and j:** $\boldsymbol{s_i + d_i \leq s_j}$

RESOURCE CONSTRAINT MODEL

If ck=1, activities should not overlap. Formally: **for each pair of activities I,j s.t. rik=rjk=1**

$$\left(s_i + d_i \leq s_j \right) OR \left(s_j + d_j \leq s_i \right)$$

A **resource with unary capacity** is called **disjunctive**.

If ck > 1, finding a good model is difficult. There are some possibilities:

- A sum constraint for each time point
- A sum constraint for each activity start

Both are complicated and lead to **weak propagation:** this is one of the reasons why **MILP is not good for the RCPSP.** A notable exception: **this approach works for SAT based solvers**.
As an alternative, we can use a global constraint.

10.2 - CUMULATIVE CONSTRAINT

In Constraint Programming (CP), the user states the decision variables and the constraints.
We just consider Constraints Satisfaction Problems (CSP). Global constraints are very powerful because they can reason globally on all variables instead of 2 pairwise check. They enforce a global consistency that is stronger than consistency between 2 variables. Moreover, Global Constraints are polynomial, so we have a polynomial time procedure for pruning the all space.

For the scheduling problem we have a specific constraint that is the CUMULATIVE constraint: it takes two lists with the same length (s, d, r) and the capacity. All those vectors are sharing the same resource with the capacity. **CUMULATIVE(s,d,r,c)** represents one single resource and it is the list of actions in competition for that resource.

- **s is a vector of start time variables si**
- **d is a vector of durations di**
- **r is a vector of requirements ri**
- **c is the capacity**

The **durations** and the **requirements** can be **either scalars of variables**.

The cumulative constraint enforces consistency on the following formula, so in practice, the resource capacity is never exceeded**:**

$$\sum_{\substack{i=0..n-1 \\ s_i \leq t < s_i + d_i}} r_i \leq c \qquad \forall t = 0..\max\{s_i + d_i\}$$

If we sum on each activity that is running on a specific pointing time, we are summing activities running concurrently (with si and si+di). We have to ensure that the resource requirement is less than or equal to the capacity of the resource. We have to constrain that the capacity never exceeds.

Feasibility checking is easy when all si are fixed (as usual):

- **Check the resource** usage only at the **activity starts**
- **Rationale**: **resource usage can increase only at the start times**

Unfortunately, **filtering is NP-hard!** If we want to provide N consistency, we have to run an exponential time algorithm and nobody will do this. A consequence of this fact is that **all filtering algorithms are suboptimal**.
In fact, an ALLDIFFERENT constraint represents a polynomial time problem, and also GCC. In this case, when it becomes a constraint, there's a polynomial time for reaching the N consistency.
For the CUMULATIVE, instead, the problem is not in the polynomial class, so there's no a polynomial algorithm that solves the single machine scheduling problem. So, in this case, achieving N consistency for this constraint is impossible: the original problem belongs to NP class. We need a polynomial time algorithm, but we don't reach N consistency: we need to prune the states. That's why we still need to search.

Some filtering algorithms for CUMULATIVE:

- Disjunctive filtering
- Timetable filtering
- Edge-finder
- Not-first/not-last rules
- Timetable edge-finding
- Energetic reasoning

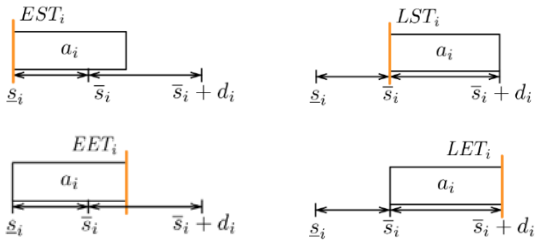
There are **so many filtering algorithms** for CUMULATIVE because **filtering is always incomplete** and because the **CUMULATIVE constraint is very important**.

10.2.1 - TIMETABLE FILTERING

Timetable Filtering is one of the weakest algorithms, but it is also one of the fastest ones. 80% of the times, this is all you need.

Some notable time point for each activity:

- **Earliest Start Time** → ESTi = $\underline{s}$j
- **Earliest End Time** → EETi = $\underline{s}$j + di
- **Latest Start Time** → LSTi = $\overline{s}$ı
- **Latest End Time** → LETi = $\overline{s}$ı + di



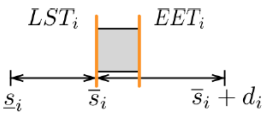
COMPULSORY PART

If we have **LSTi < EETi,** then in the **interval [LSTi, EETi]** the **activity i will certainly be executing**. Therefore, ri units of the resource will be locked. We say that the activity has a **compulsory part**.

MINIMUM USAGE PROFILE

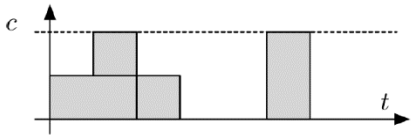
Key idea: rely on a minimum usage profile:

- Min. usage profile = **guaranteed min. consumption per time point**
- **Use the profile to determine bounds for the variables**



Here we don't have a Unary resource, so in this case we need to use the profile. If the compulsory part exceeds the c, we can prune the domain of our variables with an **algorithm called SWEEP**.

Intuition: **SWEEP is very simple, very effective and it is more effective in the lower level of the decision tree**, when we're searching and where compulsory part are coming larger. It's a polynomial time algorithm.



By aggregating all compulsory parts, we get the usage profile, that has the structure of the figure on the left. **For each time instant we have the minimum guaranteed resource usage. For each activity i:**

- **Sweep the timeline** (SWEEP is also the propagator name)
- **Search for a suitable start time**
- **Update the domain of si accordingly**

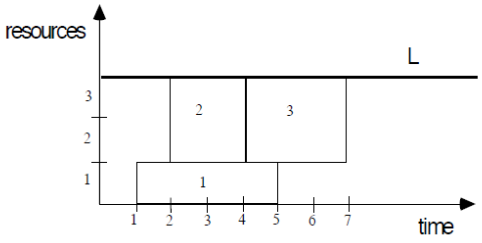
(Example of Timetable Filtering on slides 20-29, pack 13_Scheduling)

Some considerations:

- **Upper bounds on the start variables can be computed similarly**
- The **profile** can be **computed** in **O(n log n)** → Approach: sort and then scan
- **Sweeping** has **complexity O(n)**
- We need to filter n activities

Overall, the algorithm has complexity O(n^2).

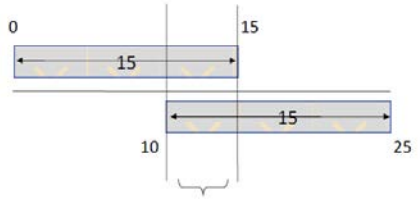
Example: Solved CUMULATIVE ([1,2,4], [4,2,3], [1,2,2], 3).



- The 1st activity starts at time 1, lasts 4 units and requires 1 unit of resources.
- The 2nd activity starts at time 2, lasts 2 units and requires 2 units of resources.
- The 3rd activity starts at time 4, lasts 3 units and requires 2 units of resources.

Here the solution is feasible and we're done. We want to exploit these constraints for pruning the domain, so now we'll see 2 examples of a filtering algorithm that is included in the cumulative constraint.

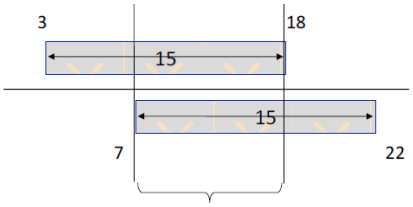
Another example:



Suppose we have an activity whose start time variable Si::[0..10] and the duration is 15.
At some point this domain shrinks because of constraint propagation, so, these bounds can change. **THE DOMAIN CAN ALWAYS DECREASE AND CANNOT NEVER INCREASE. We cannot add values, we can just remove constraints or re-put constraints**.

The **compulsory part** is between 10 an 15. Whenever the activity starts, between 10 and 15 it will be running. This compulsory part is useful because if we have a Unary resource (that cannot be used by other activities) in this interval we know that the resource is occupied for sure by this activity. EST = 0, LST = 10.

Suppose that due to propagation the domain reduces to Si::[3..7] and the duration is 15 (it never changes): now the compulsory part is between 7 an 18. **Whenever the domain reduces, the compulsory part becomes larger.**



Suppose we want another activity Sj:: [0..20], whose duration is 8, to share the same unary resource with Si. How it changes and how do we prune? 0 cannot be used because 0+8 overlaps the compulsory part... and it overlaps the compulsory part till 18! So, we can remove 18 values, from 0 to 17 and in the end we obtain Sj::[18..20].

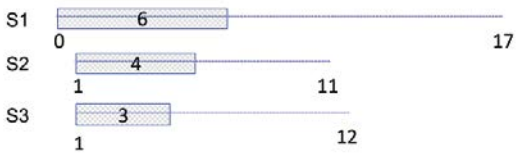
10.2.2 - EDGE FINDER

EDGE FINDER is **another propagator for CUMULATIVE**:

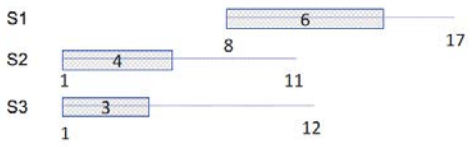
- **Considers pairs (Ω, i)**
  - **Ω = a set of activities**
  - **i = the activities to be filtered**
- **Detects if activity i cannot precede any activity in Ω**
- **Updates D(si)** based on that information
- **Complexity O(kn²)** where k = num. distinct requirements

This is a **very effective approach in some cases** (typically: **tight time windows**), for example when the Time Window = [$\underline{si}$, $\overline{si}$] (in this context).

Example: consider a unary resource and three activities. They cannot be overlapped in time because they share the same unary resource. **S1 can only run after both S2 and S3**, so the minimum starting point for S1 if after 8 time-units. We can have S2 S3 or S3 S2, no matter.
LST for S1 is 11, LST f or S2 is 7, LST for S3 is 9.



Global reasoning: Suppose that S2 or S3 are executed after S1. Then the end time of S2 and S3 are at least 13 (which is not contained in S2 and S3 domain).



10.2.3 - ENERGETIC REASONING

Another propagator for CUMULATIVE is the Energetic Reasoning:

- **Energy = required resource x time**
- **Reason on the required energy in certain time intervals**
- **Detect over usage → fail**
- **Detect potential over usage → prune**

An interesting, but **seldom useful approach**: it subsumes both timetabling at edge, but the complexity is $O(n^3)$ which is too high in many cases.

10.2.4 - TIMETABLE EDGE FINDING

Again, another propagator for the CUMULATIVE.

- **More modern approach**
- **Mixes ideas from timetabling and edge finder**
- **Stronger than Edge Finder**
- Complexity **$O(n^2)$:** convergence is reached in multiple iterations
- **Does not dominate timetabling**

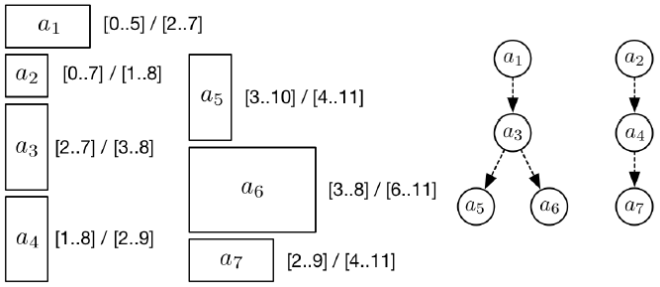
10.3 - SEARCH STRATEGIES FOR SCHEDULING PROBLEMS

In order to search for a solution for the RCPSP, we need to take into account several design decisions, such as which variable to pick, which value to assign and how we backtrack.

10.3.1 - VALUE SELECTION FOR THE RCPSP

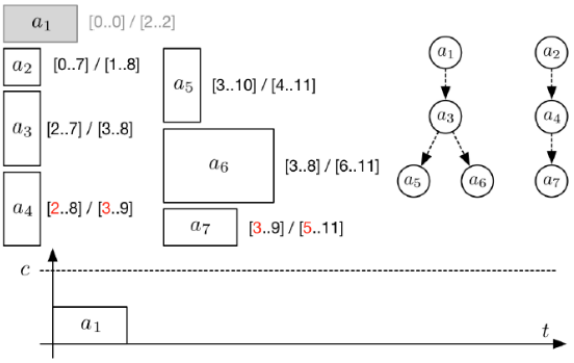
None of these constraints is complete, only the cumulative constraint is complete, so we need to search since we don't have other possibilities. We select a variable and we try to assign it with a start time.
The objective is to minimize the makespan. Increasing a si (others sj untouched) cannot improve the makespan. Consequence → WE SELECT sj.
This is true not only for the RCPSP: many scheduling problems have so-called **regular cost metrics** where regular means that increasing a single cannot improve the cost.

10.3.2 - VARIABLE SELECTION FOR THE RCPSP



a1 can start from 0 to 5 and can end from 2 to 7. 0 = EST, 5 = LST, 2 = EET, LET = 7.

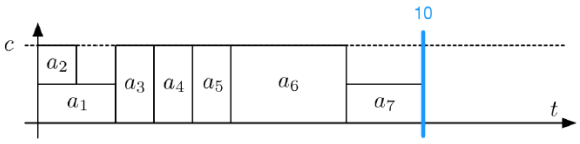
We pick up a variable for branching using the sensible criterion of minimum sj.



We pick a1 because can start at time 0 and it can start with a2. The others activities can only start after 0.
Propagation removes some constraints: the ones in red in fact are changed. The whole duration of the activity becomes a compulsory part.

Then we proceed taking a2, then a3, and so on… After having selected a4, we can select a5 because in case of ties on LET, we look simply at the activity index (a5, a6 and a7 have LET=11).

When all variables are assigned, we have a schedule.



PRB SCHEDULING

This process for constructing an RCPSP solution has name: it corresponds to a **famous heuristic, greedy, solution approach, so-called Priority Rule Based Scheduling (PRB Scheduling)**, that works well in many cases.
For our example the final makespan is 10. **PRB Scheduling may be sub-optimal** because, for example, the previous schedule could be optimized.

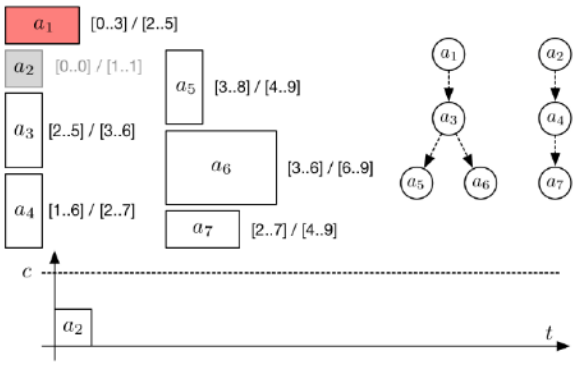
10.3.4 - BACKTRACKING IN CP SEARCH FOR SCHEDULING

As usual, proving optimality requires to search and backtrack. In backtracking if we find a scheduling with a1, this is not a very effective search strategy because starting from 2 instead of 0, it's not so different: we need **another search strategy called Postponed.**

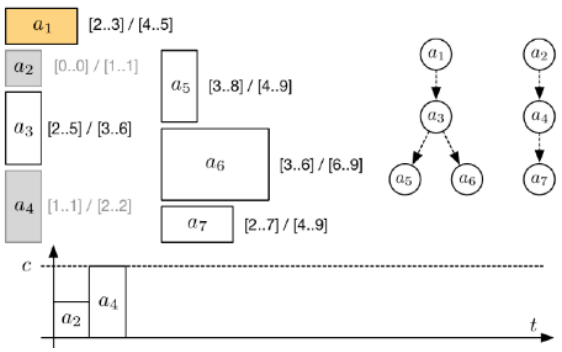
We could post s1≠0, which would ensure complete search, but it is weak, since domains tend to be very large.
A strange alternative: we mark activity as **postponed: a postponed activity cannot be selected for branching until its value changes.**
Rationale: we want to explore a different branching decision:

- We always schedule activities at their sj
- Hence, the scheduling decision changes when sj

So, in the example we did before, a1 is postponed forcing us to pick a2 for scheduling. Then, we proceed as usual, until the value of s1 is updated by propagation.



At this point, a1 becomes eligible for branching.
By proceeding along this branch, we will find the optimal schedule.



10.3.5 - SETTIMES SEARCH STRATEGY

This scheduling strategy is often called SetTimes.
Main ideas:

- Schedule-or-postpone decisions
- Always pick an activity with minimum sj
- Schedule activities at their sj

SetTimes is typically a very effective strategy because it is **based on PRB scheduling**: finds good solutions early. Moreover, it has **effective branching choices** (much better than posting si≠v) and the strategy **implicitly makes ordering decisions.**

Bu, technically, SetTimes is an **incomplete search strategy:**

- **At choice points, we do not partition the search space**
- Either we schedule an activity at sj, or we make it wait

SetTimes works because is based on a dominance rule: The cost function is regular. Hence, there is no point in not scheduling activities at their sj, unless they are delayed by previous activities.
Nonetheless, SetTimes doesn't work with Non-regular cost functions, e.g. costs for starting activities too early.
Moreover, **it doesn't work with Side-constraints that alter the structure of the problem:** many possible cases.

A consequence: other search strategies are becoming more popular, in particular **"domain splitting".**

11 – REIFIED CONSTRAINTS

Example Problem: We have to select warehouses for serving customers.

1. There are N warehouses and M customers
2. Each customer i should be served by a single warehouse
3. Each customer i has a demand di
4. Each warehouse j has a limited capacity ci
5. There is a travel cost tij for serving customer i from warehouse j

Goal: find the assignment that minimizes the cost: we don't really mind when the customers are served, so this is not a scheduling problem, but it is an **assignment problem**.

First model: not easy at all to model.

- $xi \in \{0..n-1\} \quad \forall i = 0..m-1$
- Index = customer
- Value = warehouse

Constraints:

- 2 → trivial
- 4 → assigning customer i to warehouse j consumes some capacity

Second model: **changing the representation** and using an **inverted approach**. Again not so easy.

- Index = warehouse
- Value = customer
- One warehouse can serve multiple customers

Third model: **binary model.** This is the **typical modelling approach in Integer Linear Programming (ILP).**

- **One variable for each possible assignment**
- $xi,j \in \{0,1\} \quad \forall i = 0..m-1 \quad \forall j = 0..n-1$

Constraints:

2 → $\sum_{j=0..n-1} xi,j = 1 \quad \forall i = 0..m-1$

4 → $\sum_{i=0..m-1} di\ xi,j \leq cj \quad \forall j = 0..n-1$

5 → $\min f(x) = \sum_{j=0...n-1} \quad \sum_{i=0..m-1} tij\ xij$

Now we managed to model the problem, but:

- It is not compact
- Constrain propagation may be weak

Fourth model: we can extend our modelling tools.

HP: let's use our classical $xi \in \{0..n-1\}$ variables of the first model.

We need the ability to:

- **Take into account a demand di for the warehouse j if xi=j**
- **Take into account a cost tij for the warehouse j if xi=j**

We could achieve both goals by treating constraints as expressions: z=(xi=j).

Intuitively:

- Z=1 iff the constraint xi=j is satisfied
- Z=0 iff the constraint is not satisfied

This is possible in many constraint solvers.

A REIFIED CONSTRAINT IS AN EXPRESSION THAT CORRESPONDS TO THE FEASIBILITY STATE OF A CONSTRAINT.

Our notation:

- A **constraint** that **appears as a term in an expression** is **reified**
- A **constraint between brackets is reified**

11.1 – CONSISTENCY FOR REIFIED CONSTRAINTS

GAC for reified constraints: The (original) domain D(C) of a reified constraint is always {0,1}.

- **Value 1** in D(c) has a support iff c **can be feasible**
- **Value 0** in D(c) has a support iff c **can be infeasible**

Example: assignment costs using reified constraints:

$$\min f(x) = \sum_{j=0..n-1} \sum_{i=0..m-1} t_{i,j} (x_i = j)$$

11.2 – FILTERING FOR REIFIED CONSTRAINTS

FILTERING RULES

Let (c) be the reification of constraint c:

- Start by filtering c
- Then, If we have a domain wipeout → 1 not ∈ D(c)
- Else, If is resolved → 0 not ∈ D(c)

RESOLVED CONSTRAINT: A constraint is resolved iff $cj = \prod_{xi \in X(cj)} D(xi)$, i.e. if all the possible assignments are feasible.

11.3 – META-CONSTRAINTS

A META-CONSTRAINT IS A CONSTRAINT OVER REIFIED CONSTRAINTS.

Essentially, we gain the power to "materialize" binary variables whenever they are needed.

Meta-constraints are extremely powerful modelling tools; however, **they are not always the best choice**:

- They may lead to **complicated models**
- They may lead to **larger models** (hence, more filtering time)
- They may lead to **weak filtering** (same as binary vars.)

Example: warehouse capacity as a meta-constraint:

$$\sum_{i=0..m-1} d_i (x_i = j) \leq c_j, \quad \forall j = 0..n-1$$